Spyctres User Manual

Stellar spectral analysis with Spyctres 0.5.0a1

Development manual | August 2026

About this manual

This manual describes the Spyctres 0.5.0a1 development state, originally reviewed from the 5 August 2026 source archive and updated against the current working-tree API and wavelength/validation conventions on 8 August 2026. Spyctres is alpha software: the ordinary public workflow and scientific conventions are intended to remain recognizable, but exact function signatures, command-line options, packaging, and some advanced interfaces may still change. For an analysis that must be reproduced exactly, confirm the installed version and use Python help() or --help for the checkout that produced the result.

The manual documents current behaviour. Planned refactor targets and the future modern microlensing-source SED layer are not described as implemented functionality.

Contents

Part I - Getting started

1. What is Spyctres?
2. Installation and configuration
3. Ten-minute quickstart

Part II - Understanding spectra in Spyctres

4. The Spyctres spectrum data model
5. Reading spectra

Part III - The standard spectral-analysis workflow

6. Inspecting a spectrum before fitting
7. Diagnostic windows
8. Masks and problematic regions
9. Local line analysis
10. Asking Spyctres for a fit setup
11. Analysis readiness
12. PHOENIX models in Spyctres
13. The Spyctres forward model
14. Fitting stellar spectra
15. Understanding the result

Part IV - Advanced worked examples

16. Improving a first PHOENIX fit
17. Reviewed Balmer-wing analysis
18. Stability and sensitivity checks
19. Multi-segment and multi-arm spectroscopy
20. Batch analysis

Part V - Validation, quality, and reproducibility

21. Diagnostic plotting
22. Quality assessment
23. External validation and standards
24. Reproducibility and provenance

Part VI - Troubleshooting and reference

26. Troubleshooting
28. Public API reference

Appendix A. Scientific conventions
Appendix B. PHOENIX model-library setup
Appendix F. Reproducibility checklist
Appendix H. Legacy and compatibility functionality

Part I - Getting started

Core purpose, installation, and the first end-to-end workflow.

1. What is Spyctres?

Spyctres is a Python package for the analysis of reduced one-dimensional stellar spectra. Its modern workflow is centred on explicit spectrum metadata, diagnostic inspection, scientifically reviewable fit configurations, and full-spectrum comparison with the PHOENIX HiRes synthetic spectral library. It is designed to support both quick inspection and more disciplined stellar-parameter analyses without hiding important choices about wavelength frame, masks, spectral resolution, continuum treatment, or fitting regions.

1.1 Purpose

The main task Spyctres addresses is straightforward to state but difficult to do safely: given an observed stellar spectrum, construct a physically motivated synthetic spectrum, transform it into the observational space of the data, compare model and observation over selected usable pixels, and infer which model parameters are favoured. Around that fitting problem, Spyctres provides tools for reading spectra, preserving provenance, inspecting important spectral regions, identifying possible limitations, comparing alternative fits, and reporting the result.

The ordinary public workflow is intentionally compact:

```
import Spyctres as sp

spec = sp.read_spectrum("my_spectrum.fits", reader="xshooter_merge1d")
setup = sp.suggest_fit_setup(spec)
print(setup.summary_text())

result = sp.fit_stellar_spectrum(
    spec,
    model="phoenix",
    setup=setup,
)

sp.plot_fit_referee(result)
```

The compactness of this interface should not be mistaken for a claim that every spectrum can safely be fitted with the defaults. In normal scientific use, the spectrum and the proposed setup should be inspected before the fit is interpreted. Later chapters make those checks explicit.

1.2 Current capabilities

In the current alpha, the modern workflow supports the following broad tasks:

- Read reduced stellar spectra through product-aware readers and preserve product-specific provenance.
- Represent single spectra and multi-segment data while keeping wavelength, uncertainty, mask, frame, and resolution information explicit.
- Inspect spectra and identify diagnostic wavelength windows from the actual wavelength coverage.
- Build masks for finite-data failures, archive/product quality flags, telluric regions, diffuse interstellar bands, and user-defined include/exclude regions.
- Perform local line fits as diagnostics for wavelength alignment, continuum adequacy, line shape, and related checks.
- Construct a first-pass FitSetup containing fitting regions, parameter bounds and initial values, search settings, continuum choices, and readiness information.
- Fit one or several spectral segments with PHOENIX models, including radial-velocity shifting, optional constant-Gaussian instrumental broadening, resampling, and per-segment multiplicative Legendre continua.
- Compare alternative fits and run bounded sensitivity tests rather than relying on a single optimizer result.

- Produce structured result objects, quality reports, diagnostic plots, and versioned report data suitable for reproducibility and external review.
- Run quickscan/refinement batch workflows and inspect multi-arm spectra without forcing all arms to share one artificial flux scale.

Selected public entry points

Task	Representative interface
Read a spectrum	`sp.read_spectrum(...)`
Discover readers/instruments	`sp.list_instruments()`, `sp.get_instrument_info(...)`
Plot a spectrum	`sp.plot_spectrum(...)`
Find diagnostic windows	`sp.select_diagnostic_windows(...)`
Build masks	`sp.build_mask(...)`
Suggest a fit configuration	`sp.suggest_fit_setup(...)`
Fit PHOENIX spectra	`sp.fit_stellar_spectrum(..., model="phoenix")`
Inspect fit quality	`result.quality_report()`, `sp.plot_fit_referee(...)`
Compare fits	`sp.compare_fits(...)`

1.3 What Spyctres does not currently claim

The current package is deliberately narrower than a general stellar-atmosphere inference system. In particular, a successful call to the fitter should not be interpreted as evidence that every relevant physical effect has been modelled.

- It does not certify that a result is publication quality. The package can enforce checks, record provenance, and expose warnings; scientific validity still depends on the data, model adequacy, external validation, and the analysis question.
- It does not currently provide a complete modern SED, extinction, and angular-radius inference layer. Older Spyctres functionality exists for compatibility, but it is not part of the modern beginner workflow described here.
- It does not provide a final systematic uncertainty budget simply because an optimizer converges.
- It does not currently solve arbitrary abundance patterns, stellar rotation, macroturbulence, or all regimes in which PHOENIX model physics may be incomplete.
- It does not currently apply wavelength-dependent or non-Gaussian instrumental line-spread functions in the production PHOENIX likelihood. The implemented production broadening is a constant Gaussian LSF when such broadening is requested and scientifically specified.
- It does not silently repair ambiguous wavelength frames, uncertain resolution metadata, arm-to-arm calibration differences, telluric contamination, or suspicious residual structures.
- There is intentionally no general command-line “spyctres fit” interface at present. Scientific fitting is exposed through the Python API while the fit configuration and result semantics continue to mature.

Interpretation rule

A numerical result is not automatically a scientific result. In Spyctres, the ability to run an exploratory calculation is intentionally distinct from the validity of a particular interpretation.

1.4 Design philosophy

Several principles recur throughout the package and this manual.

Inspect broadly, fit narrowly

A spectrum can contain useful classification information outside the regions that should enter a quantitative fit. Spyctres therefore encourages broad inspection of the available wavelength range, followed by a conservative selection of fit regions that have usable pixels, adequate model support, and sufficiently understood instrumental behaviour.

Unknown should remain unknown

When wavelength medium, observer-motion frame, stellar rest-frame status, uncertainty provenance, or spectral resolution are not documented, Spyctres aims to preserve that ambiguity rather than silently invent a value. A guessed metadata field can make a fit look more precise while making the interpretation less defensible.

Diagnostics are not corrections

Telluric features, DIBs, archive bad regions, arm edges, and structured residuals may be annotated or flagged without automatically being removed. The user must be able to see which pixels were excluded and why. This is especially important when an apparently problematic region also contains useful stellar information.

Automated suggestions must be inspectable

The purpose of suggest_fit_setup() is to provide a defensible starting point, not to hide analysis choices. A FitSetup is therefore a first-class object that can be printed, modified, hashed, stored with the result, and reused. If Spyctres chooses windows, bounds, initial guesses, or a search mode, those choices should be visible before a serious fit is interpreted.

Reproducibility is part of the result

Fit outputs are structured objects rather than bare parameter arrays. The reporting layer can preserve the setup hash, PHOENIX grid/cache provenance, uncertainty caveats, mask/readiness information, resolution policy, generated figure paths, and optional input checksums. The aim is that another user can determine not only what parameters were obtained, but how the calculation was configured.

1.5 Modern and legacy functionality

Spyctres still contains historical functionality for synthetic photometry, SED-related calculations, and older workflows. Some of this code remains important for existing analyses, but the modern PHOENIX spectral workflow should be learned independently of those compatibility paths. This manual therefore treats the one-import public API as the normal route and will place legacy functionality in a separate appendix until it is either modernized or formally retained as a compatibility layer.

> **Alpha packaging caveat**
>
> In Spyctres 0.5.0a1, packaging still installs several dependencies associated mainly with the legacy SED/synthetic-photometry path. This is a known packaging issue scheduled for cleanup. The manual describes the software as it exists now and therefore does not pretend that the dependency separation has already happened.

1.6 Recommended learning path

The maintained numbered examples form a scientific learning path rather than a collection of unrelated demonstrations:

- Example 1: read and inspect a clean benchmark spectrum.
- Example 2: understand diagnostic windows, masks, and local line fits.
- Example 3: improve a PHOENIX fit by changing assumptions deliberately, one at a time.
- Example 4A: perform a reviewed Balmer-wing analysis with explicit safeguards.
- Example 4B: test stability against bounded changes in continuum, masks, line selection, and resolution.
- Example 5: scale the same safeguards to batch fitting with checkpointing and quality-gated refinement.
- Example 6: inspect multi-arm spectra broadly while fitting only conservative retained windows.
- Example 7: validate broad released XSL spectra while preserving the library product's frame and global-scaling semantics.
- Example 8: run the PEPSI legacy line-window regression as a compatibility check rather than as the recommended modern workflow.

Parts III and IV develop Examples 1-6 as the main analysis path. Chapter 23 treats Examples 7 and 8 as advanced validation and regression workflows, reflecting their role in testing Spyctres rather than defining the shortest route to a first fit.

2. Installation and configuration

This chapter describes the current development installation. Spyctres 0.5.0a1 should be treated as an alpha package rather than a finished binary distribution. For development and external testing, the recommended approach is to work from the checked-out source tree in an isolated Python environment.

2.1 Create an isolated Python environment

The maintained development environment uses Python 3.12. A Conda environment is convenient but not mandatory. The following commands reproduce the environment style used for current testing:

```
conda create -n spyctres-dev-py312 python=3.12
conda activate spyctres-dev-py312

cd /path/to/Spyctres-dev
python -m pip install -e .
```

The editable installation is useful during alpha testing because changes in the checkout are immediately visible to the environment. For a fixed release or external distribution, a normal wheel installation will eventually be preferable; the current project already tests wheel construction, but packaging is still being refined.

Verify that the active Python imports Spyctres from the expected checkout:

```
python -c "import Spyctres as sp; print(sp.__file__)"
```

2.2 Current dependency status

The current source contains both the modern PHOENIX workflow and historical compatibility functionality. The dependency picture therefore has two conceptual layers, even though packaging has not yet fully separated them.

Group	Current examples	Role
Modern runtime	numpy, scipy, matplotlib, astropy, speclite	Numerical work, spectrum I/O/metadata, plotting, and modern analysis utilities.
Legacy synthetic photometry	pysynphot, synphot, stsynphot; current packaging also constrains setuptools	Historical SED/synthetic-photometry compatibility. These dependencies are planned to be separated from the modern core.
PHOENIX models	Local PHOENIX HiRes data directory	External model data used for PHOENIX interpolation and spectral fitting; not bundled as a small Python dependency.
Testing/development	pytest and development tools	Used for verification rather than ordinary scientific use.

Why this matters

If an installation problem comes from pysynphot, stsynphot, or the current Setuptools constraint, that does not necessarily indicate a problem with the modern PHOENIX fitting algorithm. At this alpha stage, dependency failures should be diagnosed by workflow rather than treated as one undifferentiated installation failure.

2.3 Check the installation before fitting

The preferred compact setup check in the current CLI is `spyctres doctor`. For an installation check that deliberately does not require a configured PHOENIX library, run:

```
spyctres doctor --skip-phoenix
```

For a more detailed development check, the lower-level setup-checker script can also ingest a representative spectrum:

```
python scripts/check_spyctres_setup.py   --spectrum
examples/data/TOO_Gaia21ccu_SCI_SLIT_FLUX_MERGE1D_UVB.fits   --instrument xshooter
```

That detailed script verifies the Python version, dependencies, Spyctres import, console entry point, PHOENIX directory and wavelength grid, template discovery, and ingestion of the bundled spectrum. Use `spyctres doctor --skip-phoenix` for the compact core check when PHOENIX is intentionally absent, and consult `--help` for the lower-level script options in the current checkout.

2.4 Discover the read-only command-line tools

The command-line interface is deliberately limited to setup checks, discovery, and inspection; scientific fitting remains a Python workflow. The current canonical commands are:

```
spyctres doctor --skip-phoenix
spyctres readers
spyctres reader-info xshooter_merge1d

spyctres inspect-spectrum   examples/data/TOO_Gaia21ccu_SCI_SLIT_FLUX_MERGE1D_UVB.fits   --
reader xshooter_merge1d
```

These commands check the core installation, list available readers, describe a specific product reader, and inspect the metadata recovered from a spectrum. The older compatibility forms `spyctres instruments`, `spyctres instrument-info`, and `--instrument` still work, but new documentation should prefer `readers`, `reader-info`, and `--reader`. There is intentionally no general `spyctres fit` command in the current alpha.

2.5 Configure the PHOENIX HiRes library

PHOENIX fitting requires a local installation of the PHOENIX HiRes synthetic spectra. Spyctres does not treat the model library as a small package-data file. The backend expects a directory containing the native PHOENIX wavelength grid and the stellar templates arranged by metallicity.

For the current PHOENIX-ACES-AGSS-COND-2011 HiRes layout, the relevant structure is of the form:

```
<PHOENIX_ROOT>/
  WAVE_PHOENIX-ACES-AGSS-COND-2011.fits
  Z-0.0/
    lte05000-4.00-0.0.PHOENIX-ACES-AGSS-COND-2011-HiRes.fits
    ...
  Z-1.0/
    ...
```

The PHOENIX backend reads the native wavelength grid, scans the installed templates, discovers the actually available Teff, [Fe/H], and log g axes, and constructs interpolation data from the local subset. The installed grid therefore matters: a fit cannot legitimately infer parameters outside the templates that are present and supported.

Spyctres resolves the PHOENIX location through its configuration/path machinery. Because the exact user-facing configuration interface is still part of the alpha packaging work, the safest operational check is not to assume that a path is active simply because it exists. Run the setup checker or PHOENIX diagnostic and confirm that Spyctres reports the intended directory, wavelength grid, and discoverable templates before starting a fit.

Model-library provenance

For reproducible analysis, record which PHOENIX model family and local grid were used. The result/report layer is designed to retain compact model/cache provenance without exposing private absolute paths by default.

2.6 Verify the public Python interface

Once the package imports, the public help registry can be used from Python or IPython. The package is designed so that ordinary users do not need to learn the internal module tree.

```
import Spyctres as sp

print(sp.format_public_api_guide())
print(sp.list_public_function_groups())

help(sp.read_spectrum)
help(sp.fit_stellar_spectrum)
```

In Jupyter or IPython, normal tab completion and question-mark help can also be used, for example sp.fit_stellar_spectrum?.

2.7 A minimal smoke test

Before attempting a PHOENIX fit, run the first numbered example in its lightweight mode:

```
python examples/example1_quickstart.py --no-show
```

This path is intentionally useful even without a configured PHOENIX library. It checks the public import, bundled benchmark data, reader, inspection logic, diagnostic-window selection, and first-pass setup logic without requiring a heavy fit.

2.8 Installation troubleshooting

Symptom	Likely class of problem	First check
import Spyctres fails	Python environment or package installation	Confirm the active interpreter; run python -m pip check; reinstall the editable checkout.
The package imports but PHOENIX fitting fails immediately	PHOENIX path, wavelength file, or template discovery	Run the setup checker and confirm the reported PHOENIX root and wavelength grid.
A reader name is rejected	Wrong reader alias or unsupported product	Run spyctres readers and spyctres reader-info for the matching product reader.
pysynphot/pkg_resources warnings appear	Legacy compatibility dependency	Distinguish the warning from the modern PHOENIX path; this dependency boundary is scheduled for cleanup.
A spectrum loads but fitting is blocked or warned	Scientific readiness, not installation	Inspect metadata, masks, uncertainty provenance, and the readiness report before forcing computation.
The fit runs but the result looks implausible	Model/data mismatch, setup, boundaries, LSF, continuum, uncertainties, or artifacts	Do not treat optimizer success as validation; inspect residuals and follow the later diagnostic chapters.

3. Ten-minute quickstart

The first Spyctres example is deliberately a “happy path.” It uses a bundled spectrum of 18 Sco (HIP 79672) from the Gaia FGK Benchmark Stars material, rather than beginning with the more difficult X-SHOOTER stress case used in later examples. The aim is to learn the sequence of decisions before confronting artifacts, line-core sensitivity, imperfect continuum treatment, or multi-arm complications.

Goal of the quickstart

By the end of this chapter you should know how to ingest a spectrum, inspect it, identify potentially useful wavelength windows, obtain and review a first-pass FitSetup, optionally run a PHOENIX fit, and understand at a high level what the fitter did to produce its answer.

3.1 Start from the maintained example

The most robust way to reproduce the current quickstart is to use the numbered example directly:

```
python examples/example1_quickstart.py --help
python examples/example1_quickstart.py --no-show
```

The corresponding notebook, examples/example1_quickstart.ipynb, is preferable when learning interactively because each output can be inspected before the next step is run. The exact fitting switches may evolve during alpha development, so --help should be treated as the authoritative interface for the script in your checkout.

3.2 Import Spyctres

```
import Spyctres as sp
```

The modern examples use a single public namespace. Internal modules may be reorganized during the structural refactor, but a user should not need to follow those changes to read a spectrum or fit it.

3.3 Read the benchmark spectrum

The quickstart uses the bundled example-data helper to locate the benchmark spectrum and reads it with the dedicated Gaia benchmark ASCII reader. Conceptually, the operation is:

```
path = sp.example_data_path(...)
spec = sp.read_spectrum(path, reader="gbs_v3_ascii")
```

Why the reader matters

A reader is more specific than an instrument label. It describes the actual data product and therefore determines how wavelength, uncertainty, mask, frame, resolution, and provenance information are interpreted. Later chapters document these fields in detail.

3.4 Inspect before fitting

Plot the spectrum before asking an optimizer to do anything:

```
sp.plot_spectrum(spec)
```

At this stage the important questions are observational rather than computational. Is the wavelength coverage what you expected? Are there obvious gaps, edge effects, discontinuities, or pathological flux values? Are uncertainties present? Does the reader report the wavelength medium and frame? Is a resolution descriptor available, assumed, or unknown? A fit should not be used to discover basic problems that can be seen directly in the data.

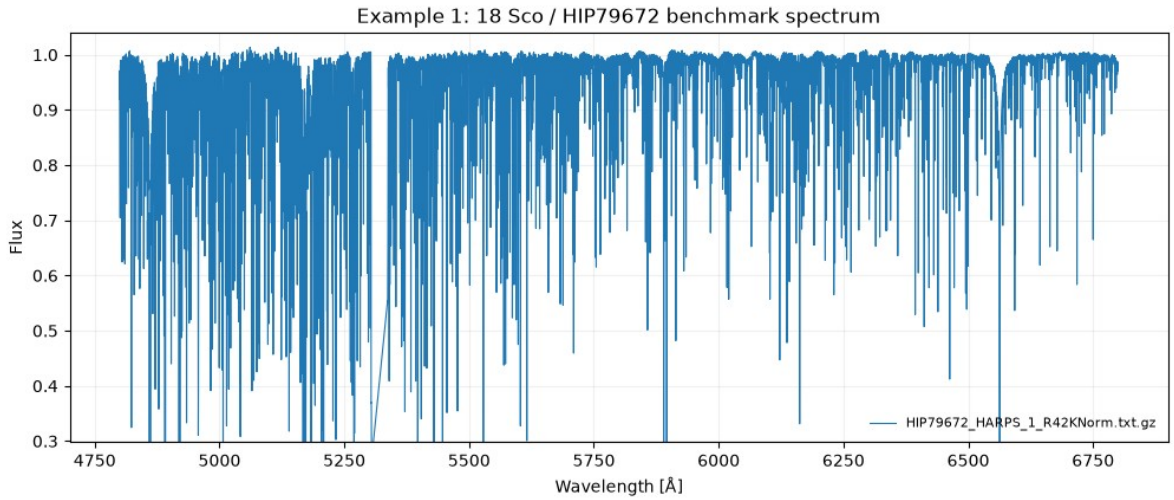

Figure 3.1. Full 18 Sco / HIP79672 benchmark spectrum used in the Spyctres quickstart.

3.5 Ask which diagnostic regions are covered

```
windows = sp.select_diagnostic_windows(spec)
sp.plot_diagnostic_windows(spec, windows)
```

Diagnostic-window selection is coverage driven. Spyctres asks which physically informative regions are actually present in the spectrum rather than assuming that every dataset should be analysed through the same small set of lines. The selected windows are candidates for inspection and later analysis; selection does not mean that every window should automatically be included in a fit.

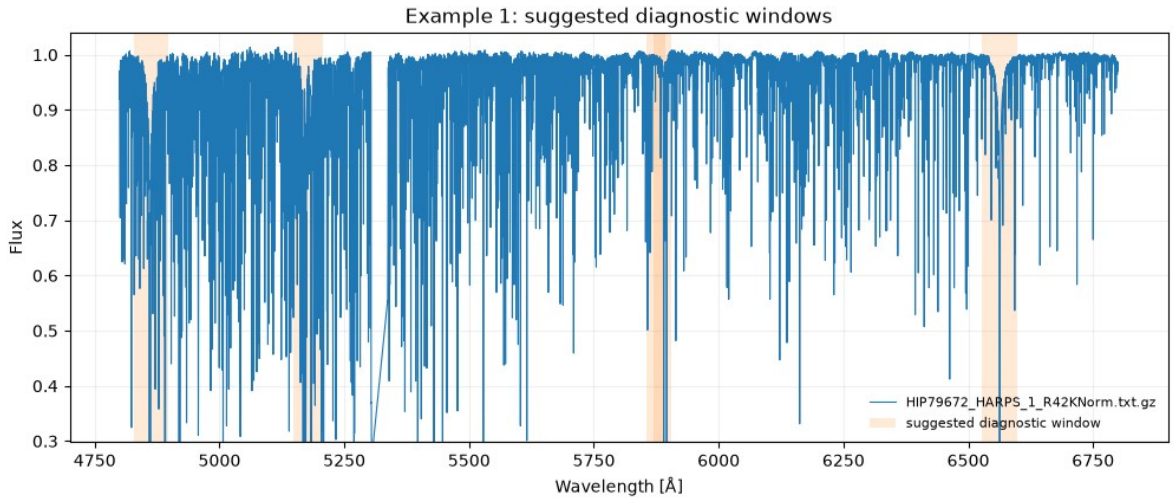

Figure 3.2. Suggested diagnostic windows for the 18 Sco quickstart spectrum. The shaded regions mark windows selected by Spyctres from the available wavelength coverage.

3.6 Request a first-pass fit setup

```
setup = sp.suggest_fit_setup(spec)
print(setup.summary_text())
```

A FitSetup is the bridge between the observed spectrum and the numerical fitter. It records the choices that determine what problem the optimizer is actually solving. Depending on the data and mode, the setup may

contain selected fit regions, parameter bounds, initial guesses, the search budget, continuum settings, resolution assumptions, and readiness interpretation.

The crucial step is the print statement. The examples deliberately show the setup before the fit rather than silently applying automatic choices. If a region, bound, resolution assumption, or continuum degree is scientifically inappropriate, this is where it should be noticed.

Before accepting the setup, ask:

- Do the fit regions contain the features relevant to the intended inference?
- Are important contaminated or unreliable regions excluded or at least flagged?
- Does the resolution assumption correspond to what is actually known about the spectrum?
- Are the parameter bounds broad enough to contain plausible solutions without pretending that unsupported model space exists?
- Is the continuum model flexible enough to absorb calibration shape errors without erasing the stellar information of interest?
- Is the intended use quick classification, parameter estimation, radial velocity, or a more carefully reviewed analysis?

3.7 What a PHOENIX fit actually does

The call fit_stellar_spectrum(..., model="phoenix") is a high-level interface to a multi-stage forward-model and optimization procedure. Even in a quickstart, it is important to understand the main operations rather than treating the fit as a black box.

1. Prepare the observed segments and usable-pixel masks. Only the selected fitting regions and valid pixels contribute to the objective.
2. Load the configured PHOENIX library and determine the model grid that is actually available locally. The current interpolator uses stellar effective temperature, surface gravity, and metallicity as its principal atmospheric axes.
3. Construct candidate stellar models by interpolating within the supported PHOENIX grid. Spyctres is intentionally conservative about model boundaries and should not silently turn extrapolation beyond the supported grid into an apparently valid solution.
4. Apply a stellar radial-velocity shift using the package astronomical sign convention.
5. If instrumental broadening is requested and adequately specified, convolve the model with the current production line-spread model: a constant Gaussian LSF. Wavelength-dependent and non-Gaussian LSFs are not yet part of the production likelihood.
6. Resample the transformed model onto the observed pixel wavelengths.
7. For each segment, solve the coefficients of a multiplicative Legendre continuum model linearly. These continuum coefficients are nuisance parameters: they allow smooth residual flux-shape differences to be handled without asking the nonlinear optimizer to search over them simultaneously with every stellar parameter.
8. Compare model and data over the accepted pixels using the uncertainty information. When the supplied uncertainties are meaningful 1-sigma standard deviations and no additional caveat changes the interpretation, this corresponds to the familiar weighted-residual/chi-square logic.
9. Use an initial radial-velocity scan and, where enabled, a coarse physical initialization to identify promising starting regions.
10. Run bounded multistart local optimization over the genuinely nonlinear parameters. Multiple starting points reduce dependence on a single initial guess, but they do not guarantee that the physical model is adequate.
11. Reconstruct the best model and retain optimizer attempts, fit diagnostics, setup/provenance information, and quality flags in a structured result object.

> **Why the continuum is solved separately**
>
> The multiplicative Legendre continuum is linear in its coefficients once the nonlinear stellar model is specified. Solving those coefficients exactly inside each nonlinear evaluation is a variable-projection strategy: the optimizer spends its effort on the genuinely nonlinear stellar parameters rather than wandering through a large set of continuum coefficients.

Schematically, for observed pixels i in a spectral segment, Spyctres is comparing the data f_i with a transformed PHOENIX model m_i(theta) multiplied by a smooth continuum C_i(a):

$$f_i \sim C_i(a) \times m_i(\theta)$$

Here theta denotes nonlinear stellar/kinematic parameters such as the atmospheric parameters and radial velocity, while a denotes the linear continuum coefficients. The exact objective and available parameterization are documented later in the fitting chapters; the important quickstart idea is that the code transforms the physical model into the observational space before comparing it with the accepted data pixels.

3.8 Run the optional fit

Once PHOENIX is configured and the setup has been inspected, the core call is:

```
result = sp.fit_stellar_spectrum(
    spec,
    model="phoenix",
    setup=setup,
)
```

The returned result is more than a parameter vector. It carries best-fit values together with diagnostics and the information needed to interpret how the fit was performed. This design is intentional: a result that cannot be traced back to its fitting assumptions is difficult to review or reproduce.

3.9 Inspect the fit

```
print(result.quality_report_text())
sp.plot_fit_referee(result)
```

The result object carries the fitted values and diagnostics, while quality_report_text() provides a concise interpretation-oriented summary. Inspect that report together with the data/model/residual plot. A good-looking scalar objective cannot substitute for residual inspection. For additional fields or serialization methods in a particular checkout, use help(result) or help(type(result)).

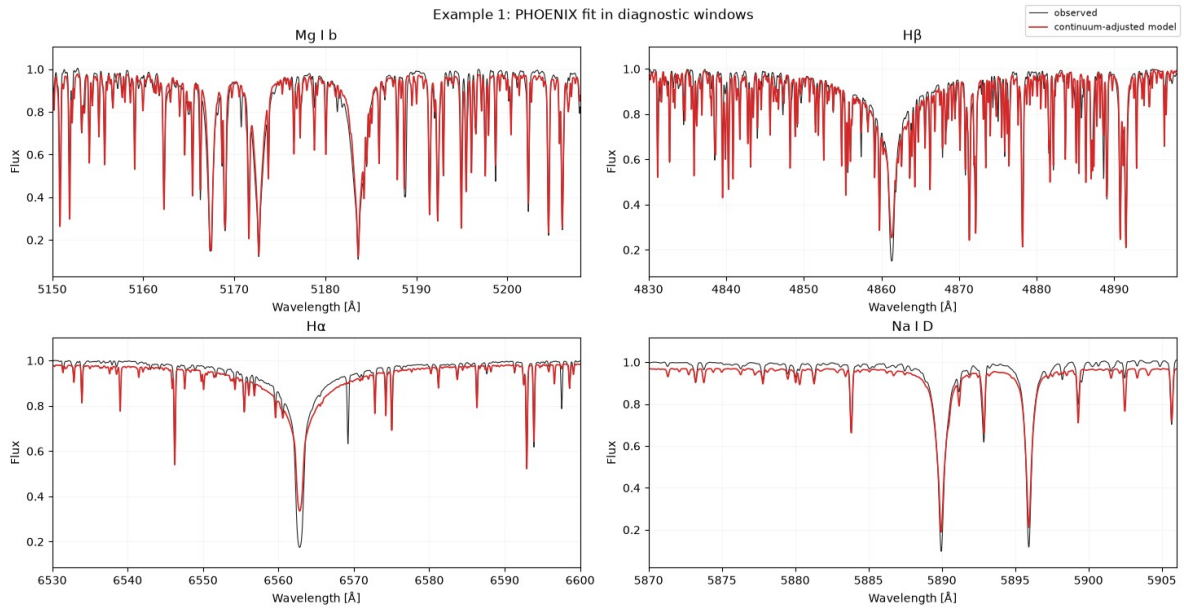

Figure 3.3. Referee-style comparison of the optional PHOENIX fit to the 18 Sco spectrum in selected diagnostic windows. The red curve shows the continuum-adjusted model.

3.10 Interpreting the first result

The quickstart is a first-pass classification/fitting exercise. It is useful for confirming that the complete workflow is operating, and for obtaining a plausible region of stellar-parameter space, but it should not be promoted automatically to a reviewed atmospheric-parameter determination.

In particular, a high reduced chi-square is not by itself a unique diagnosis. Possible contributors include very small formal uncertainties, correlated residuals, continuum mismatch, an inaccurate resolution/LSF assumption, abundance differences relative to the PHOENIX grid, missing model physics, or residual artifacts. Conversely, a reduced chi-square near unity does not prove that the model assumptions are correct; incorrect uncertainties or over-flexible nuisance modelling can also make an objective look deceptively satisfactory.

The first fit should therefore answer a limited set of questions:

- Does the fitted model reproduce the main stellar features at roughly the correct wavelengths and depths?
- Is the radial-velocity alignment plausible?
- Are the inferred atmospheric parameters comfortably inside the supported model region rather than stuck on a boundary?
- Are the largest residuals random-looking, or do they form coherent structures around important lines?
- Do the continuum corrections look like smooth calibration adjustments rather than an attempt to absorb line-shape failures?
- Are any quality flags telling you that the numerical result should not be used for the intended scientific interpretation?

3.11 What you have, and what you do not yet have

After the quickstart you have	You do not yet have
• A verified Spyctres installation and public workflow.	• A guarantee of publication-quality atmospheric parameters.
• A spectrum ingested through an explicit reader.	• A complete systematic uncertainty budget.
• A visual inspection and diagnostic-window context.	• A proof that every spectral region is adequately modelled.
• A reviewable first-pass FitSetup.	• A modern SED/extinction/angular-radius inference.
• An optional PHOENIX fit produced by a defined forward model and bounded optimizer.	• A substitute for sensitivity tests and external validation.
• A structured result and fit diagnostics.	

Part II - Understanding spectra in Spyctres

Spectrum containers, observational metadata, and product-aware readers.

4. The Spyctres spectrum data model

A spectral fit is only as well defined as the data object being fitted. A wavelength array and a flux array are not enough: the same numbers can imply different physics depending on whether wavelengths are air or vacuum, whether they are topocentric or stellar-rest, whether an uncertainty array is a standard deviation or an inverse variance, which pixels are considered usable, and what spectral resolution the model should reproduce. Spyctres therefore uses explicit spectrum containers as the boundary between file-specific ingestion and scientific analysis.

The current input layer exposes two central objects: SpectrumSegment for one one-dimensional spectral segment and SpectrumCollection for a group of segments that should remain distinct but may be analysed jointly. The I/O layer also carries ResolutionDescriptor and InstrumentInfo objects, together with product-specific metadata. Unknown information is deliberately allowed to remain unknown rather than being filled with a convenient guess.

4.1 The canonical idea: preserve the observational state

The purpose of the canonical container is not to erase the differences between instruments. It is to express those differences in a common vocabulary. After a reader has finished, later Spyctres code should be able to ask the same questions of an X-SHOOTER arm, an SDSS spectrum, an XSL release product, or a benchmark-star ASCII file:

- What wavelengths and fluxes are active?
- Which pixels are valid for analysis?
- Are 1-sigma uncertainties available?
- Are wavelengths in air or vacuum?
- What reference frame do the active wavelengths represent?
- Has a stellar-rest correction already been applied, or is a correction value merely recorded as provenance?
- What is known about the instrumental resolution or LSF?
- Is the flux absolute, relative, continuum-normalized, or not reliably classified?
- Which product, reduction, arm, epoch, or archive metadata should follow the data into the analysis?

Why this matters for fitting

The fitter transforms a synthetic spectrum into the observational space of the data. If the observational state is wrong, a numerically successful fit can still be physically wrong. A double wavelength correction, inverted mask, misread inverse variance, or inappropriate resolution can alter the inferred stellar parameters without causing the optimizer to fail.

4.2 SpectrumSegment

SpectrumSegment is the basic one-dimensional spectrum container. A segment represents one wavelength/flux sequence with a single internally consistent interpretation. In the current public design it stores at least the wavelength array, flux array, optional uncertainty array, valid mask, metadata, wavelength medium, wavelength frame, and a name. Reader-specific resolution information is also preserved by the current I/O layer through the resolution/provenance machinery.

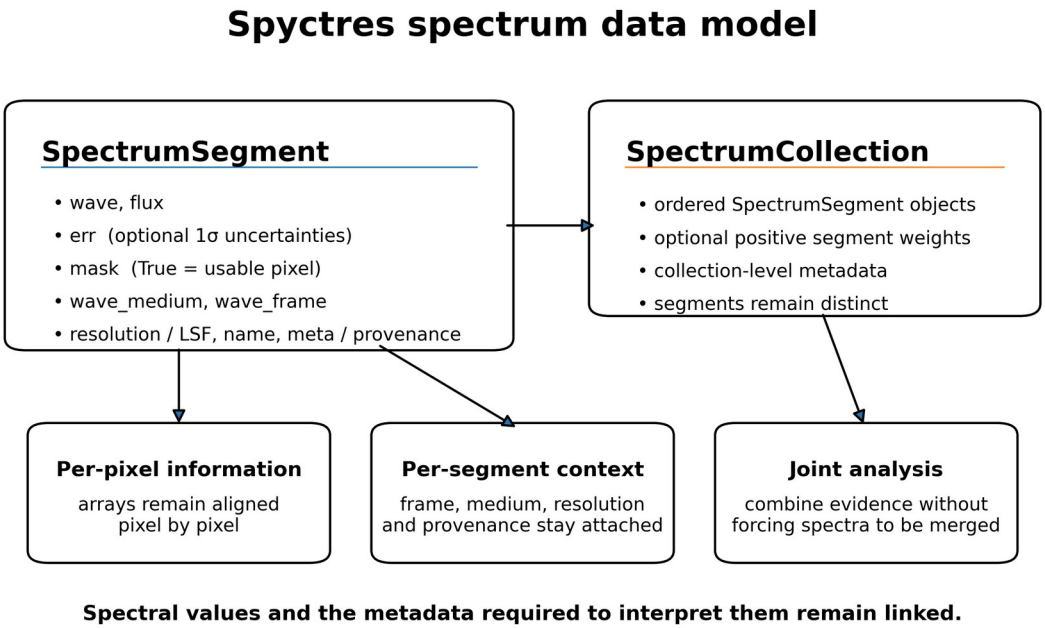

Figure 4.1. Conceptual structure of the Spyctres spectrum containers. A SpectrumSegment keeps the spectral arrays and their scientific context together, while a SpectrumCollection groups ordered segments for joint analysis without forcing them to be concatenated.

Component	Meaning in the current alpha	Scientific consequence
wave	One-dimensional wavelength array, normally expressed in Angstrom by the current readers.	Defines where the data live. Units alone do not specify air/vacuum medium or reference frame.
flux	One-dimensional observed flux or normalized-flux representation.	Its physical interpretation depends on product calibration and normalization state.
err	Optional one-dimensional 1-sigma standard-deviation array.	When meaningful, it sets the relative statistical weight of pixels in a chi^2-like objective.
mask	Boolean valid mask. True means the pixel is usable.	Only accepted pixels should contribute to a fit. This polarity is deliberately explicit.
wave_medium	air, vacuum, or unknown.	Controls line wavelengths and any explicit air/vacuum conversion.
wave_frame	Observer/reference-frame label such as topocentric, heliocentric, barycentric, stellar_rest, or unknown.	Determines what a fitted wavelength shift can legitimately mean.
meta	Product-specific provenance and auxiliary metadata.	Carries information that is important but not yet fully normalized into typed fields in the alpha.
name	Human-readable segment identifier.	Important for multi-arm plots, reports, and per-segment diagnostics.

4.3 Array shape, sorting, and finite-data checks

The wavelength and flux arrays are one-dimensional and have the same length. If an uncertainty array is present, it must also match that length. A mask has the same shape. Readers normally return wavelength-sorted segments; helper operations such as sorting, subsetting, and wavelength windowing preserve the associated flux, uncertainty, and mask arrays together.

When a SpectrumSegment is constructed without an explicit mask, the current container can form a conservative finite-data mask from the arrays: wavelengths and fluxes must be finite, and an existing uncertainty must be finite and positive. This is a basic numerical validity check, not an astrophysical quality

assessment. A finite pixel can still be contaminated by a telluric line, detector artifact, cosmic ray, diffuse interstellar band, or poorly modelled stellar feature.

4.4 The mask convention

Mask polarity

For the spectrum valid mask, True means usable and False means rejected. This is the opposite of the convention used by some masked-array interfaces. In Spyctres documentation, an exclusion mask is therefore named explicitly: for an exclusion mask, True means reject. Do not use the words mask and exclusion mask interchangeably.

This distinction is important because pixel selection enters the objective directly. If V is the set of valid pixels, a simple weighted residual term has the form

```
chi^2 = sum_{i in V} [ (f_i - m_i) / sigma_i ]^2
```

Changing V changes the scientific problem, even if the model, optimizer, and starting parameters are unchanged. Spyctres therefore treats inherited product masks, user exclusions, telluric masks, and warning-only regions as traceable analysis choices rather than invisible preprocessing.

The valid mask on ingestion should answer a narrow question: which pixels are already known to be numerically or product-wise unusable? Later masking chapters add scientifically motivated exclusions. A warning region is not automatically an exclusion region.

4.5 Uncertainties

Where an uncertainty array is present, Spyctres uses the public convention that err contains 1-sigma standard deviations, not variance and not inverse variance. Product-aware readers are responsible for converting native forms into this convention. For example, a reader for a product that stores inverse variance must convert valid positive inverse variances to standard deviations and must not assign finite errors to invalid entries.

The statistical role of sigma_i is straightforward: smaller formal uncertainties give a pixel more weight in a chi^2-like objective. The scientific interpretation is less straightforward. Pipeline errors may omit correlated noise, continuum-calibration uncertainty, LSF uncertainty, or model inadequacy. A very large reduced chi^2 can therefore indicate underestimated errors or missing systematics rather than an optimizer failure. Conversely, rescaling errors to force reduced chi^2 to unity is not automatically justified.

Missing errors

If err is None, the spectrum can still be inspected and used by workflows that define an explicit alternative uncertainty policy, but a standard chi^2 interpretation is unavailable until the weighting model is specified. Spyctres should not invent precision merely because a file lacks errors.

4.6 Wavelength units and wavelength medium

The numerical wavelength unit and the physical wavelength medium are separate concepts. Current Spyctres readers normally place the active wavelength array in Angstrom, while wave_medium records whether those values are air wavelengths, vacuum wavelengths, or unknown. Converting nanometres to Angstrom changes units; converting air to vacuum changes the wavelength definition. Those operations must not be conflated.

A line list and a spectrum should be compared in the same medium. Air/vacuum conversion should be explicit and applied exactly once. If the product documentation does not establish the medium, unknown is scientifically preferable to a guessed label.

4.7 Reference frame and stellar-rest status

The active wavelength frame describes the wavelengths that are actually stored in the segment. Typical current labels include topocentric, heliocentric, barycentric, stellar_rest, and unknown. A correction keyword in a FITS header does not by itself tell you that the active wavelength array still requires that correction. It may instead record a transformation that the reduction pipeline already applied.

This distinction is particularly important for released library products. An official XSL spectrum can already be in the stellar rest frame while retaining barycentric-correction values as processing provenance. Treating such a value as a pending correction and applying it again would shift the spectrum twice.

State	Interpretation	What not to assume
topocentric	Active wavelengths retain the observatory frame.	Do not interpret a fitted shift as the stellar systemic RV without accounting for observer motion.
heliocentric / barycentric	Observer-motion correction has been represented in the active wavelengths according to the product definition.	Do not apply a second correction simply because a header records its value.
stellar_rest	A stellar-rest transformation has already been applied to the active wavelengths.	A later fitted RV is normally a residual alignment parameter, not the star's original absolute RV.
unknown	The current product does not establish the frame safely.	Do not fill the field from a filename convention or an unverified assumption.

4.8 Radial-velocity semantics

Spyctres uses the standard astronomical sign convention in its forward-model wavelength shift: positive stellar RV corresponds to a redshift/recession. What the fitted number means physically depends on the data frame. This is why RV interpretation belongs next to frame metadata rather than being treated as an optimizer-only parameter.

- For a correctly barycentric spectrum, the fitted shift can be interpreted relative to the barycentric wavelength scale, subject to model and calibration systematics.
- For a topocentric spectrum, the fitted shift contains observer-motion information unless an appropriate correction is handled elsewhere.
- For an already stellar-rest spectrum, a non-zero fitted RV generally measures residual misalignment between the released product and the model rather than the original stellar systemic velocity.
- For an unknown frame, Spyctres may be able to align model and data numerically, but a physical RV interpretation should be considered blocked or ambiguous.

4.9 Spectral resolution and the line-spread function

Spectral resolution determines how a high-resolution model must be broadened before comparison with the observed spectrum. For a constant resolving power R,

```
R = lambda / Delta_lambda
```

where Delta_lambda is conventionally associated with the width of an instrumental resolution element. In the current production PHOENIX forward model, instrumental broadening is represented by a constant Gaussian LSF when such broadening is requested and scientifically specified. The model is broadened on a suitable wavelength grid before final resampling onto the observed pixels.

The I/O layer can preserve resolution information even when the fitting engine does not use that exact representation. This is deliberate. For example, SDSS wdisp information can be preserved as LSF provenance and converted to an opt-in tabulated descriptor, while active wavelength-dependent convolution remains disabled by default. Metadata being available is not the same as the fitter having applied it.

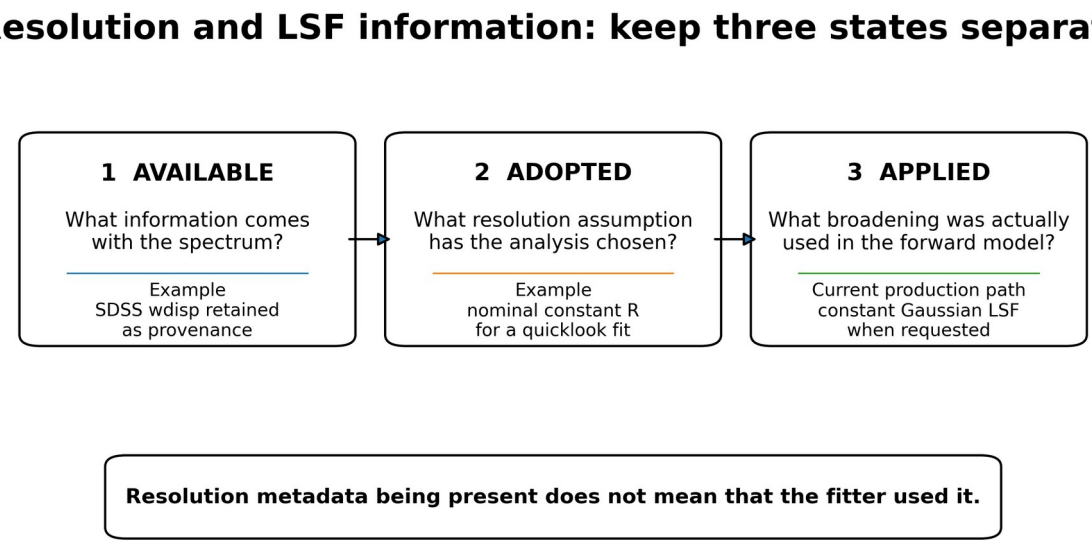

Figure 4.2. Resolution and line-spread-function information in Spyctres. The manual distinguishes information available in the product, the resolution adopted for a particular analysis, and the broadening actually applied in the forward model.

Resolution rule
Always distinguish "known or recorded resolution" from "resolution actually applied in this fit". A nominal slit-based R, a measured empirical LSF, a tabulated wavelength-dependent descriptor, and an assumed quicklook R are not scientifically equivalent.

4.10 Flux calibration and normalization state

The current alpha does not yet enforce one fully typed flux-calibration record across every reader. Some products provide BUNIT or calibration metadata, while normalized libraries may carry their state through product semantics and provenance. The user should therefore determine what the flux array can physically constrain before using it beyond line-shape fitting.

Flux state	What it means scientifically	Typical implication
absolute flux	Flux scale is intended to retain physical calibration, subject to stated slit/aperture and calibration uncertainties.	Potentially usable for flux-sensitive modelling if calibration provenance is adequate.
relative shape	Broad spectral shape is meaningful but overall scale may be arbitrary or uncertain.	Useful for spectral slope only with explicit nuisance calibration treatment.
continuum-normalized	Local or global continuum has been divided out.	Useful for line shapes; cannot by itself constrain an absolute angular scale.
unknown	The product semantics are insufficiently established.	Do not infer physical flux scaling from the numerical range alone.

This distinction will become more important if a future modern SED layer is added, but it already matters in the present spectral fitter. A multiplicative continuum can absorb smooth calibration mismatch, but excessive continuum freedom can also remove astrophysical information. The fitting chapters therefore treat the continuum as part of the model, not as an invisible plotting adjustment.

4.11 The meta dictionary and provenance

The meta dictionary carries product-specific information that does not yet have a single standardized public field in the alpha. Depending on the reader, this may include object name, arm, observation time, exposure time, FITS product category, units, slit information, correction keywords, archive quality information, nominal resolution, and source-file provenance.

Because meta is intentionally product-aware, users should not assume that every key exists for every reader. Code intended to work across instruments should rely on the stable container fields and documented reader information rather than hard-coding a private key discovered in one file. If an analysis depends on a product-specific meta value, record that dependency explicitly in the analysis or report.

4.12 Copying, subsetting, and wavelength windows

Current SpectrumSegment helpers support safe copies, wavelength sorting, subsetting, and wavelength-window extraction while keeping corresponding flux, uncertainty, and valid-mask entries aligned. A typical narrow working segment can be constructed conceptually as follows:

```
hbeta = spec.window(wmin=4800.0, wmax=4920.0, name="Hbeta")
```

Windowing does not make a region scientifically clean. It only changes the support of the segment. Telluric, artifact, line-core, or user exclusions remain separate mask decisions. For model convolution, some workflows use padded support around a narrower fit window so that edge convolution is evaluated with surrounding pixels while chi^2 is accumulated only in the intended inner region.

4.13 SpectrumCollection

SpectrumCollection groups multiple SpectrumSegment objects without pretending they form one homogeneous spectrum. This is the natural representation for multiple echelle orders, selected diagnostic windows, or UVB/VIS/NIR arms when those pieces have different resolution, masks, continua, or calibration histories.

A collection stores the ordered segment list, optional positive per-segment weights, collection-level metadata, and an optional name. Unit weights are the default. Segment order and weighting are scientifically relevant and are treated as invariants in the planned refactor because changing them can alter a joint fit even when every individual segment remains unchanged.

```
collection = sp.SpectrumCollection([uvb, vis, nir])

for seg, weight in zip(collection.segments, collection.weights):
    print(seg.name, weight, seg.wave_medium, seg.wave_frame)
```

4.14 How a collection enters a joint fit

In a joint fit, shared nonlinear stellar parameters can be constrained by several segments while each segment retains its own observational sampling and nuisance treatment. Conceptually, if Q_s is the objective contribution from segment s and w_s is its collection weight, the joint objective has the form

```
Q_total = sum_s  w_s * Q_s
```

The exact normalization and likelihood details are covered later, but two consequences are already important. First, a segment with many high-weight pixels can dominate the result unless the weighting policy is scientifically justified. Second, combining segments does not require them to share an artificial continuum scale: Spyctres can retain separate segment continua and observational metadata.

4.15 Why not concatenate everything?

Concatenation can be useful for visualization or for data that are genuinely homogeneous, but it is not the default scientific answer to multi-arm spectroscopy. If segments disagree in wavelength medium or frame, a merged object can only represent that disagreement by losing specificity, for example by falling back to unknown. More importantly, concatenation can obscure which pixels came from which arm, which resolution applies, and which continuum or calibration nuisance belongs to each piece.

Recommended practice

Keep segments separate whenever their resolution, wavelength/frame state, flux calibration, masks, or nuisance continuum treatment differ. Merge only when the merged representation has a clear scientific meaning.

4.16 A minimal inspection checklist

After reading any spectrum, inspect the object before selecting diagnostic windows or building a fit setup. The following code deliberately uses only generic container attributes:

```
import numpy as np

print(type(spec))
print("name:", spec.name)
print("coverage [A]:", np.nanmin(spec.wave), np.nanmax(spec.wave))
print("pixels:", spec.wave.size)
print("valid pixels:", np.count_nonzero(spec.mask))
print("uncertainties present:", spec.err is not None)
print("wavelength medium:", spec.wave_medium)
print("wavelength frame:", spec.wave_frame)
print("metadata keys:", sorted(spec.meta))
```

Then inspect the reader/resolution information using the public help or reader-specific metadata exposed by your checkout. The goal is not to print every keyword. It is to establish, before fitting, which assumptions are known, inferred, nominal, assumed, or unknown.

5. Reading spectra

Spyctres does not treat file reading as a neutral parsing step. A reader is a scientific adapter for a specific kind of reduced data product. It knows where that product stores wavelengths, fluxes, errors and quality information, and it translates documented product semantics into the common Spyctres container without silently inventing missing information.

5.1 The public entry point

```
spec = sp.read_spectrum(path, reader="xshooter_merge1d")
```

The path tells Spyctres which file to open. The reader tells it what that file means. The output is a SpectrumSegment or SpectrumCollection with the common conventions described in Chapter 4. Reader-specific implementation details are intentionally hidden behind the public interface.

5.2 Why reader= is preferred over instrument=

The maintained examples prefer reader= over instrument= because one instrument can produce several scientifically different products. An X-SHOOTER merged 1D spectrum, a released XSL DR3 spectrum produced from X-SHOOTER observations, and a rawer extracted arm are not interchangeable simply because they originate from the same hardware. They can differ in wavelength frame, flux calibration, masking, arm merging, rescaling, and resolution metadata.

Terminology
Think "which data product am I reading?" rather than only "which telescope or spectrograph made this file?". Instrument aliases remain useful for discovery and compatibility, but reader names are the more precise user-facing description.

5.3 Discovering available readers and instruments

The current public discovery layer lets users inspect supported instrument families rather than guessing strings from source code. From Python:

```
import Spyctres as sp

print(sp.list_instruments())
print(sp.get_instrument_info("xshooter"))
```

The read-only CLI provides the same discovery and file-inspection functions using the current reader-oriented names:

```
spyctres readers
spyctres reader-info xshooter_merge1d
spyctres inspect-spectrum my_spectrum.fits --reader xshooter_merge1d
```

Reader aliases may still evolve during alpha development. Prefer the canonical reader names reported by `spyctres readers` / `spyctres reader-info` and by the installed checkout rather than maintaining your own alias list. The compatibility `instruments` / `instrument-info` forms remain accepted.

5.4 The reader contract

A well-behaved Spyctres reader should do the following:

- Parse the product according to its documented file structure.
- Return one or more canonical spectrum segments with aligned wavelength, flux, uncertainty, and valid-mask arrays.
- Convert native uncertainty representations to the public 1-sigma convention where this is documented and safe.
- Record wavelength medium and active frame when they are known; preserve unknown when they are not.

- Preserve correction provenance without confusing a recorded correction value with a correction that still needs to be applied.
- Preserve product-specific resolution or LSF information and distinguish it from the resolution actually applied by a later fit.
- Preserve quality flags, archive semantics, and useful observation/product metadata.
- Avoid hidden wavelength corrections, arm rescaling, continuum normalization, or astrophysical reinterpretation that is not part of the documented product definition.

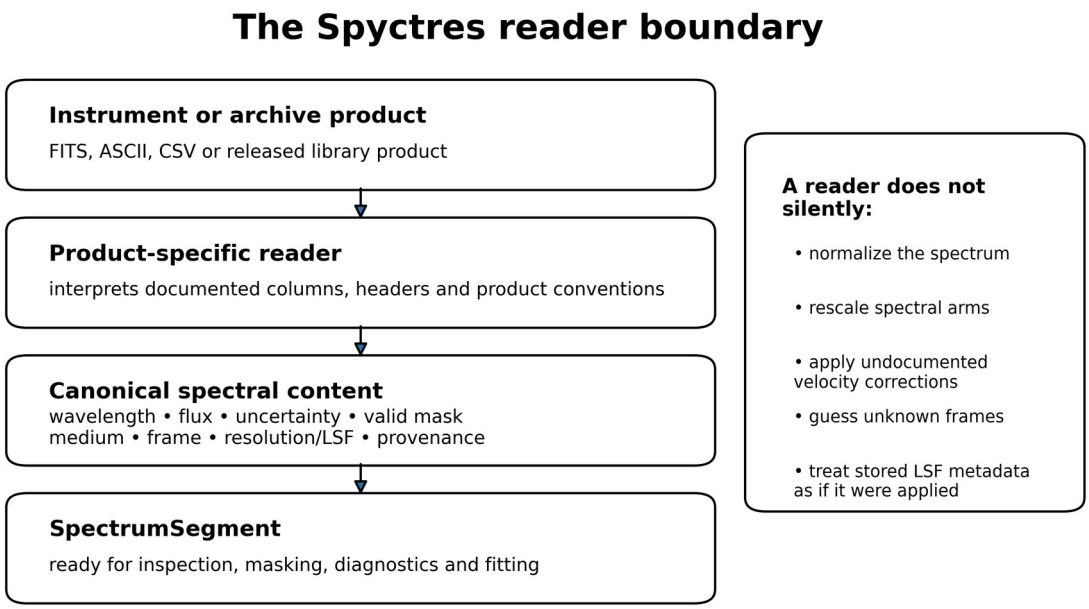

Figure 5.1. The Spyctres reader boundary. A reader interprets documented product semantics and returns canonical spectrum containers, but does not silently normalize, rescale, or guess unknown metadata.

5.5 Current reader families

The reviewed 0.5.0a1 input layer contains reader support for the following product families. The names in the second column are representative public names used by the examples or current registry; use discovery in your checkout for the complete alias list.

Product family	Representative reader/name	Important current semantics
Gaia FGK Benchmark Stars ASCII	gbs_v3_ascii	Reference-star library product used for teaching/validation; not Gaia RVS instrument data; benchmark parameters are not injected as hidden fit priors.
X-SHOOTER merged 1D	xshooter_merge1d / xshooter aliases	Reads reduced arm products, errors and quality information where available; preserves product/frame information and nominal resolution provenance rather than silently correcting wavelengths.
X-shooter Spectral Library (XSL) DR3	xsl / xsl_dr3	Released products are already rest-frame merged/scaled according to XSL semantics; Spyctres does not silently re-align or re-scale them.
PEPSI .nor	pepsi / pepsi_nor	Reads the tabular product and variance where present; resolution may be inferred from documented metadata/fibre information; velocity semantics remain product-specific.
SDSS / SEGUE spec FITS	sdss / sdss_spec	Reads loglam, flux, ivar and and_mask; preserves wdisp/LSF provenance, but wavelength-dependent LSF convolution is not applied by default.

Product family	Representative reader/name	Important current semantics
UVES-POP ASCII	uves_pop	Reads external UVES-POP ASCII spectra with wavelength-unit detection; unknown frame/resolution information remains unknown unless documented.
FLOYDS reduced ASCII/CSV	floyds	Reads simple reduced one-dimensional exports and preserves available comments/metadata; sparse metadata are not filled by guesswork.
Gemini / GMOS ASCII	gemini / gmos	Reads supported reduced ASCII products; product-specific metadata are retained where available and otherwise remain explicit unknowns.

5.6 Gaia FGK Benchmark Stars reader

The bundled quickstart uses a Gaia FGK Benchmark Stars spectrum, such as 18 Sco / HIP79672. These files are benchmark-library spectra, not spectra from the Gaia RVS instrument. Spyctres uses them because they provide clean, well-studied stars for teaching and empirical validation.

The dedicated reader places the library spectrum into the canonical container and preserves its product provenance. Reference stellar parameters associated with the benchmark star are not silently used as priors in the PHOENIX fit. This is an important validation principle: recovery should test the fitter rather than feed the answer back into it.

5.7 X-SHOOTER merged 1D reader

The X-SHOOTER reader handles reduced one-dimensional arm products. In the maintained workflow it interprets the product structure, associates the error and quality arrays where present, reconstructs or reads the wavelength solution, converts the wavelength unit to the canonical representation, and constructs the valid mask from finite data, positive uncertainties and product quality information.

Resolution can be inferred from documented arm/slit information when the product supports that inference. Such a value is nominal instrument provenance, not a measurement of the effective LSF of that exposure. The reader also preserves correction-related header information. A barycentric or heliocentric correction keyword is not automatically applied merely because it exists in the header.

X-SHOOTER caution

For a serious fit, inspect the arm, slit/resolution provenance, wavelength frame, quality mask, uncertainty behaviour, and flux-calibration state. These can matter more than the fact that the file parsed successfully.

5.8 XSL DR3 reader

XSL requires different semantics even though the observations were made with X-SHOOTER. The released DR3 products used by Spyctres are already processed library products: their arms have been merged/scaled according to the XSL release, the released spectra are in the stellar rest frame, and their wavelength scale is air. Spyctres therefore preserves those semantics rather than treating the data as raw X-SHOOTER arms.

PHOENIX HiRes models use a vacuum wavelength grid. When an XSL segment is compared with PHOENIX, Spyctres reconciles the wavelength-medium convention by converting the PHOENIX wavelength grid into the target segment wavelength medium before applying the candidate RV shift, instrumental LSF convolution, and final resampling. Users should therefore not manually convert a normally read XSL spectrum again. An explicit user conversion is appropriate only when the metadata are being deliberately overridden for a file whose wavelengths have themselves been transformed.

A barycentric-correction value present in XSL provenance is evidence about the reduction history, not an instruction for the user to shift the released wavelengths again. Likewise, Spyctres does not automatically re-align or independently rescale XSL arms in an attempt to make a PHOENIX model look better. Hidden reprocessing could double-correct the data or conceal a real calibration/model mismatch.

5.9 PEPSI reader

The PEPSI .nor path reads the tabular wavelength and flux representation and converts a documented variance column to 1-sigma uncertainty where present. Resolution information may be inferred from explicit product metadata or nominal fibre information when this is supported by the product.

Velocity/frame handling is intentionally conservative because PEPSI product semantics can vary by release. Some products are already stellar-rest corrected, while others can use different frame conventions. Spyctres does not infer a correction solely from a filename suffix. The product documentation and metadata, not the convenience of the fitter, determine the frame interpretation.

5.10 SDSS / SEGUE reader

The SDSS spec FITS reader translates the standard spectral columns into Spyctres conventions: logarithmic wavelength information is converted to the active wavelength array, flux is preserved, positive inverse variance is converted to 1-sigma uncertainty, and the archive AND mask contributes to the valid-pixel state.

SDSS wdisp information is scientifically valuable because it contains wavelength-dependent resolution information. The current alpha preserves that information as provenance and can expose an opt-in tabulated descriptor, but it does not apply wavelength-dependent SDSS LSF convolution in the production PHOENIX likelihood by default. A clearly documented constant-R quicklook assumption is preferable to a partially validated wavelength-dependent convolution that gives false precision.

5.11 UVES-POP, FLOYDS, and Gemini/GMOS readers

These readers broaden the range of reduced spectra that can enter the same Spyctres workflow without claiming that every archive product is identical.

- The UVES-POP ASCII reader detects whether the wavelength column is expressed in nm or Angstrom and converts the numerical unit accordingly. Unit conversion does not imply a wavelength-medium or reference-frame correction.
- The FLOYDS reader supports simple reduced ASCII/CSV exports, including basic comment metadata when present. If a product omits uncertainties, frame, or resolution, those quantities should not be fabricated.
- The Gemini/GMOS reader handles the supported reduced ASCII representation and preserves available product information. As with every external reader, the fact that an instrument family is supported does not mean every historical reduction format from that instrument is automatically understood.

5.12 Reading an external spectrum without a dedicated reader

When a product has no dedicated reader, the safest approach is to construct a SpectrumSegment explicitly after you have established the data conventions yourself. For example:

```
seg = sp.SpectrumSegment(
    wave=wave_A,
    flux=flux,
    err=sigma,
    mask=valid,
    wave_medium="vacuum",
    wave_frame="barycentric",
    meta={
        "source": "my reduction",
        "uncertainty_source": "pipeline 1-sigma",
        "notes": "R approximately 45000 from instrument setup",
    },
    name="target_VIS",
)
```

In this case the user, not Spyctres, owns the interpretation. Do not label a spectrum barycentric because that seems likely, or vacuum because a line list is in vacuum. If a convention is not established, use unknown and resolve it before making an interpretation that depends on it.

5.13 Reader overrides: use them as assertions, not repairs

Some reader functions expose options that let an experienced user supply wavelength-medium, frame, extension, unit, or resolution information when the automatic product interpretation is incomplete. Such options should be treated as explicit assertions backed by external knowledge. They should not be used merely to suppress a warning or to force a fit to run.

Bad pattern

"The fit was blocked because the frame was unknown, so I set wave_frame="barycentric"." This converts an uncertainty into an assumption without evidence. The correct sequence is to establish the product convention first, then record the justified value.

5.14 What readers deliberately do not do

- They do not determine the star's spectral type or atmospheric parameters while reading the file.
- They do not apply a barycentric, heliocentric, or stellar-rest correction merely because a related header keyword exists.
- They do not silently convert air to vacuum or vacuum to air unless that conversion is an explicit, documented operation.
- They do not automatically re-scale multi-arm spectra to make their continua agree.
- They do not automatically mask every telluric, DIB, or suspicious stellar feature; those are later scientific masking decisions.
- They do not imply that recorded LSF information has been used by the fitter.
- They do not promote missing or ambiguous metadata to apparently precise values.

5.15 Inspect immediately after reading

The first operation after read_spectrum should normally be inspection, not fitting. For a single segment:

```
spec = sp.read_spectrum(path, reader="xshooter_merge1d")

print(type(spec))
print(spec.name)
print(spec.wave_medium, spec.wave_frame)
print("coverage:", spec.wave.min(), spec.wave.max())
print("valid fraction:", spec.mask.mean())
print("errors:", "present" if spec.err is not None else "missing")
print(spec.meta)

sp.plot_spectrum(spec)
```

For a collection, inspect each segment separately. A collection-level summary cannot reveal an arm with an inverted quality mask, a different frame, or an unexpectedly low valid-pixel fraction.

5.16 Worked reading examples

5.16.1 Benchmark quickstart

```
import Spyctres as sp

path = sp.example_data_path(...)
spec = sp.read_spectrum(path, reader="gbs_v3_ascii")
sp.plot_spectrum(spec)
```

This is a deliberately simple first read. It demonstrates the canonical object without the richer FITS/header semantics of an instrument product.

5.16.2 X-SHOOTER UVB stress case

```
import Spyctres as sp

path = "examples/data/TOO_Gaia21ccu_SCI_SLIT_FLUX_MERGE1D_UVB.fits"
spec = sp.read_spectrum(path, reader="xshooter_merge1d")
```

```
print(spec.wave_medium, spec.wave_frame)
print(spec.meta)
sp.plot_spectrum(spec)
```

This second example is intentionally less clean. It is useful because the reader output includes the observational details that later drive masks, diagnostic-window selection, readiness checks, and resolution assumptions. Parsing the file is only the first step in deciding whether a particular inference is safe.

5.17 Common ingestion problems

Symptom	Likely issue	Recommended response
Reader name is rejected	Wrong alias or unsupported product format.	Use list_instruments(), get_instrument_info(), CLI discovery, and help(read_spectrum).
Spectrum loads but frame is unknown	Product metadata do not establish the active frame.	Consult product/reduction documentation; do not guess. RV interpretation may remain blocked.
Almost all pixels are invalid	Quality-array polarity, uncertainty conversion, NaNs, or wrong extension/product interpretation.	Inspect native arrays and reader metadata before overriding the mask.
No uncertainties are present	The product lacks an error array or reader cannot identify it.	Use workflows that explicitly support missing errors or supply a justified uncertainty model.
Resolution is missing or only nominal	The product does not provide a measured LSF.	Record the limitation; use a justified assumed R only when appropriate and test sensitivity.
Spectrum appears shifted after reading	Possible frame/medium mismatch or product already corrected.	Check active frame, wavelength medium, and correction provenance before applying any shift.
Multi-arm continuum levels disagree	Flux calibration/slit-loss/arm-scaling issue rather than a parsing failure.	Keep arms separate and treat calibration explicitly; do not silently rescale at ingestion.

5.18 A pre-fit ingestion checklist

1. Confirm that the selected reader matches the actual data product, not merely the instrument name.
2. Plot the spectrum and verify the expected wavelength coverage and gross flux behaviour.
3. Confirm that valid-mask polarity and valid-pixel fraction are sensible.
4. Confirm whether 1-sigma uncertainties are present and plausible.
5. Establish wavelength medium and active frame, or leave them explicitly unknown.
6. Determine whether any barycentric/stellar-rest correction has already been applied.
7. Inspect resolution/LSF provenance and distinguish it from what the fitter will actually apply.
8. Establish whether the flux is absolute, relative-shape, continuum-normalized, or unknown.
9. Retain arm/product/observation provenance needed to reproduce the analysis.
10. Only then proceed to diagnostic windows, masks, readiness, and fit setup.

Part III - The standard spectral-analysis workflow

Inspection, diagnostics, masks, FitSetup, PHOENIX fitting, and result interpretation.

6. Inspecting a spectrum before fitting

The first task after ingestion is not parameter estimation. It is to establish that the spectrum behaves as the reader metadata claim and that the data contain enough trustworthy information for the intended analysis. Many expensive fitting failures can be diagnosed more reliably in a simple plot than by tuning an optimizer.

6.1 Start with the entire available spectrum

```
import Spyctres as sp

spec = sp.read_spectrum(path, reader="...")
sp.plot_spectrum(spec)
```

A full-spectrum view answers questions that disappear in a collection of narrow line panels. Check the wavelength extent, broad flux shape, gaps, edges, discontinuities, duplicated regions, sudden resolution changes, and obvious reduction artifacts. For a multi-segment collection, inspect the arms separately before attempting any joint analysis. Arm-to-arm flux offsets are observational information, not something to hide automatically.

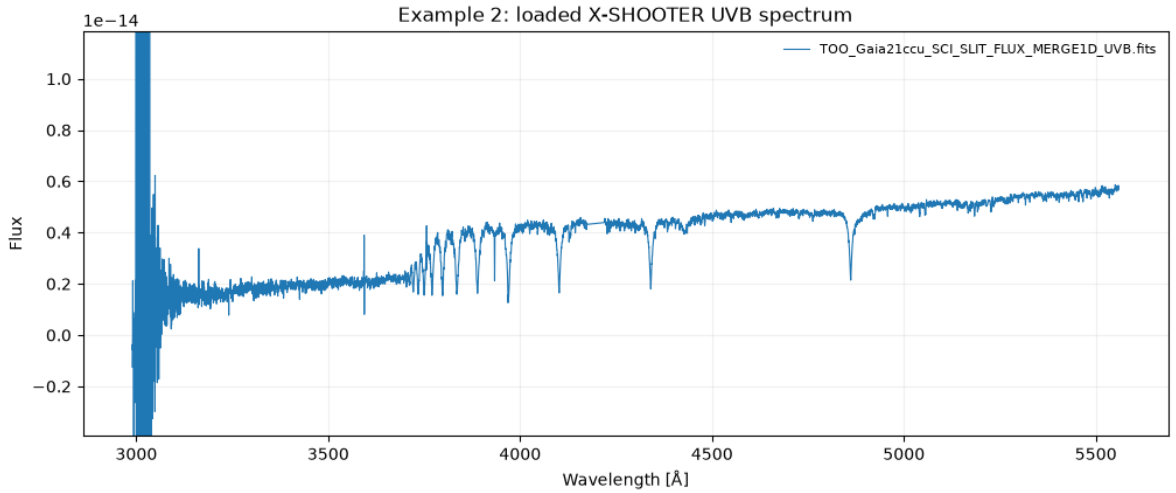

Figure 6.1. Full X-SHOOTER UVB spectrum of Gaia21ccu used in the maintained examples. The unstable blue edge and large excursions near 3000 A are obvious in the full-spectrum view and illustrate why gross data-quality problems should be identified before fitting narrow diagnostic regions.

6.2 Inspect metadata and arrays together

Plot inspection should be paired with a short metadata audit. At minimum, verify the wavelength medium and active frame, the valid-pixel fraction, whether 1-sigma uncertainties are present, and what resolution information is available. The relevant question is not merely "does a header keyword exist?" but "what scientific state do the active arrays represent?"

Check	Why it matters before fitting	Typical warning sign
Coverage	Determines which diagnostic features and parameter leverage are available.	Requested fit windows fall outside the actual spectrum or lie on unreliable edges.
Flux behaviour	Can reveal reduction discontinuities, severe slope errors, failed merging, or saturation.	Large jumps, clipped peaks, negative regions, or arm offsets not explained by the product.
Uncertainties	Set pixel weights when a weighted objective is used.	Zeros, NaNs, implausibly tiny values, or errors unrelated to visible noise.

Check	Why it matters before fitting	Typical warning sign
Valid mask	Defines which pixels are eligible for fitting.	Most pixels rejected unexpectedly, or known bad pixels marked usable.
Medium/frame	Controls line wavelengths and RV interpretation.	Apparent global line shifts caused by air/vacuum or frame confusion.
Resolution/LSF	Determines how narrow model features should appear.	Model/data widths cannot agree even when wavelengths align.
Flux state	Determines what continuum information can legitimately constrain.	A normalized spectrum treated as an absolute-flux SED.

6.3 Inspect the uncertainty model, not only the error array

If formal uncertainties are supplied, compare them with the visible scatter. Very small formal errors can make a visually reasonable fit produce a very large reduced chi-square. Conversely, overly large or correlated errors can make a poor physical model look statistically acceptable. Spyctres records uncertainty caveats because the optimizer cannot determine from the residuals alone whether the quoted sigma values are a complete statistical description.

Interpretation rule

Reduced chi-square is meaningful only relative to the adopted uncertainty model and the assumptions of independent residuals. A numerically large or small value is a diagnostic, not an automatic verdict.

6.4 Inspect resolution before fitting line shapes

A model spectrum normally has substantially higher intrinsic sampling than an observed spectrum. If the instrumental resolution is wrong, line depths and widths can be mismatched in a way that the continuum or atmospheric parameters partially compensate for. Establish whether the current resolution is measured, nominal, assumed, or unknown, and remember the distinction from Chapter 4: resolution information being present does not imply that it will be applied in the likelihood.

6.5 Decide whether to stop

Do not proceed automatically because read_spectrum() succeeded. Stop and resolve the data problem first when the active frame is ambiguous for an RV analysis, the wavelength solution is visibly inconsistent, the uncertainty array is pathological, the majority of a key diagnostic region is invalid, or the resolution assumption would dominate the intended inference. Spyctres is designed to make such limitations visible rather than to convert every input into a fit.

Pre-fit principle

A fit should answer an astrophysical question. It should not be used as a debugging tool for an incompletely understood data product.

7. Diagnostic windows

Most stellar spectra contain more wavelength coverage than should be treated as equally informative. Spyctres therefore separates broad inspection from diagnostic-window selection. A diagnostic window is a region worth looking at because it contains features that may constrain temperature, gravity, metallicity, radial velocity, hot/cool-star classification, or molecular behaviour. It is not automatically a fitting region.

7.1 Select windows from the data that actually exist

```
windows = sp.select_diagnostic_windows(spec)
sp.plot_diagnostic_windows(spec, windows)
```

The selector is coverage driven. It evaluates the loaded SpectrumSegment or SpectrumCollection and asks which catalogued regions have enough overlap and enough usable pixels to be informative. The catalog is intentionally broader than the bundled UVB example and includes optical and near-infrared diagnostics such as Balmer lines, Ca, Mg, Na, He, TiO, CO, Paschen, and Brackett regions where appropriate.

7.2 How window scoring works

The current selector is intentionally inexpensive and does not run PHOENIX. Its score combines several kinds of evidence: catalog priority, actual wavelength overlap, usable-pixel fraction, a robust in-window flux-contrast proxy, and optionally a soft effective-temperature applicability weight. The selector returns both selected and rejected windows, together with enough provenance to explain why a region was kept or rejected.

Term	Purpose	Important limitation
Catalog priority	Encodes that some windows are generally more diagnostically useful than others.	It is not a measurement of information content for this specific star.
Coverage/overlap	Rejects regions that are mostly outside the observed range.	Coverage alone says nothing about contamination or model adequacy.
Usable-pixel fraction	Penalizes regions dominated by invalid pixels.	A high usable fraction can still contain astrophysical contaminants.
Flux contrast	Provides a cheap indication that structure exists in the region.	It is not a line-index or atmospheric-parameter estimate.
Soft Teff applicability	Can prioritize hot-star or cool-star diagnostics when a rough temperature is known.	It is deliberately soft so a wrong initial Teff cannot hide contradictory evidence.

7.3 Diagnostic windows are advisory

A window can be useful for inspection while being unsuitable for a particular fit. For example, Na or Ca lines can carry stellar information while also being affected by interstellar absorption; near-infrared regions can be informative but telluric sensitive; Balmer cores may be physically or reduction sensitive even when the wings are useful. Risk tags belong to the decision process, not to an automatic blanket rejection.

Window != mask != fit region

A diagnostic window says "look here". A mask says "these pixels may or may not be used". A fit region says "this wavelength interval contributes to this particular objective". Keeping these concepts separate prevents convenient plotting choices from becoming hidden scientific assumptions.

7.4 Broad windows versus exact line measurements

The selector uses broad regions because its task is workflow planning. Exact equivalent-width definitions, line indices, or profile measurements belong to the local line-fitting layer. Broad windows are more robust across instruments and resolutions and make it possible to see the surrounding continuum, blends, and contamination before deciding how a line should be analysed.

7.5 Bounded window combinations

Spyctres can describe a small set of useful combinations, such as the top windows, role-balanced sets, single-window sanity checks, or leave-one-window-out variants. The design deliberately avoids an exhaustive search over every subset. Searching all combinations can become expensive and can reward solutions that merely remove difficult pixels. Combination execution therefore remains an explicit analysis choice rather than an automatic default.

8. Masks and problematic regions

Masking is one of the most consequential operations in spectral fitting because it changes which data constrain the model. Spyctres therefore treats masking as an explicit, composable, and reportable step. The aim is not to maximize the number of rejected pixels; it is to remove or downweight information that is invalid for the intended model while keeping enough diagnostic leverage to test that model.

8.1 The common mask workflow

```
reviewed_mask = sp.build_mask(
    spec,
    archive="mask",
    tellurics="warn",
    dibs=False,
)
```

The exact options should be checked with help(sp.build_mask) in the current checkout, but the scientific pattern is stable: start from the reader-provided valid pixels, add explicit user or product exclusions where justified, and keep warning-only features distinct from pixels that are actually rejected.

8.2 Sources of invalid or risky pixels

Source	Typical treatment	Reason to keep provenance
Non-finite data / invalid uncertainty	Reject.	These pixels cannot support a well-defined numerical residual.
Reader quality array / archive bad region	Usually inherit the product rejection.	The reason originates in the product/reduction, not in the fitter.
Telluric absorption	Warn or exclude according to the analysis and available transmission information.	Broad fallback masks and model-based masks are not equivalent.
Diffuse interstellar bands	Warn or exclude when they interfere with a stellar diagnostic.	They are astrophysical but not part of a pure stellar-atmosphere model.
Interstellar Na/Ca	Often diagnostic warning or targeted exclusion.	Stellar and interstellar absorption can overlap.
Cosmic rays / artifacts	Exclude when demonstrated.	An artifact should not pull atmospheric parameters or continuum coefficients.
Line cores	Sometimes explicitly excluded for a wing-based analysis.	Core masking is a scientific modelling choice and must be sensitivity-tested.
User-defined regions	Use only with a documented rationale.	Ad hoc removal can make chi-square improve by deleting model mismatch.

8.3 Warning-only regions

Not every suspicious region should be removed. A warning can be more informative than an exclusion because it allows the user to inspect whether the feature actually affects the result. This is particularly important for DIB-like residuals, known instrument features, or stellar regions with imperfect model physics. Spyctres plotting can show masked regions distinctly so a reviewer can see the model behaviour even where pixels did not contribute to the objective.

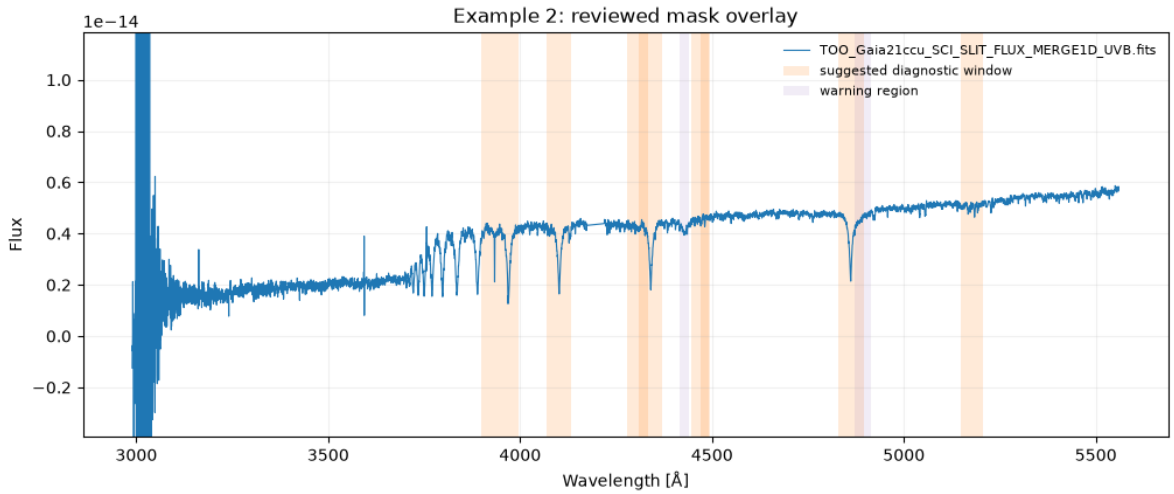

Figure 8.1. Review overlay for the Gaia21ccu X-SHOOTER UVB spectrum. Suggested diagnostic windows are shaded separately from warning regions. The warning shading is advisory and should not be interpreted as automatic pixel exclusion; diagnostic selection, warning annotation, and masking are distinct decisions.

8.4 Compose masks without losing ownership

A robust workflow keeps separate concepts for inherited valid pixels, new exclusion masks, and advisory warnings. If a pixel is already invalid because of the reader quality array, a later manual mask should not rewrite history and describe it as a new user rejection. Provenance should make it possible to answer: who rejected this pixel, at what stage, and for what reason?

8.5 Masking and inference

A mask can alter the balance between temperature-, gravity-, metallicity-, and RV-sensitive information. Removing line cores can stabilize a wing fit but also reduce the information available to discriminate models. Removing an entire difficult window can lower chi-square while making the inferred parameter vector less constrained. This is why later stability tests should vary scientifically uncertain masks rather than treating one arbitrary mask as ground truth.

Bad masking practice

Do not iteratively mask every large residual until the reduced chi-square looks acceptable. That procedure conditions the data on the current model failure and can produce an artificially clean but scientifically biased fit.

9. Local line analysis

Local line fitting is a diagnostic layer. It is useful for checking wavelength alignment, line centers, approximate widths, continuum behaviour, and whether a simple profile is remotely adequate. It is not a substitute for the PHOENIX atmospheric fit, because a Gaussian or similarly simple local profile does not encode the full stellar-atmosphere dependence of line wings, blends, pressure broadening, abundance patterns, or instrumental effects.

9.1 Discover known lines before guessing names

```
print(sp.list_known_lines())
```

The line catalog provides the names and aliases understood by the local fitting interface. Using the catalog rather than hard-coding personal aliases makes examples and scripts reproducible.

9.2 Fit one line, then inspect it

```
# Exact line/config arguments depend on the current checkout.
line_result = sp.fit_line(spec, ...)
sp.plot_line_fit(line_result)
```

A local fit normally extracts a region around the requested feature, applies the active valid mask, estimates or fits a local continuum according to the configured model, fits the requested profile, and returns a structured LineFitResult. The important output is not only the optimizer status but the plotted relation between data, local continuum, model profile, and residuals.

9.3 What a local profile can diagnose

- A systematic centroid offset can reveal wavelength-frame or RV-alignment problems.
- An implausible fitted width can reveal a resolution mismatch, blend, or profile inadequacy.
- A sloping or curved residual baseline can reveal that the local continuum model is inadequate.
- Asymmetric residuals can reveal blends, stellar asymmetry, tellurics, interstellar absorption, or reduction artifacts.
- Different lines giving inconsistent offsets can reveal that a single global velocity correction is not the only problem.

9.4 Why broad Balmer lines are a useful failure example

The maintained Example 2 intentionally shows that a simple local Gaussian can converge on a Balmer line while still fitting it poorly. This is pedagogically important. Numerical convergence means only that the optimizer found a minimum within the chosen profile family. It does not establish that the profile family is physically appropriate.

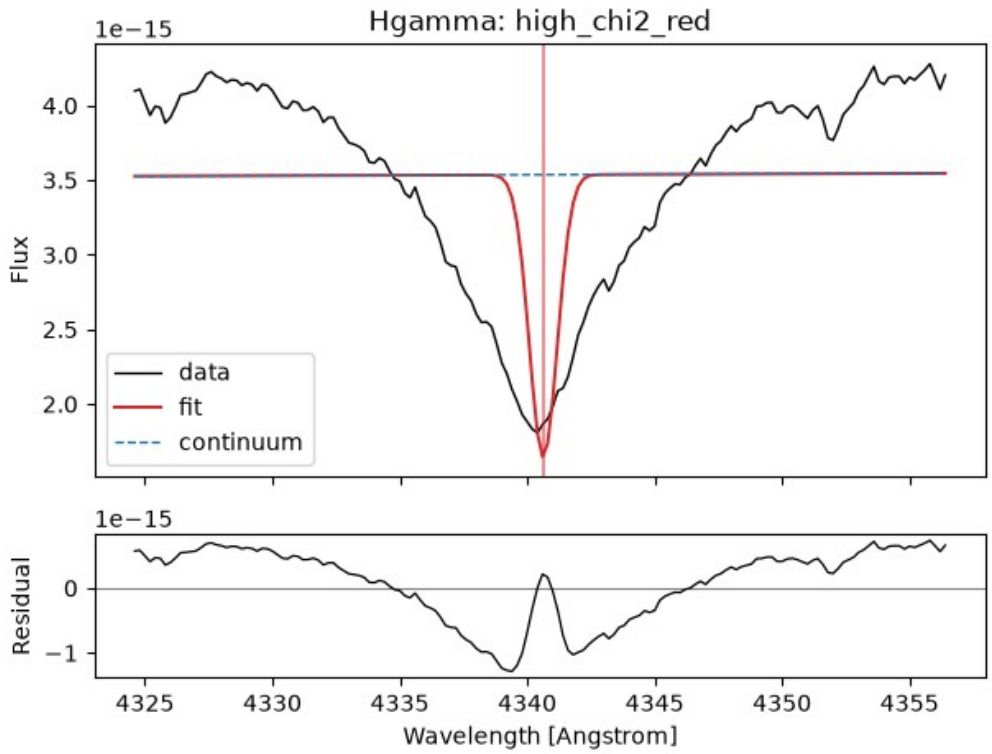

Figure 9.1. Deliberately inadequate local Gaussian fit to Hgamma from Example 2. The optimizer has produced a numerical fit, but the broad systematic residual pattern shows that the simple profile family does not describe the Balmer line. This is a direct example of convergence without physical adequacy.

Do not infer Teff/logg/[Fe/H] from a generic local Gaussian fit

Local line fits are primarily diagnostic unless a specifically validated line-calibration method is being used. Atmospheric parameters in the standard Spyctres workflow come from the physical PHOENIX comparison, not from interpreting generic Gaussian parameters as stellar physics.

9.5 Multiple lines

```
line_results = sp.fit_lines(spec, ...)
comparison = sp.compare_line_fits(line_results)
```

The value of fitting several lines is consistency. A common offset across many lines can support an RV/alignment interpretation; disagreement can indicate blends, continuum issues, line-specific physics, or a wavelength-dependent calibration problem. The comparison should therefore be read as a diagnostic pattern rather than ranked solely by the smallest local chi-square.

10. Asking Spyctres for a fit setup

FitSetup is the object that turns an inspected spectrum into a precisely stated optimization problem. This is the point at which scientific choices become machine-executable: which wavelength regions are fitted, which bounds are allowed, how the search is initialized, what continuum model is used, what resolution assumption is adopted, and what analysis intent the setup is meant to support.

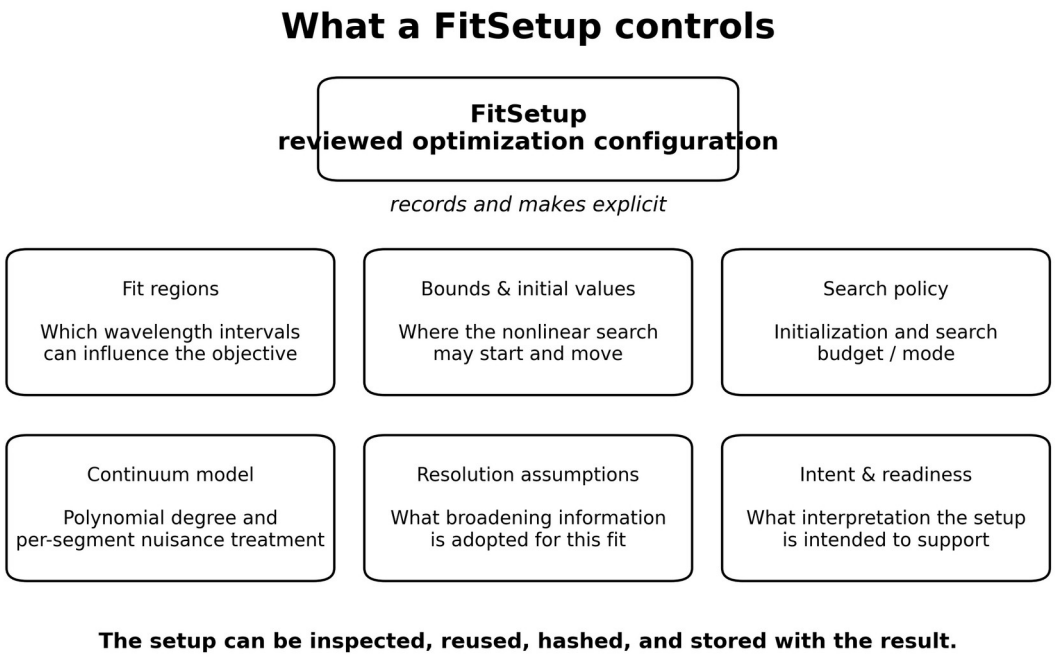

Figure 10.1. Main components controlled or recorded by a FitSetup. The purpose of the object is to make the optimization problem inspectable and reproducible rather than allowing fitting regions, bounds, search policy, continuum choices, resolution assumptions, and scientific intent to remain hidden defaults.

10.1 Generate a first proposal

```
setup = sp.suggest_fit_setup(
    spec,
    mode="standard",
    intent="atmospheric_parameters",
)
print(setup.summary_text())
```

The exact accepted intent strings should be confirmed in the current checkout, but the workflow is deliberate: Spyctres proposes a starting configuration without loading PHOENIX or running the expensive fit. The user sees the proposal before it becomes an optimization.

10.2 What a FitSetup records

Setup element	What it controls	What to review
Fit regions	Which wavelength intervals can contribute to the objective.	Are these regions informative for the intended parameters and clear of known severe problems?
Bounds	Allowed ranges for nonlinear parameters.	Do they reflect supported model space without cutting through plausible solutions?
Initial values	Starting region for the nonlinear search.	Are they plausible but not being treated as hidden truth?
Search mode/budget	How much initialization and multistart work is attempted.	Is a quicklook budget being mistaken for a definitive search?

Setup element	What it controls	What to review
Continuum degree	Flexibility of the per-segment multiplicative Legendre continuum.	Can it absorb calibration shape without erasing line-shape information?
Resolution policy	Resolution/LSF assumption passed toward the fit.	Is it measured, nominal, or assumed; has its uncertainty been considered?
Readiness interpretation	How the spectrum/setup relate to the intended scientific use.	Are blockers being overridden for computation only, or actually resolved?
Stable hash / serialization	Reproducible identity of the reviewed configuration.	Has the exact setup used for the fit been stored with the result?

10.3 Setup modes are search strategies, not quality labels

Names such as quicklook, standard, and reviewed_analysis describe different levels of search and workflow discipline. A stronger search can reduce dependence on a poor initial guess, but it cannot turn bad metadata or an inadequate physical model into a trustworthy result. The most expensive mode is not automatically the most scientifically correct mode.

10.4 Modify one assumption at a time

When a first fit is poor, change the setup deliberately. For example, compare a justified alternative continuum degree, a better supported resolution, a narrower set of reliable regions, or a stronger search mode. Changing several assumptions simultaneously makes it difficult to understand why the result moved. The maintained Example 3 is structured around this one-change-at-a-time logic.

10.5 Stable setup hashes

The setup hash is a reproducibility tool. It allows a result or report to identify the configuration that was actually used rather than merely describing it in prose after the fact. If two fits have different setup hashes, the analysis assumptions differ somewhere and should be compared explicitly.

Do not hide automatic choices

If Spyctres selected windows, bounds, a continuum degree, an RV grid, or a search mode, those choices should be visible before a serious result is interpreted. Automation is useful only when it remains reviewable.

11. Analysis readiness

A spectrum can be perfectly readable and still be unsuitable for a particular inference. Spyctres therefore separates "can this calculation run?" from "is this interpretation supported?" Readiness is intent aware: the same data may be adequate for visual inspection or quick classification while being unsafe for a precise radial velocity or atmospheric-parameter claim.

11.1 Audit before serious fitting

```
audit = sp.analysis_readiness_audit(spec, ...)
print(audit)

# Lower-level/compatibility workflows may also expose:
# sp.audit_spectrum_for_fit(...)
```

The maintained reviewed-analysis examples use analysis-readiness language. The underlying audit machinery reports whether the spectrum is ready for a requested intent, along with blockers, warnings, and interpretations that should be considered invalid. Exact object formatting can evolve during alpha development, so inspect the returned report rather than relying on a single boolean.

11.2 Typical intents

Intent class	Typical tolerance	Examples of issues that may block it
Visual inspection	Highest tolerance. The user is looking at the data rather than inferring a parameter.	Very little beyond unreadable/empty data.
Quicklook classification	Can tolerate some uncertain calibration or artifacts if clearly flagged.	Insufficient diagnostic coverage, catastrophic masking, severe reduction failure.
Atmospheric parameters	Requires defensible regions, uncertainties/masks, model domain, and resolution assumptions.	Artifact-dominated regions, ambiguous preparation, grid-edge/model-domain mismatch.
Radial velocity	Requires trustworthy wavelength/frame semantics and line alignment.	Unknown active frame, uncertain correction history, wavelength-calibration problems.
Reviewed scientific analysis	Requires the strongest documentation and sensitivity checks.	Unresolved artifacts, unexplained structured residuals, untested LSF/mask/continuum assumptions.

11.3 Blockers, warnings, and invalid interpretations

A blocker means the requested interpretation should not proceed as though the problem were resolved. A warning means the analysis may still be useful but the caveat must remain visible. An invalid interpretation is more specific: the computation may produce a number, but that number should not be used for a particular physical claim. This vocabulary prevents a single "ready" flag from being overloaded across very different scientific questions.

11.4 Exploratory overrides

Spyctres can support explicitly labelled exploratory computation even when a readiness gate is not satisfied. In Example 3 this is exposed through setup.allow_exploratory(reason=...). The reason should be recorded. An exploratory override changes permission to compute; it does not erase the blocker or convert the result into a reviewed scientific measurement.

Computation != interpretation

An override can be appropriate when diagnosing why a spectrum fails, comparing setup sensitivity, or testing code behaviour. The resulting parameters must retain the original readiness caveats.

11.5 Readiness is evaluated before and after fitting

Pre-fit readiness asks whether the data and configuration are adequate to attempt the intended inference. Post-fit interpretation must additionally consider residual structure, parameter boundaries, continuum behaviour, optimizer consistency, and model adequacy. A spectrum can pass a pre-fit audit and still produce a scientifically poor fit.

12. PHOENIX models in Spyctres

The modern standard fitter compares the observed spectrum with the PHOENIX HiRes synthetic spectral library. PHOENIX supplies the stellar surface spectrum; Spyctres is responsible for discovering the installed subset, interpolating within supported model space, transforming the model into the observational space, and keeping enough provenance to identify which library/cache state was used.

12.1 The installed library defines the usable model space

Spyctres reads the native PHOENIX wavelength grid and scans the local template tree. It discovers which effective-temperature, surface-gravity, and metallicity values are actually present and constructs interpolation data from that installed subset. The nominal PHOENIX family may be larger than the files installed on a particular machine; the fit can only rely on model space that is locally available and supported by the interpolator.

Axis	Physical meaning	Typical fitting role
Teff	Stellar effective temperature.	Strongly changes continuum/line excitation and Balmer or molecular behaviour depending on regime.
log g	Surface gravity.	Changes pressure-sensitive line profiles, ionization balance, and broad wings in appropriate regimes.
[Fe/H]	Global metallicity coordinate of the PHOENIX grid used by the current fitter.	Changes line blanketing and many metal-line strengths; not equivalent to fitting arbitrary individual abundances.

12.2 Interpolation is not extrapolation

Candidate spectra are generated by interpolation within the supported PHOENIX grid. A result near an interpolation or installed-grid boundary needs special scrutiny because the likelihood may prefer parameter values beyond the region the model can represent. Spyctres should not silently extrapolate an atmosphere and then report the extrapolated point as an ordinary fit.

Grid-edge result

A parameter at or near a grid boundary is a diagnostic. It can mean the star lies outside the supported model domain, the chosen wavelength regions prefer an unsupported solution, or another modelling error is pushing the fit toward an edge. It is not evidence that the boundary value itself is correct.

12.3 Cache and provenance

Loading and interpolating a large PHOENIX library is expensive, so Spyctres uses caching. Cache behaviour is part of reproducibility: a result report can carry a compact model/cache manifest or hash without exposing local absolute template paths. If the installed template axes or cache schema change, the analysis provenance should make that detectable.

12.4 Physical limitations of the atmosphere model

The PHOENIX grid is a physical atmosphere model, but it is not a complete model of every observed star. Peculiar abundance patterns, rapid rotation, strong activity, hot-star regimes outside the supported grid/physics, carbon-rich spectra, or other mismatches can produce structured residuals that an optimizer cannot cure. A low objective value within an inadequate model family does not validate the family.

12.5 PHOENIX flux and the current standard workflow

In the standard normalized/shape-flexible spectral workflow, a per-segment multiplicative continuum is allowed to absorb smooth flux-shape differences. Therefore the fit is primarily using spectral morphology rather than treating the raw absolute PHOENIX surface flux as an angular-radius measurement. Absolute flux scaling, extinction, and angular radius belong to the planned modern SED layer, not to the current standard PHOENIX fitting path.

13. The Spyctres forward model

A stellar-atmosphere spectrum cannot be compared directly with observed pixels. The synthetic spectrum must first be placed into the same kinematic and instrumental space as the data. The standard PHOENIX forward model performs these operations in a controlled order before the objective is evaluated.

13.1 Overview

1. Evaluate or interpolate the PHOENIX stellar spectrum at the candidate atmospheric parameters.
2. Apply the candidate stellar radial-velocity shift using the package astronomical sign convention.
3. Apply instrumental broadening when a scientifically specified constant-Gaussian LSF/resolution is requested.
4. Resample the transformed model onto the wavelength grid of each observed segment.
5. Inside the fitting objective, multiply each segment by its best-fitting smooth Legendre continuum before computing residuals on accepted pixels.

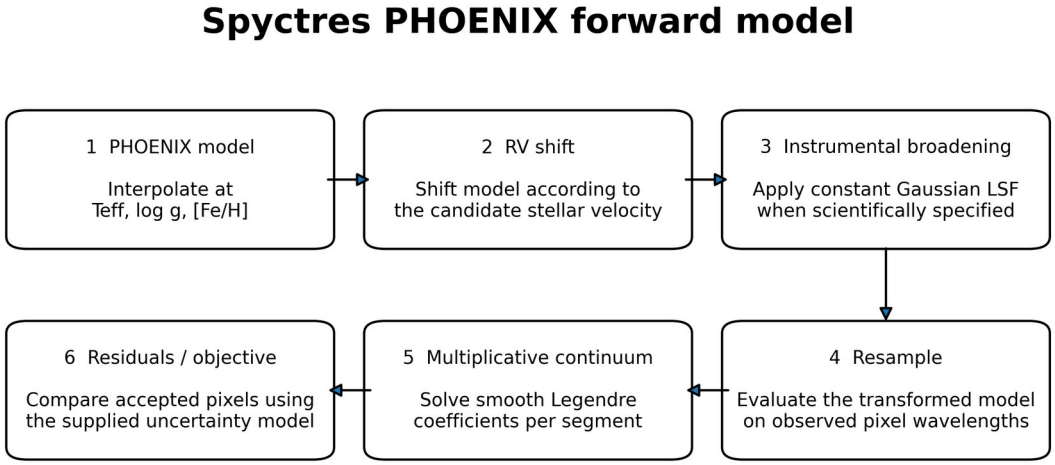

Figure 13.1. Order of operations in the standard PHOENIX forward model. The high-resolution atmosphere model is shifted, broadened when specified, and resampled onto the observed pixels before the segment-specific multiplicative continuum is solved and the accepted-pixel residuals are evaluated.

13.2 Atmospheric model evaluation

Let theta_atm = (Teff, log g, [Fe/H]) denote the atmospheric coordinates. PHOENIX provides a high-resolution model F_PHX(lambda; theta_atm). Interpolation is performed only within the supported installed grid. At this stage the spectrum is still a theoretical high-resolution stellar model, not yet a model of the instrument.

13.3 Radial-velocity shift

The model wavelength scale is shifted according to a stellar radial velocity v_r. Spyctres uses the standard astronomical sign convention: a positive radial velocity corresponds to recession/redshift. For velocities small compared with the speed of light, the convention is approximately λ_shifted = λ × (1 + rv_kms/c), so positive rv_kms moves model features to longer wavelengths. The physical interpretation of v_r depends on the active frame of the observed spectrum. If the observation is already in the stellar-rest frame, a fitted shift is a residual alignment parameter rather than automatically the star's absolute radial velocity.

Frame rule

Do not interpret the fitted velocity independently of Chapter 4 frame metadata and the reader correction history. The same numerical shift can have different physical meanings for topocentric, barycentric, or stellar-rest input wavelengths.

13.4 Instrumental broadening

The production broadening path currently uses a constant Gaussian line-spread function. The implementation broadens the model on a suitable logarithmic wavelength grid and warns when the requested kernel is undersampled. Wavelength-dependent and non-Gaussian LSFs may be preserved as metadata or future/experimental representations, but they are not the default production likelihood path in this alpha.

For an approximately constant resolving power R, the characteristic instrumental width scales with wavelength. The key scientific requirement is that the broadening applied to the model corresponds to the analysis assumption actually being made, not merely to some resolution number found in the file header.

13.5 Resampling to observed pixels

After kinematic shifting and broadening, the model is resampled onto the observed wavelength coordinates. The objective is then evaluated at the data pixels rather than by forcing the observed spectrum onto the PHOENIX native grid. This keeps the observational sampling and mask aligned with the data.

13.6 Multiplicative Legendre continuum

Real spectra often retain smooth differences from the theoretical continuum because of flux calibration, blaze residuals, slit losses, normalization choices, reddening over a limited range, or imperfect arm scaling. The standard fitter accounts for these differences with a smooth multiplicative Legendre polynomial for each segment.

```
For segment s and pixel i:

model_s,i(theta, a_s) = C_s(x_i; a_s) * M_s,i(theta)

C_s(x; a_s) = sum_{k=0}^{K} a_s,k P_k(x)
```

M_s,i(theta) is the shifted, broadened, resampled PHOENIX model; P_k are Legendre basis functions on a scaled coordinate; and a_s,k are segment-specific continuum coefficients. The continuum is multiplicative so it changes the smooth spectral shape without adding an arbitrary emission component to the line spectrum.

13.7 Why continuum flexibility must be limited

A continuum that is too rigid can force the atmospheric parameters to compensate for calibration shape. A continuum that is too flexible can absorb broad stellar features and reduce the information that should constrain Teff or log g. The appropriate degree is therefore a scientific nuisance-model choice and should be included in sensitivity tests for serious analyses.

Forward-model principle

The physical stellar model is transformed into the observational space before comparison. Spyctres should not "correct the data until they look like the model".

14. Fitting stellar spectra

This chapter describes what fit_stellar_spectrum(..., model="phoenix") actually optimizes. The modern standard path is a deterministic, bounded, multistart optimization around a PHOENIX forward model. It is not a black-box lookup and it is not, by itself, a posterior sampler.

14.1 The public call

```
result = sp.fit_stellar_spectrum(
    spec,
    model="phoenix",
    setup=setup,
)
```

The reviewed FitSetup supplies the fitting regions, bounds, initialization/search policy, continuum choices, and relevant resolution assumptions. Passing the setup object rather than reconstructing keyword arguments later is important because the exact reviewed configuration is embedded in the result/report provenance.

14.2 Step 1: construct the fitting data set

For each segment, the fitter forms the pixels that are both inside the selected fit regions and valid under the active mask. Segment ordering and any explicit per-segment weights are preserved. Empty or fully masked segments cannot contribute useful information and should be treated explicitly rather than silently assigned artificial data.

The statistical residual for accepted pixel i of segment s can be written schematically as

```
r_s,i(theta) = [ f_s,i - C_s,i(a_s*) M_s,i(theta) ] / sigma_s,i
```

where a_s* denotes the continuum coefficients that minimize the segment residual for the current nonlinear stellar parameters theta. When uncertainties are absent, modified, or subject to an error floor, the result/report must retain the corresponding caveat because the numerical objective no longer has the simple interpretation of a fully specified independent Gaussian likelihood.

14.3 Step 2: solve the continuum linearly

For fixed nonlinear stellar parameters, the multiplicative Legendre continuum is linear in its coefficients. Spyctres therefore solves those coefficients directly inside each nonlinear objective evaluation instead of adding every continuum coefficient to the nonlinear optimizer. This is a variable-projection strategy: analytically or linearly eliminate nuisance parameters that can be solved efficiently, then optimize only the genuinely nonlinear part of the problem.

```
theta  = nonlinear parameters (atmosphere, RV, ...)
a_s    = linear continuum coefficients for segment s

For each trial theta:
    1. build M_s(theta)
    2. solve best a_s by weighted linear least squares
    3. compute residuals using C_s(a_s) * M_s(theta)
    4. return the combined objective
```

This has two advantages. It reduces the dimension of the nonlinear search, and it ensures that every nonlinear trial is compared using the best continuum allowed by the specified continuum model. It does not remove the need to choose a scientifically sensible continuum degree.

14.4 Step 3: combine segments and weights

For several spectral segments, the objective combines the accepted residuals while retaining per-segment continua and the specified segment weighting. This allows UVB/VIS/NIR arms or separate windows to constrain one atmospheric solution without pretending that they share the same raw continuum normalization. The combined objective must still be interpreted carefully if uncertainty scales differ strongly between segments.

14.5 Step 4: radial-velocity initialization

A poor RV starting point can place model and data lines far enough apart that a local optimizer follows the wrong residual landscape. Spyctres therefore performs an initial radial-velocity grid scan to identify promising alignment regions before the final nonlinear search. This scan is an initializer, not automatically the final RV inference.

14.6 Step 5: optional coarse physical initialization

Where enabled by the selected search mode, the fitter can perform a coarse search over atmospheric parameter space to find plausible starting regions. The purpose is to reduce dependence on an arbitrary seed. It is not an exhaustive evaluation of every PHOENIX model, and the density of the initializer should not be confused with the eventual parameter precision.

14.7 Step 6: bounded multistart local optimization

Spyctres then launches local optimizations from several starting points within explicit bounds. Multistart fitting is a pragmatic defense against local minima: if several plausible seeds converge to the same basin, confidence in the numerical minimum increases. However, all starts share the same physical model, masks, continuum form, resolution assumption, and wavelength regions. Agreement between starts therefore tests optimizer robustness, not model validity.

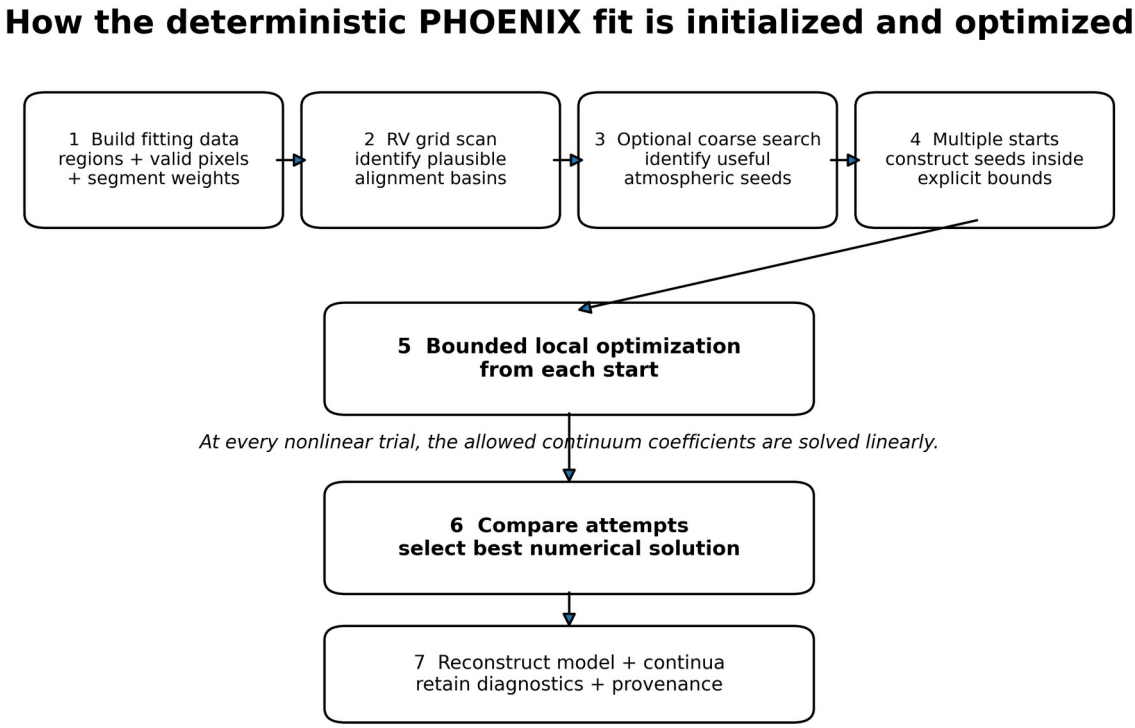

Figure 14.1. Deterministic initialization and optimization sequence used by the standard PHOENIX fitter. The RV scan and optional coarse atmospheric search provide starting information; bounded local optimization is then repeated from multiple starts. Continuum coefficients are solved linearly inside each nonlinear trial before the optimizer attempts are compared.

What multistart does not prove

Several optimizer starts converging to the same solution does not prove that PHOENIX is adequate, that uncertainties are correct, that the LSF is right, or that the selected windows are unbiased. Those are separate validation questions.

14.8 Parameter bounds and model boundaries

A fit can encounter several conceptually different boundaries: the optimizer bounds supplied by the setup, the local search box, the interpolation cube, the installed PHOENIX subgrid, the full atmosphere-grid boundary, or a broader unsupported physical regime. These should not be collapsed into one generic "boundary" idea when interpreting a result. A solution at a bound is a reason to investigate what is limiting the search.

14.9 The objective and reduced chi-square

With reliable independent 1-sigma errors and a standard weighted residual objective, the summed squared residuals have the familiar chi-square interpretation. A reduced statistic divides by an effective number of degrees of freedom. In real spectra, however, continuum nuisance parameters, correlated residuals, model mismatch, error floors, masked pixels, and imperfect uncertainty calibration all complicate the textbook interpretation. Spyctres therefore treats chi-square as an important diagnostic but not a sufficient quality criterion.

14.10 Convergence

Optimizer success means that the numerical algorithm satisfied its convergence criterion for at least one attempted start. It does not mean the residuals are structureless, that parameters are interior to the model domain, or that the physical interpretation is valid. Always inspect the reconstruction and quality report after convergence.

14.11 Reconstruct the best model

At the selected solution, Spyctres reconstructs the transformed PHOENIX model and the fitted continua for each segment, retains the optimizer attempts and diagnostics, and returns a structured PhoenixFitResult rather than a bare parameter vector. This reconstruction is what should be plotted and reviewed.

14.12 What the standard fitter does not yet do

- It does not automatically sample a full Bayesian posterior in the standard public workflow.
- It does not make a constant-Gaussian LSF scientifically adequate for every instrument.
- It does not fit arbitrary abundance patterns, stellar rotation, macroturbulence, or every missing physical effect.
- It does not turn a formal covariance matrix into a complete systematic uncertainty budget.
- It does not replace stability tests against continuum degree, masks, line selection, resolution, and other plausible analysis choices.

15. Understanding the result

A Spyctres fit result is deliberately more than a best-fitting parameter vector. It is a record of the numerical solution, the model reconstruction, the fitting assumptions, the diagnostics, and the limits on interpretation. Reading the result therefore has two stages: first determine what the optimizer found, then determine whether that solution is scientifically credible for the question you asked.

15.1 Start with the quality report and plot

```
print(result.quality_report_text())
sp.plot_fit_referee(result)
```

The quality report summarizes interpretation-relevant diagnostics. Use it together with diagnostic plots: plot_fit_referee() is intended to expose data, model, and residual structure, while selected-window model plots provide a compact feature-by-feature comparison. A scalar objective should never substitute for inspecting where the model succeeds and fails.

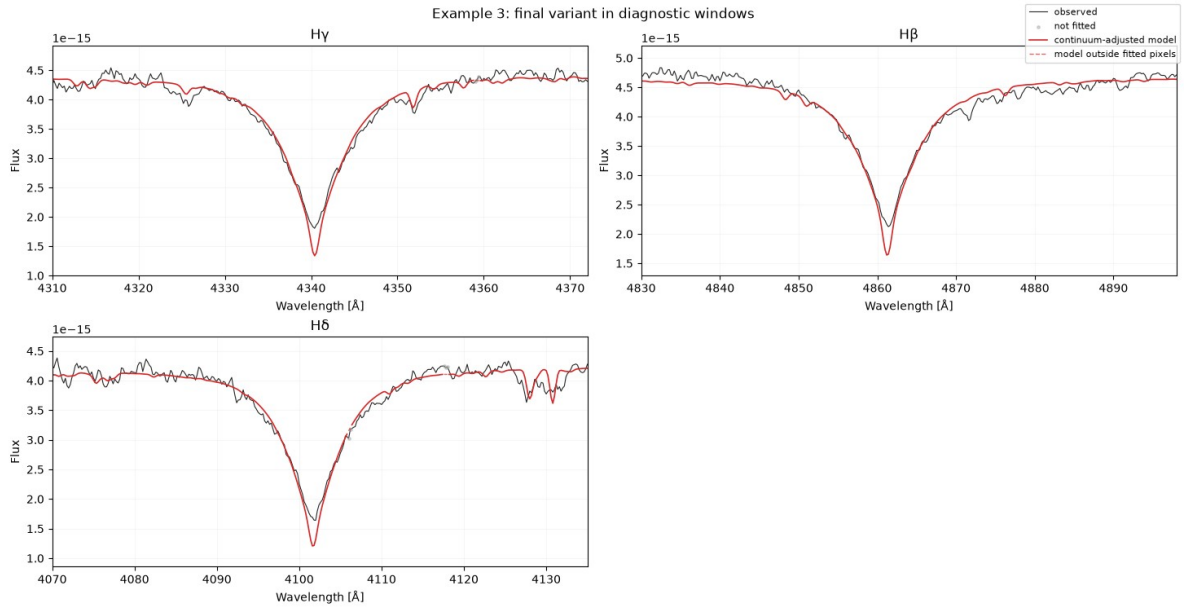

Figure 15.1. Selected-window comparison for the final Example 3 fit variant. The continuum-adjusted PHOENIX model reproduces much of the broad Balmer morphology, while visible line-core and local shape mismatches remain. Pixels not used by the fit and model behaviour outside fitted pixels are shown distinctly. This is why an apparently successful final variant still requires feature-level inspection.

15.2 Best-fit atmospheric parameters

Read Teff, log g, [Fe/H], and RV together rather than independently. Their inferred values can be correlated through line sensitivity, continuum treatment, wavelength coverage, and resolution. Before quoting any parameter, check whether it lies comfortably inside the supported model/search region and whether the spectral features expected to constrain it are actually present in the fit.

15.3 Radial velocity

The fitted velocity is the shift required by the forward model relative to the active wavelength frame. Its physical interpretation must follow the reader metadata. For a topocentric or barycentric spectrum it may correspond to a stellar velocity in that frame, subject to the package conventions and corrections; for a stellar-rest spectrum it is primarily a residual alignment diagnostic.

15.4 Objective statistics

A high reduced chi-square can be caused by underestimated formal uncertainties, correlated noise, incorrect resolution, continuum mismatch, abundance differences, unmodelled physics, or artifacts. A value near one can be equally misleading if the uncertainties are inflated or the nuisance model is too flexible. The statistic is most useful when interpreted alongside the residual pattern and uncertainty provenance.

15.5 Residual morphology

Residual pattern	Possible interpretation	Next check
Broad smooth curvature	Continuum/calibration mismatch.	Compare continuum degree and flux-calibration state.
Symmetric line-width mismatch	Resolution/LSF or missing broadening physics.	Check adopted R/LSF and model limitations.
Line centers systematically shifted	RV/frame/wavelength calibration issue.	Review frame semantics and local line centroids.
Only specific metal lines fail	Abundance/model mismatch or blends.	Check whether [Fe/H] is being asked to represent non-solar abundance patterns.
Balmer cores fail but wings agree	Core physics, activity, normalization, or core-mask issue.	Use explicit core masking and sensitivity tests for a wing analysis.
Localized spikes	Cosmic rays, bad pixels, telluric/interstellar contamination.	Inspect mask provenance rather than increasing continuum flexibility.
Different arms show different structures	Arm calibration/resolution or product-specific issues.	Inspect segments separately and compare per-arm assumptions.

15.6 Continuum coefficients

The fitted Legendre coefficients are nuisance parameters, but they are scientifically diagnostic. They should describe a smooth, plausible correction. If a high-degree continuum has to bend strongly around broad stellar features, the nuisance model may be absorbing astrophysical information. Compare continuum variants when the inferred stellar parameters depend strongly on this choice.

15.7 Boundary and search diagnostics

A solution at an optimizer or grid boundary should be called out explicitly. Also compare the attempted starts: a much better objective from only one isolated start can indicate a complicated landscape, while many starts converging consistently supports numerical robustness. Neither pattern establishes physical adequacy, but both inform how much trust to place in the optimizer.

15.8 Quality flags and readiness after the fit

Post-fit quality flags should be interpreted in the context of the intended use. Structured residuals, artifact review requirements, ambiguous RV semantics, grid-edge solutions, or suspicious uncertainties can limit reviewed scientific interpretation even if a quicklook classification remains useful. The result should preserve these caveats rather than requiring the user to remember them from the notebook session.

15.9 Formal uncertainty is not the total uncertainty

Any covariance or local uncertainty derived around the numerical optimum describes the local objective under the adopted model and data weights. It does not automatically include model-grid limitations, continuum choice, LSF uncertainty, mask sensitivity, abundance mismatch, wavelength calibration, or external systematics. Serious atmospheric-parameter uncertainties therefore require the stability and validation work developed in later parts of the manual.

15.10 Save a reproducible report

```
# Representative current result/report methods
result.save_report_json("fit_report.json")

# A compact backwards-compatible JSON may also be available:
# result.save_json("fit.json")
```

Versioned report envelopes are designed for reviewer, archival, web, and Django hand-off. They can include the Spyctres version, git commit when available, setup hash, PHOENIX grid/cache provenance, uncertainty caveats, mask/readiness/resolution policy, generated plot paths, and optionally an input-file checksum. Absolute local paths are sanitized by default so reports can be shared safely.

15.11 A minimum interpretation checklist

1. Verify that the model reproduces the main fitted stellar features at the correct wavelengths and approximate depths.
2. Check that the fitted RV has the correct interpretation for the active frame.
3. Check that Teff, log g, and [Fe/H] are not being pinned by a search or model boundary.
4. Inspect whether the largest residuals are random-looking or form coherent line-specific structures.
5. Check that the continuum correction is smooth and plausible rather than compensating for line-shape failures.
6. Review quality flags, readiness status, uncertainty caveats, and any exploratory override.
7. Confirm that the exact FitSetup and model/cache provenance have been stored.
8. For any result intended for serious scientific use, proceed to bounded sensitivity tests and external validation rather than quoting the first deterministic optimum as final.

End point of the standard workflow

At the end of Chapter 15 you should have a fit that is computationally reproducible and scientifically qualified. You do not yet have a complete systematic uncertainty budget or proof of model adequacy. Those require the reviewed-analysis, stability, and validation workflows that follow in later parts of the manual.

Part IV - Advanced worked examples

Controlled fit improvement, Balmer analysis, stability tests, multi-arm spectra, and batch fitting.

16. Improving a first PHOENIX fit

Example 3 begins where the ordinary workflow often leaves a real dataset: a PHOENIX fit has run, the broad classification may be plausible, but the residuals, reduced chi-square, model-grid position, or sensitivity to analysis assumptions prevent a stronger interpretation. The purpose of this example is not to search harder until the statistic improves. It is to identify which scientific or numerical assumption is controlling the answer.

16.1 Start from a named baseline

A useful comparison requires a reference fit whose setup, data selection, and result are preserved. Do not overwrite the baseline while experimenting. The baseline should be generated from an inspectable FitSetup and retained as a separate result object or report.

```
setup = sp.suggest_fit_setup(spec, mode="standard", intent="atmospheric_parameters")
print(setup.summary_text())

baseline = sp.fit_stellar_spectrum(
    spec,
    model="phoenix",
    setup=setup,
)
```

The exact supported mode and intent strings should be checked in the current checkout. The important pattern is stable: create the setup, inspect it, fit with that exact setup, and keep the result under a name that states what it represents.

16.2 Diagnose before changing anything

A poor-looking fit does not identify its own cause. First decide what kind of failure is present. Different residual morphologies call for different tests.

Observed symptom	Plausible cause	Useful controlled test
Smooth residual curvature across windows	Continuum/calibration mismatch.	Change continuum degree modestly; inspect whether stellar parameters move.
Lines systematically too broad or narrow	Resolution/LSF assumption or missing broadening physics.	Change the adopted resolution within a defensible range.
One region dominates the residuals	Contamination, local model mismatch, or bad data.	Compare with that region removed or isolated; do not delete it silently.
Solution at hot/cool or gravity grid edge	True parameter near boundary, wrong region choice, or model inadequacy.	Inspect boundary status; do not interpret a boundary hit as a precise estimate.
Different starts lead to noticeably different minima	Nonlinear landscape or insufficient initialization.	Use a stronger search mode or inspect optimizer attempts.
High chi-square but broadly correct morphology	Very small formal errors, correlated residuals, or model incompleteness.	Inspect uncertainty provenance and residual structure; do not tune until chi-square is one.

16.3 Change one assumption at a time

The central discipline of Example 3 is one-factor-at-a-time comparison. If the wavelength regions, resolution, continuum degree, and search mode all change simultaneously, a better fit cannot be attributed to any particular improvement. The comparison becomes a new opaque model rather than a diagnostic experiment.

1. Preserve the baseline setup and result.
2. Choose one scientifically plausible alternative assumption.
3. Create a new setup or fit configuration that differs only in that assumption.
4. Run the new fit and retain it under a descriptive variant name.

5. Compare parameters, objective statistics, boundaries, quality flags, and residual morphology.
6. Decide whether the change reveals robustness, a systematic dependence, or a failure mode.

16.4 Fit-region variants

A fit-region experiment asks whether the result depends on a particular diagnostic window. This is scientifically more informative than simply expanding the wavelength range. New regions should add independent leverage, not merely more pixels. Conversely, removing a difficult region should be treated as a test of sensitivity, not an automatic improvement.

Do not optimize the mask by chi-square
If the analysis repeatedly removes whichever region fits worst, the final statistic becomes partly a measure of how effectively the difficult evidence was discarded. A region should be excluded because the data or model make it invalid for the stated inference, not because its residuals are inconvenient.

16.5 Resolution variants

Resolution sensitivity is especially important when the product provides only nominal resolving power or when the slit/seeing combination makes the effective LSF uncertain. The model line widths and detailed Balmer/metal-line shapes can move as the assumed resolution changes. A defensible analysis therefore asks whether modest, plausible changes in R alter Teff, log g, [Fe/H], or RV enough to matter.

The test must respect the current forward model: the production likelihood applies a constant Gaussian LSF when broadening is requested. A wavelength-dependent or non-Gaussian instrumental profile cannot be validated merely by changing a single constant R.

16.6 Continuum-degree variants

The multiplicative Legendre continuum is intended to absorb smooth residual calibration shape, not stellar line physics. A lower degree may leave broad flux-shape mismatch; an unnecessarily high degree can begin to absorb astrophysical structure. Continuum degree should therefore be treated as a nuisance-model assumption whose effect on the inferred stellar parameters is measured.

Outcome	Interpretation
Parameters are stable and residual curvature improves modestly.	The nuisance continuum is probably handling calibration shape without dominating the stellar inference.
Parameters shift substantially as degree changes.	The spectrum does not constrain the stellar parameters independently of continuum treatment as strongly as assumed.
High degree strongly improves chi-square around broad stellar features.	Check whether the continuum is fitting away line-wing information.
Low degree leaves arm-scale curvature but line profiles agree.	The atmospheric inference may still be locally useful, but global flux-shape interpretation is weak.

16.7 Search-mode variants

A stronger numerical search tests optimizer robustness; it does not create new scientific information. If quicklook and stronger modes converge to the same interior solution, numerical stability is supported. If only the strongest search finds a substantially different minimum, inspect why. The lower objective may reflect a genuine basin, but it may also sit at a grid boundary or exploit a questionable nuisance configuration.

Search intensity is not evidence
The most expensive optimizer result is not automatically the most physically credible result. Numerical effort answers “did we search this objective more thoroughly?”, not “is this objective the right scientific model?”

16.8 Exploratory overrides

Spyctres deliberately separates whether a calculation is allowed to run from whether its output is ready for a particular interpretation. In an exploratory workflow, a blocked setup can be explicitly allowed to run with a

recorded reason. This is useful when the purpose is to diagnose a failure, compare residuals, or determine whether an assumption is worth revisiting.

```
exploratory_setup = setup.allow_exploratory(
    reason="diagnostic comparison; not for final atmospheric interpretation"
)
```

An exploratory override should survive into the result provenance. It does not erase blockers or convert a warning into approval. In a paper or hand-off, an exploratory result should remain labelled as such.

16.9 Compare fits as a set

```
comparison = sp.compare_fits([
    baseline,
    continuum_variant,
    resolution_variant,
    region_variant,
])

print(sp.format_fit_comparison_table(comparison))
```

The comparison is most useful when the variant names encode the changed assumption. Compare not only reduced chi-square but parameter shifts, grid-edge status, fitted-pixel count, continuum degree, resolution assumptions, and quality flags. A small change in chi-square with a large change in Teff or log g is often more important than a large change in chi-square with stable parameters.

16.10 When is the first fit “improved”?

A fit is improved when the revised assumptions are better justified and the result is more stable and interpretable, not merely when the objective decreases. A reasonable stopping point is reached when the dominant residual structures are understood, plausible nuisance choices no longer move the scientific conclusion materially, and any remaining limitations can be stated explicitly.

Example 3 conclusion

A hot grid-edge solution, high chi-square, or structured residuals remain warning signs even if several variants agree. Agreement among variants is evidence about robustness to those tested choices, not proof that PHOENIX or the instrumental model is complete.

17. Reviewed Balmer-wing analysis

Example 4A applies the general workflow to a deliberately difficult real spectrum: the X-SHOOTER UVB spectrum of Gaia21ccu. The example is designed to show what additional discipline is required when Balmer wings carry much of the temperature information and the spectrum contains artifacts, continuum uncertainty, and line-core regions that should not automatically drive the fit.

17.1 What “reviewed analysis” means here

The phrase “reviewed analysis” is intentionally narrower than “publication quality”. Spyctres can require explicit regions, masks, wavelength conventions, resolution assumptions, uncertainty provenance, readiness checks, residual inspection, and bounded sensitivity tests. It cannot certify that the atmosphere grid is complete, that every systematic has been measured, or that an external reviewer would accept the inference.

Reviewed does not mean certified

The package can make the analysis auditable and can block known unsafe interpretations. Scientific adequacy still depends on the data, the physical model, external validation, and expert review.

17.2 Why Balmer wings are treated separately

Broad Balmer wings are primarily strong effective-temperature diagnostics in appropriate stellar regimes. They can also carry gravity-dependent information, but the strength and reliability of that leverage depend on temperature, gravity, wavelength coverage, and the adopted atmosphere physics. Their narrow cores can respond differently to atmospheric physics, activity, normalization, instrumental effects, or reduction artifacts. The reviewed example therefore makes the wing/core distinction explicit instead of assuming that every pixel near a Balmer line belongs in the same likelihood. A Balmer-wing-only fit should not by itself be treated as an independent metallicity measurement.

This does not mean that cores are “bad data”. They remain useful diagnostics. Masking them in a wing-focused fit is a statement about the chosen forward model and inference target, and that statement must be visible and sensitivity-tested.

17.3 Recipe-led preparation

```
balmer_case = sp.recipes.prepare_xshooter_balmer_case(
    spec,
    window_mode="notebook",
    norm_mode="sideband",
    sideband_width=10.0,
    sideband_order=1,
    core_mask=3.0,
)
```

The values above illustrate the maintained Gaia21ccu teaching case rather than universal defaults. The recipe encapsulates a sequence that would otherwise be easy to reproduce inconsistently: define Balmer windows, establish local sidebands, normalize or represent the local shape according to the selected mode, apply the explicit core exclusion, preserve the reader quality mask, and record what was done. Exact options should be checked with help() in the current checkout.

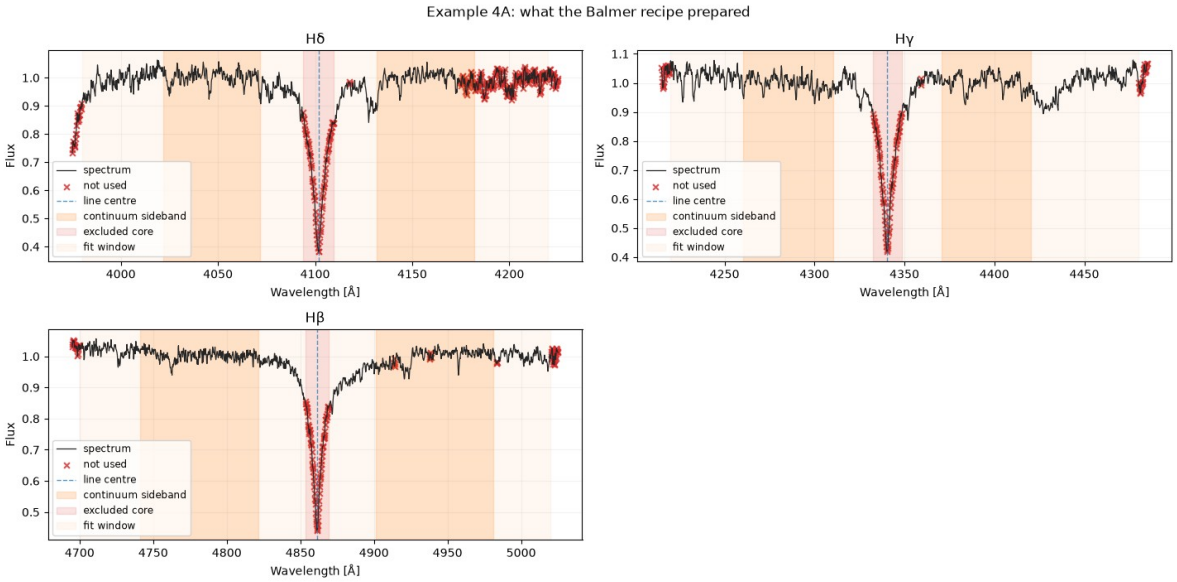

Figure 17.1. Example 4A: what the reviewed Balmer recipe prepared. The shaded regions distinguish the retained fit windows, sidebands used for local continuum normalization, and the explicitly excluded Balmer cores. Red crosses mark pixels not used in the reviewed wing analysis.

17.4 Local sideband normalization

Local normalization is used because the scientific target is the line profile rather than the absolute spectral-energy distribution. Sidebands adjacent to each Balmer feature define a smooth local reference, reducing sensitivity to broad flux-calibration shape and slit losses. This is not a claim that the continuum is known perfectly. Sideband placement and polynomial order are themselves analysis assumptions.

Choice	What it controls	Failure mode to watch
Sideband width	How much local continuum information is used.	Too narrow: noise/local features dominate. Too broad: curvature or unrelated features bias the local continuum.
Sideband order	Flexibility of the local continuum model.	Too flexible: line wings can be partially absorbed. Too rigid: response curvature remains in the profile.
Window boundaries	How much wing and neighboring spectrum enter the analysis.	Can change leverage or include artifacts/blends.
Core-mask half-width	How much central line structure is excluded from the wing objective.	Too small: core mismatch drives parameters. Too large: useful wing information is discarded.

17.5 Preserve the reader quality mask

Pixels already rejected by the X-SHOOTER product quality information remain rejected for that reason. The manual core mask is then an additional scientific exclusion, not a replacement for product-quality provenance. This distinction matters when reporting the fraction of each Balmer window that was actually fitted.

17.6 Run the analysis-readiness audit

```
audit = sp.analysis_readiness_audit(
    balmer_case.collection,
    regions=balmer_case.fit_regions_by_segment,
    exclude_masks=balmer_case.exclusion_masks,
    intended_use="atmospheric_parameters",
)
print(audit)
```

The readiness audit is applied to the prepared `balmer_case.collection`, with the recipe's retained regions and exclusion masks passed explicitly. The current maintained examples use analysis-readiness language rather than asking the software to certify publication readiness. A blocker means the requested interpretation is not supported without further review or an explicit exploratory override; a warning means the calculation may be usable but the caveat must remain attached to the result. Read the audit before fitting rather than retrofitting it to a preferred solution.

17.7 Construct and inspect the reviewed FitSetup

```
setup = balmer_case.suggest_fit_setup(
    mode="standard",
    intent="atmospheric_parameters",
    continuum_degree=2,
)
print(setup.summary_text())
```

The setup should identify the retained Balmer regions, the active mask, parameter bounds and seeds, the adopted resolution information, continuum treatment, and readiness interpretation. In the Gaia21ccu UVB case, the main fitted information comes from Hdelta, Hgamma, and Hbeta wing regions rather than from an indiscriminate full-arm fit.

17.8 Fit the Balmer regions jointly

```
result = sp.fit_stellar_spectrum(
    balmer_case.collection,
    model="phoenix",
    setup=setup,
)
```

The fit is run on `balmer_case.collection`, the prepared multi-segment object returned by the Balmer recipe. Joint fitting asks one atmospheric parameter set to reproduce several Balmer regions simultaneously. This is stronger than fitting each line independently and averaging the answers, because the same Teff, log g, metallicity, and RV must remain compatible with all retained regions under the stated nuisance treatment. Per-segment or per-region continuum flexibility can absorb smooth calibration differences without requiring the input arms or windows to be merged onto one artificial scale.

17.9 Inspect model line windows and masked cores

The preferred diagnostic plot shows observed profiles, the continuum-adjusted PHOENIX model, and the model continuation through masked core pixels. Showing the model over excluded pixels is useful because it reveals whether the omitted core mismatch is localized or whether the entire line profile is inconsistent. The visual style must make clear that masked pixels did not contribute to the objective.

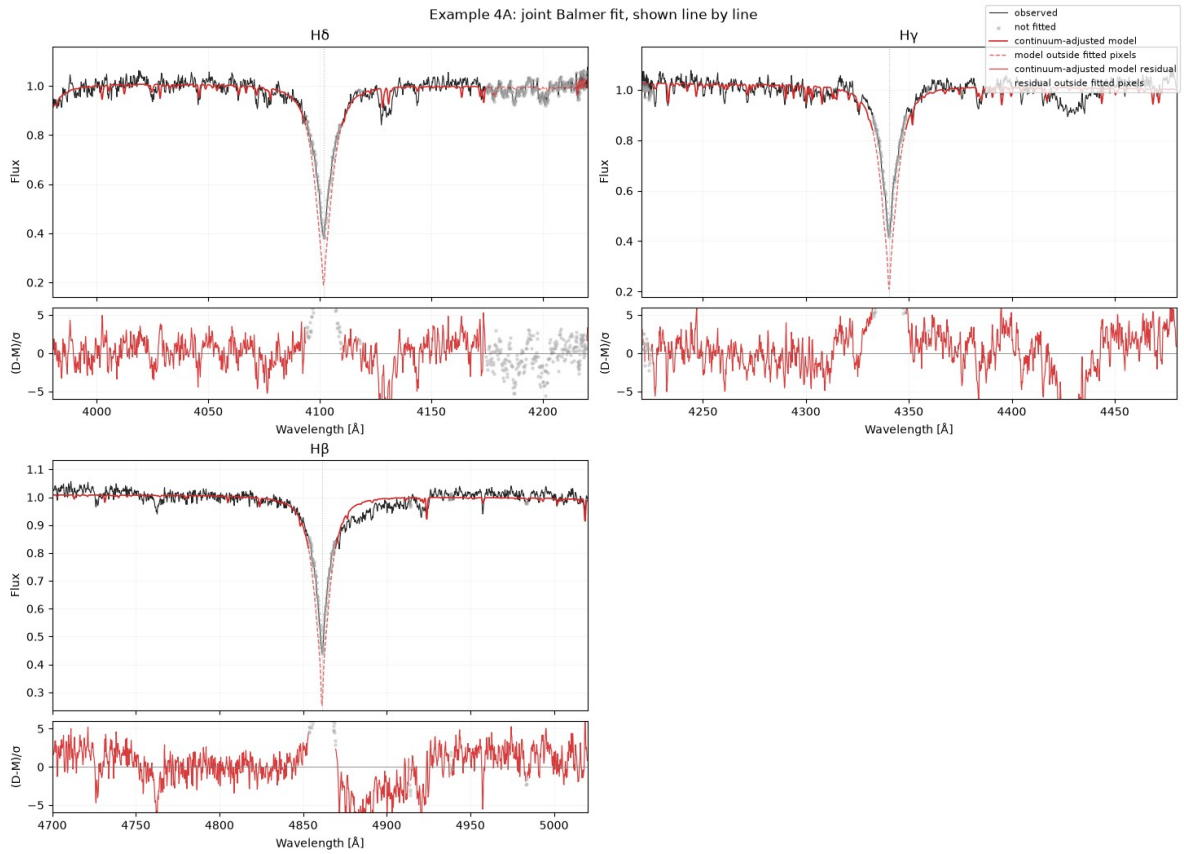

Figure 17.2. Example 4A: joint Balmer fit shown line by line. The observed profiles are compared with the continuum-adjusted PHOENIX model, while the dashed model trace continues through excluded core pixels. The residual panels make clear where the fit is informative and where structured mismatch remains.

17.10 Individual-line consistency

After the joint fit, inspect Hdelta, Hgamma, and Hbeta separately. The goal is not to demand identical residuals. It is to identify whether one line requires a qualitatively different solution or contains a localized structure that should be investigated. A line that disagrees strongly with the others can point to normalization, contamination, line-specific model limitations, or an inappropriate mask.

17.11 What Example 4A establishes - and what it does not

It establishes	It does not establish
A transparent Balmer-wing fitting recipe with explicit masks and normalization.	A complete systematic uncertainty budget.
A joint PHOENIX atmospheric solution for the selected windows.	That every relevant physical broadening or abundance effect is modelled.
A readiness record and inspectable residuals.	That the result is automatically suitable for publication.
A baseline suitable for controlled sensitivity tests.	That the chosen core width, continuum order, or resolution is uniquely correct.

18. Stability and sensitivity checks

Example 4B asks the question that must follow a carefully constructed baseline: if we change plausible analysis choices within a bounded range, does the scientific conclusion remain stable? This is a systematic-sensitivity exercise. It is not a brute-force search for whichever variant produces the smallest chi-square.

18.1 Start from the reviewed baseline

Every stability variant should inherit the same data product and the same baseline scientific intent. The baseline from Example 4A is therefore the reference point. A variant should change one named assumption unless the purpose of a deliberately coupled test is stated explicitly.

18.2 Continuum-degree sensitivity

Refit with a small set of defensible continuum degrees. Compare not only the statistic but the atmospheric parameters and the shape of the residuals. If Teff or log g move substantially with continuum degree, the broad line information is entangled with the nuisance continuum more strongly than a single fit would reveal.

18.3 Balmer-core-mask sensitivity

Vary the core half-width over a small, interpretable range. A stable wing solution should not depend catastrophically on moving the core boundary by a modest amount. At the same time, increasing the mask always removes information, so apparent stability at very large masks can be misleading if only weak wing leverage remains.

Information retention matters

A larger mask can improve the objective simply by excluding the most difficult pixels. Record the number or fraction of fitted pixels as the mask changes, and interpret parameter stability together with the amount of information retained.

18.4 Leave-one-line-out and line-selection tests

A joint Balmer solution can hide tension if one line dominates the likelihood. Leave-one-line-out variants ask whether the inferred atmosphere changes when Hdelta, Hgamma, or Hbeta is omitted. Single-line checks ask a complementary question: does any one line demand a qualitatively incompatible parameter region?

Pattern	Interpretation
All leave-one-out variants agree within a small range.	The result is not controlled by one Balmer line alone.
Removing one line causes a large parameter shift.	That line contains disproportionate leverage or conflict; inspect its normalization, artifacts, and model adequacy.
Single-line fits disagree but the joint fit is intermediate.	The joint solution may be a compromise rather than a physically consistent description.
One line has structured residuals at every variant.	Treat the structure as a diagnostic problem; do not simply hide it by increasing the mask.

18.5 Resolution/LSF sensitivity

Repeat the fit across a justified range of resolving power or LSF assumptions. The test asks whether the inferred stellar parameters are robust to realistic uncertainty in instrumental broadening. It does not validate a wavelength-dependent or non-Gaussian LSF, because the production likelihood currently uses the constant-Gaussian broadening path when active.

18.6 Compare the variants compactly

```
comparison = sp.compare_fits(variant_results)
print(sp.format_fit_comparison_table(comparison))
```

The display table should be compact enough that parameter movement is visible. Retain the complete result objects separately so the table does not become the only record of the experiments. The useful quantities include variant name, changed assumption, fitted-pixel count, Teff, log g, [Fe/H], RV, reduced chi-square or equivalent statistic, boundary status, and quality flags.

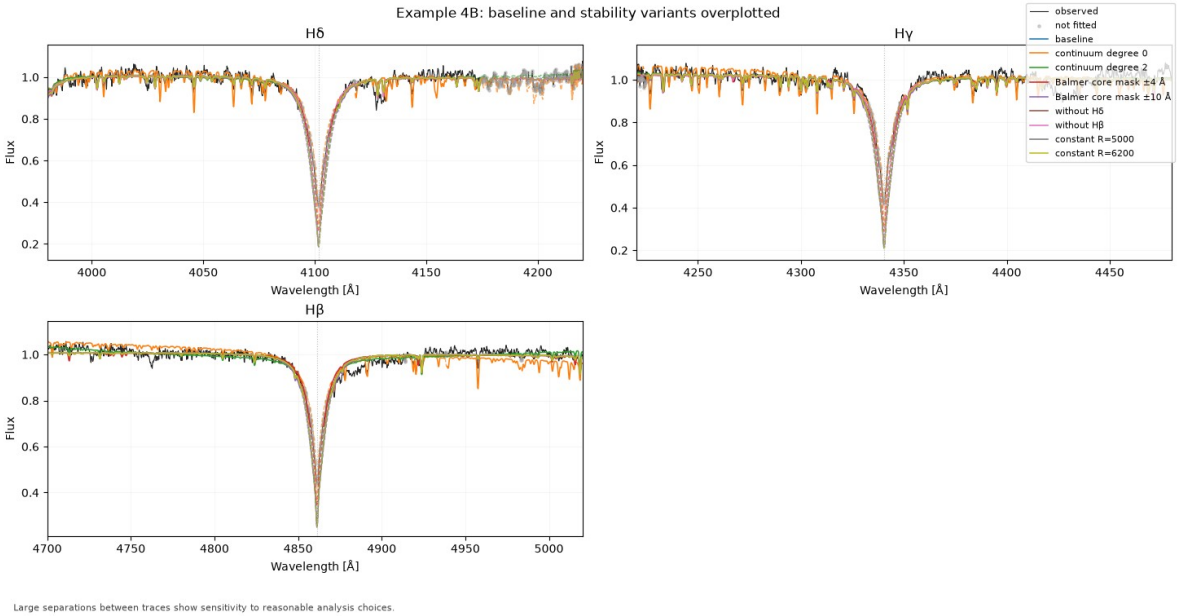

Figure 18.1. Example 4B: baseline and bounded stability variants overplotted. The figure compares the reviewed baseline with continuum-degree, core-mask, line-selection, and constant-resolution variants. Large separations between traces indicate sensitivity to reasonable analysis choices.

18.7 Residual-window triage

Example 4B also demonstrates a different kind of diagnostic: localized residual structures are compared with curated non-stellar or known-problem regions. Spyctres can search for catalog overlaps, annotate possible non-stellar features, and diagnose selected residual windows. These tools are hypothesis generators, not automatic corrections.

```
features = sp.find_known_nonstellar_features(...)
annotated = sp.annotate_nonstellar_features(...)
diagnostics = sp.diagnose_known_residual_windows(...)
```

If a residual lies near a catalogued DIB or telluric feature, the correct conclusion is initially “possible overlap”. A positional coincidence does not establish a physical detection, and it does not imply that the region should be masked. First ask whether the residual is consistent across spectra, arms, epochs, and model variants.

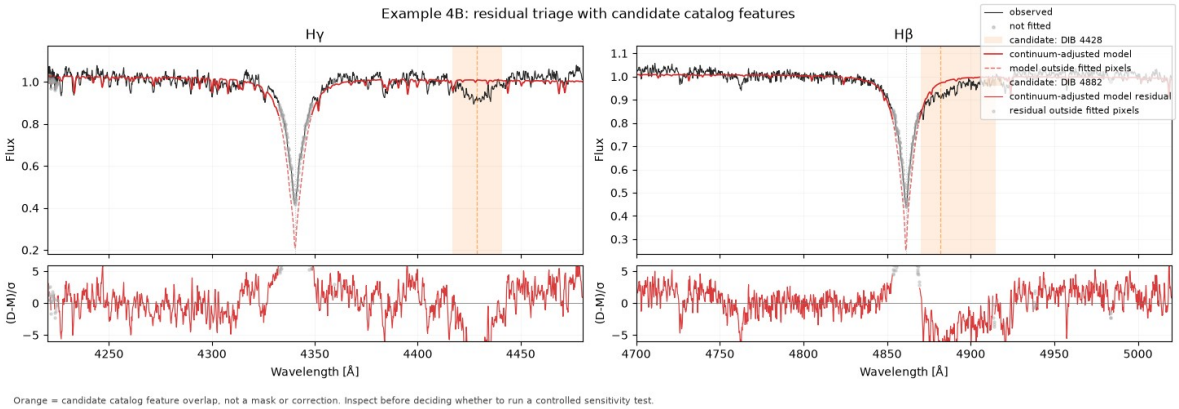

Figure 18.2. Example 4B: residual triage with candidate catalog features. The shaded orange regions mark candidate catalog overlaps only. They are not automatic masks or corrections, and the figure is meant to support controlled sensitivity testing rather than premature feature identification.

Diagnosis before masking
Do not turn every residual annotation into a new exclusion. Otherwise the sensitivity analysis becomes a procedure for progressively removing evidence that the model does not reproduce.

18.8 Plot model and comparison windows

The advanced plotting helpers allow the baseline and selected variants to be inspected in the same line windows. This is more informative than comparing only parameter tables because it shows which part of a profile changed when a continuum, mask, line selection, or resolution assumption changed.

```
sp.plot_model_line_windows(...)
sp.plot_fit_comparison_line_windows(...)
```

18.9 Turning stability into an uncertainty statement

The spread among bounded variants is evidence about sensitivity to those specific choices. It is not automatically a statistically calibrated uncertainty interval. A defensible report can state, for example, that the atmospheric solution is stable to the tested continuum orders, core masks, line selections, and resolution assumptions, while separately reporting formal fit uncertainties and external systematic limitations.

If the variants move the scientific conclusion materially, that is not a failed exercise. It is the result: the inference is conditional on analysis choices that need stronger external constraints or a more complete model.

18.10 When to stop adding variants

Sensitivity testing should be bounded by plausible uncertainty, not by imagination. Add a variant when there is a concrete reason to doubt an assumption or when a reviewer would reasonably ask whether that choice matters. Once the dominant plausible choices have been tested, further arbitrary combinations usually add computational volume without increasing scientific clarity.

19. Multi-segment and multi-arm spectroscopy

Example 6 extends the workflow from one spectrum to X-SHOOTER UVB, VIS, and NIR arms. The central principle is “inspect broadly, fit narrowly”. The full wavelength coverage is scientifically useful for classification and consistency checks, but this does not mean every arm or every available diagnostic should enter one global objective.

19.1 Keep the arms as separate segments

UVB, VIS, and NIR are represented as a SpectrumCollection. Each arm retains its own wavelength grid, valid mask, uncertainty array, resolution metadata, frame/medium state, and provenance. Keeping the arms separate prevents the ingestion step from inventing a single common flux scale or a single instrumental resolution.

```
collection = sp.SpectrumCollection([uvb, vis, nir], name="Gaia21ccu X-SHOOTER")
```

Do not merge merely for convenience
A concatenated wavelength array can hide arm-specific masks, resolutions, flux-calibration states, and joins. A collection allows one atmospheric model to use several segments without pretending they are one homogeneous detector product.

19.2 Select diagnostic windows over the combined coverage

```
windows = sp.select_diagnostic_windows(collection)
sp.plot_diagnostic_windows(collection, windows)
```

Combined coverage allows Spyctres to surface diagnostics that are absent from the UVB-only analysis. Depending on the star and usable pixels, these can include optical hydrogen and helium lines, Mg/Na/Ca regions, Paschen and Brackett features, molecular bands, and the CO 2.3 micron bandhead. The selector remains advisory: a window can be valuable classification context while remaining unsuitable for fitting.

19.3 Positive and negative classification evidence

Multi-arm inspection is useful because classification is not based only on which expected feature is present. The absence of strong cool-star molecular or atomic features can constrain alternatives, while Balmer and helium morphology can provide positive evidence for a hotter source. Such negative evidence should be qualified by signal-to-noise, telluric contamination, mask fragmentation, and possible composite light.

19.4 Full collection versus fit-only collection

The maintained workflow uses the full collection for diagnostic plots and scientific context, then constructs a separate fit-only collection from arms and windows judged conservative enough to constrain the model. The current 0.5.0a1 code includes a helper to reduce manual arm/window bookkeeping.

```
fit_collection = sp.build_fit_collection_from_windows(
    collection,
    retained_windows,
)
```

The exact helper signature should be confirmed in the current checkout. Conceptually, it aligns the retained diagnostic windows with the correct segments without flattening away arm identity. This reduces a common source of multi-arm mistakes: accidentally applying a wavelength interval to the wrong arm or fitting an arm that was intended only for context.

19.5 Why the bundled example fits UVB plus VIS but treats NIR conservatively

For the Gaia21ccu teaching case, the optional multi-arm fit uses the UVB Balmer lines together with VIS Halpha as a consistency constraint. NIR regions are inspected but are not default fit constraints because usable pixels in important regions can be fragmented or telluric sensitive in this reduction. This is a data-quality decision, not a statement that NIR spectroscopy is intrinsically less valuable.

Arm	Role in the maintained example	Main caution
UVB	Primary Balmer structure: Hdelta, Hgamma, Hbeta.	Blue-end quality, artifacts, local continuum and core sensitivity.
VIS	Halpha consistency and additional optical context.	Tellurics, arm response, line-specific model/continuum differences.
NIR	Hydrogen and cool-star/molecular diagnostic context.	Fragmented valid pixels and telluric sensitivity in this particular reduction.

19.6 Arm-to-arm flux alignment

Imperfect absolute alignment between arms is not necessarily fatal for local line-profile classification because each line window can be treated with its own smooth continuum nuisance model and each segment retains its own metadata. It would be unacceptable, however, to infer a physical SED, extinction, or angular scale from arbitrarily rescaled arms without an explicit calibration model.

Spyctres therefore does not silently rescale UVB, VIS, and NIR into a common physical continuum in this workflow. If an arm join is poor, keep the affected region diagnostic-only or model the calibration explicitly in a future flux-sensitive analysis.

19.7 Resolution and weighting remain per segment

Different arms can have different resolving powers and uncertainty quality. A joint fit should transform the common stellar model according to each segment's adopted broadening and evaluate each segment on its own pixels. Segment weights, when used, should have a scientific rationale; they should not be tuned merely to force visual agreement between arms.

19.8 Optional multi-arm fit

```
setup = sp.suggest_fit_setup(
    fit_collection,
    mode="standard",
    intent="quicklook_classification",
)
print(setup.summary_text())

result = sp.fit_stellar_spectrum(
    fit_collection,
    model="phoenix",
    setup=setup,
)
```

The maintained example frames this fit as classification and cross-arm consistency, not a final multi-arm atmospheric-parameter determination. A high chi-square or quality flags can therefore coexist with a useful qualitative conclusion, provided the limitations remain explicit.

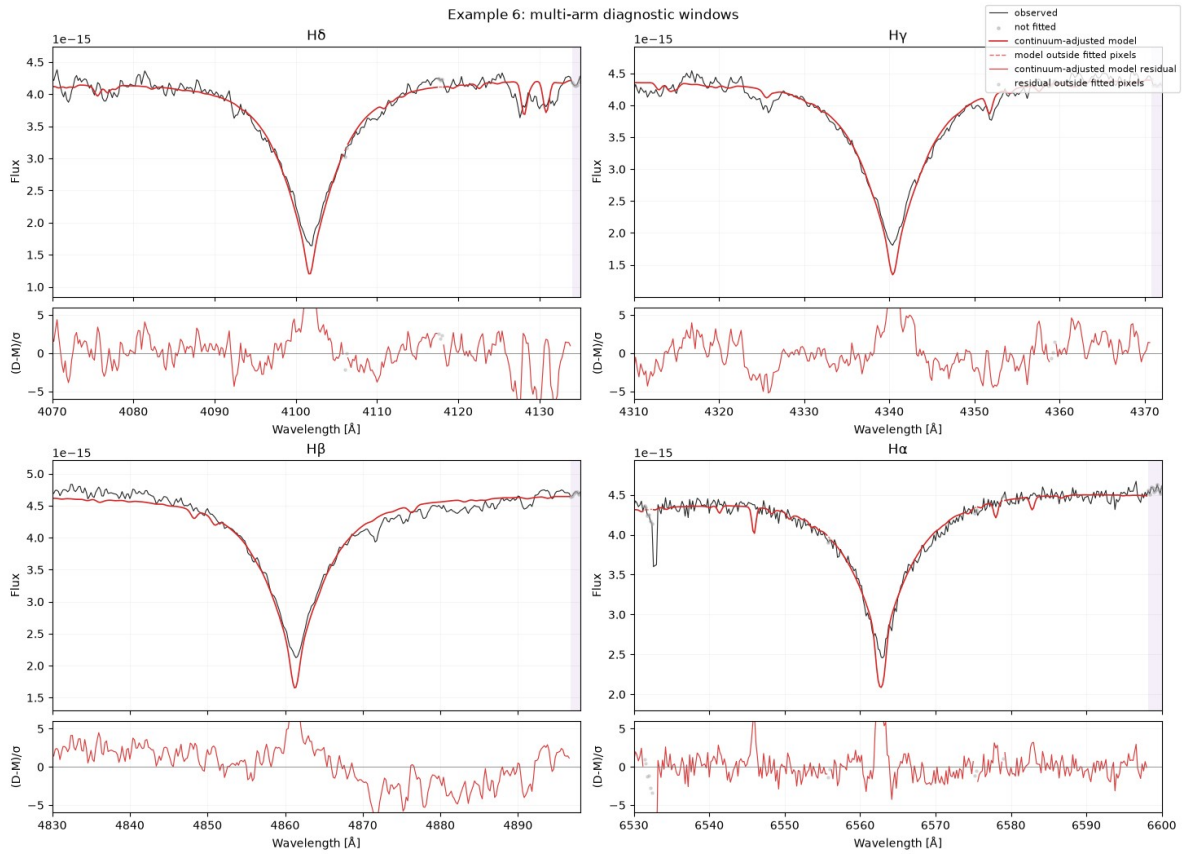

Figure 19.1. Example 6: retained multi-arm fit windows and line-by-line model comparison across UVB and VIS. Although the notebook title refers to "multi-arm diagnostic windows", the most useful message here is which windows actually entered the optional fit and how the common atmospheric solution behaves in each retained arm.

19.9 What must not happen automatically

- No hidden arm rescaling to make the continua line up.
- No assumption that every selected diagnostic window is fit-ready.
- No automatic inclusion of telluric-sensitive NIR regions merely because they exist.
- No extra wavelength-frame correction when the reader already records the product state.
- No claim that a local multi-arm classification fit is a physical SED fit.

19.10 A defensible multi-arm conclusion

A good multi-arm report states which arms were inspected, which windows supplied positive or negative classification evidence, which arms/windows actually entered the objective, how each arm's resolution and masks were handled, and why any excluded arm was retained only as context. The scientific strength comes from making those roles explicit, not from maximizing wavelength coverage in the optimizer.

20. Batch analysis

Example 5 scales Spyctres from one spectrum to tens or hundreds. The scientific problem changes as soon as the analysis becomes a batch: a workflow that is acceptable when a human watches every fit can become dangerous when repeated automatically. The batch design therefore separates cheap triage from focused refinement, checkpoints intermediate state, and allows readiness and quality flags to stop expensive fits rather than hiding problematic targets inside a large output table.

20.1 The quickscan-refine pattern

1. Load or initialize the PHOENIX library once for the batch rather than rebuilding it for every target.
2. Read each spectrum through the correct product-aware reader and run lightweight inspection/readiness logic.
3. Run a cheap quickscan to identify plausible parameter ranges, RV alignment, and targets worth further work.
4. Write checkpoint JSON and a compact summary table as the batch progresses.
5. Apply readiness and quality gates to decide which spectra may proceed to focused refinement.
6. Refine only the retained targets with a stronger, narrower fit configuration.
7. Record skipped targets and the reasons they were skipped.

Batch principle

Automation should increase throughput, not decrease scientific standards. A target that would make you stop and inspect a single-spectrum notebook should not become “successful” merely because it appeared in row 73 of a batch CSV.

20.2 Why not run the strongest fit on every spectrum?

A blind expensive search wastes computation on spectra with missing metadata, artifacts, poor masks, ambiguous frames, or obviously unsuitable wavelength coverage. More importantly, it can turn problematic data into precise-looking numbers. A quickscan is therefore a triage stage: it narrows the search and establishes priorities. It is not a calibrated final parameter estimate.

20.3 Load PHOENIX once

PHOENIX library discovery, wavelength-grid loading, interpolation structures, and caches are relatively expensive setup operations. Batch workflows should reuse them. This is both faster and more reproducible because the same model-grid manifest and cache state apply across targets unless the user deliberately changes them.

20.4 Checkpointing

Long batches should be restartable. Checkpoint JSON records the completed work and its status so an interrupted job does not have to begin from target one. A summary CSV provides an operational view of the batch, but the JSON/report products should retain the richer provenance and quality information that a flat table cannot represent.

Artifact	Role	What it should not replace
Checkpoint JSON	Resume state, per-target status, structured batch progress.	The detailed per-fit result/report where scientific provenance is needed.
Summary CSV	Rapid sorting/filtering of target status and key parameters.	Residual inspection, mask provenance, or readiness reasoning.
Per-target result/report	Full fit diagnostics, setup, flags, provenance, and shareable record.	Batch-level throughput/operational status.

20.5 Resume and force semantics

Resume should skip work already completed according to the checkpoint. Force should deliberately rerun work even when an existing result is present. These are operational controls, not scientific overrides.

Forcing a rerun does not mean forcing a target through readiness gates unless the workflow has a separately explicit exploratory mechanism for that purpose.

20.6 Quality-gated refinement

The focused stage should be skipped when blockers or serious quality flags make the requested interpretation unsafe. Examples include unresolved artifact review, strongly structured residuals, ambiguous RV/frame semantics, or other metadata problems. The skip reason is part of the result, not an error to suppress.

Quickscan outcome	Batch action
Plausible solution, adequate metadata, no blocking quality flags.	Proceed to focused refinement if requested.
Useful rough classification but readiness warning remains.	Record quickscan; refine only if the warning is compatible with the intended use.
Artifact review or metadata blocker.	Do not automatically refine; flag for human inspection.
Boundary solution or severe structured residuals.	Treat as triage result; inspect before stronger fitting.
Read/parse failure.	Record failure and continue the batch without corrupting other targets.

20.7 Batch output is not a calibration sample

A table of successful optimizer results is not automatically a scientifically homogeneous catalog. Spectra can differ in wavelength coverage, signal-to-noise, uncertainties, resolution knowledge, masks, and fit regions. Before comparing parameters across targets, confirm that the analysis configuration and data quality support the intended population-level comparison.

20.8 Throughput measurements

Runtime depends strongly on hardware, PHOENIX cache state, wavelength coverage, selected windows, search mode, and whether refinement is triggered. Timings from one development machine should therefore be treated as implementation smoke checks, not performance guarantees. For serious survey planning, benchmark the actual target mix in the intended environment and record the cache/hardware context.

20.9 A representative command-line wrapper

The numbered example is the preferred teaching entry point. The exact command-line options should always be checked with --help in the checkout being used:

```
python examples/example5_batch_fitting.py --help
```

A lower-level batch quickscan/refine script has also been used during validation. The important concepts to look for in the current interface are input spectra, explicit reader/product choice, quickscan versus refinement, checkpoint/report output, summary CSV, resume behaviour, and deliberate force semantics.

20.10 What to inspect after a batch

- How many spectra were read successfully, quickscanned, refined, skipped, or failed?
- Why were targets skipped? Are the reasons scientifically sensible rather than merely numerical?
- How many solutions hit model/search boundaries?
- Are high chi-square or structured-residual flags concentrated in one instrument/product group?
- Did any targets use different fit regions, resolution assumptions, or uncertainty handling?
- Do the most scientifically important targets still receive individual residual inspection?

20.11 Scaling without losing provenance

The batch wrapper should preserve the same information that matters in a single fit: reader/product identity, setup, regions, masks, wavelength/frame semantics, resolution policy, PHOENIX provenance, objective diagnostics, readiness state, and quality flags. Scaling to hundreds of spectra makes this provenance more important, not less, because manual memory can no longer reconstruct what happened to each target.

20.12 The advanced-example workflow as a whole

Stage	Question answered
Example 3 / Chapter 16	Which assumption is limiting or controlling the first PHOENIX fit?
Example 4A / Chapter 17	Can a complex Balmer-wing analysis be made explicit and reviewable?
Example 4B / Chapter 18	Does the conclusion survive plausible changes in nuisance and selection choices?
Example 6 / Chapter 19	Which multi-arm evidence should be inspected, and which subset is safe to fit?
Example 5 / Chapter 20	How can the same safeguards be preserved when the workflow is automated over many spectra?

Part V - Validation, quality, and reproducibility

Diagnostic plotting, quality assessment, external validation, and provenance.

21. Diagnostic plotting

A diagnostic plot is not decoration added after the fit. It is part of the scientific interface. A useful plot should make a specific failure mode visible, preserve the distinction between fitted and unfitted pixels, and avoid transformations that make the model appear more successful than the underlying data justify. Spyctres therefore provides several plot families rather than one universal figure.

21.1 Choose the plot from the question

Question	Useful plot type	What to inspect
What did I actually ingest?	Simple spectrum or spectrum-audit plot.	Coverage, edges, discontinuities, flux pathologies, uncertainty/mask behaviour, arm joins.
Which useful spectral regions are present?	Diagnostic-window overview.	Coverage of known stellar diagnostics and which windows are only candidates for later use.
Is one local line behaving sensibly?	Local line-fit plot.	Centroid, local continuum, width/shape mismatch, masking, and whether a simple profile is physically adequate.
Does the PHOENIX model reproduce the fitted features?	Referee-style data/model/residual plot or selected model-line windows.	Line morphology, structured residuals, fitted versus excluded pixels, continuum-adjusted model.
Are reasonable analysis variants consistent?	Fit-comparison line-window plot.	Where traces diverge, not merely which variant has the lowest scalar objective.
Does a released library product look globally consistent?	Validation plot such as the XSL payload plot.	Global spectral shape, joins, model mismatch, and whether plotting scale choices distort the comparison.

21.2 Plot the spectrum before plotting the fit

The first plot should still be the data itself. A full-spectrum plot can reveal a truncated wavelength range, pathological blue/red edge, unexpected arm discontinuity, or normalization state that is difficult to recognize after the analysis has been restricted to selected windows. If a later fit plot looks suspicious, return to the original spectrum rather than diagnosing everything in model space.

```
sp.plot_spectrum(spec)
```

Plotting cannot repair metadata

A visually plausible spectrum can still have an unknown or incorrect wavelength frame, wavelength medium, uncertainty interpretation, or resolution assumption. Plots complement the explicit metadata checks from Parts II and III; they do not replace them.

21.3 Diagnostic-window plots are maps, not masks

The diagnostic-window overlay answers a coverage question: which scientifically interesting regions are present and sufficiently usable to inspect? Shading a window does not mean every pixel in it is valid, that the region entered the objective, or that the feature is expected to constrain the same parameters in every stellar regime.

```
windows = sp.select_diagnostic_windows(spec)
sp.plot_diagnostic_windows(spec, windows)
```

21.4 Local line plots diagnose the model assumption

A local Gaussian or similarly simple line fit can be extremely informative even when it is a poor physical model. The failed Hgamma example in Part III is a good case: the large coherent residuals demonstrate that numerical convergence does not make a simple profile adequate for a broad Balmer line. The point of the plot is to reveal that mismatch, not to make the local fit look successful.

21.5 Referee-style fit plots

For PHOENIX results, a reviewer-oriented plot should show the observed data and continuum-adjusted model together with residual information at a scale where important spectral structure remains visible. A full optical spectrum compressed into one narrow axis can conceal local problems; selected windows make it easier to see whether H lines, metal features, and residual structures tell a consistent story.

```
print(result.quality_report_text())
sp.plot_fit_referee(result)
```

The plot and the quality report should be read together. The figure shows where the model fails; the report records boundary, uncertainty, readiness, and provenance information that may not be visible in the pixels.

21.6 Show excluded pixels without pretending they were fitted

A useful convention in the current plotting layer is that the model can be shown through masked or otherwise excluded regions with a different visual style. This allows the user to inspect, for example, a Balmer core that was deliberately removed from a wing fit while preserving the fact that those pixels contributed no likelihood information. The same principle applies to warning-only regions and residual diagnostics: visibility is not equivalent to inclusion.

21.7 Residuals: use structure, not only amplitude

Residual plots are most useful when they expose coherent morphology. A sequence of adjacent positive/negative residuals across a line wing carries different information from isolated bad pixels even if the maximum absolute residual is similar. Where residuals are normalized by the reported 1-sigma uncertainty, values can be interpreted as approximate sigma units only to the extent that the uncertainty model itself is valid and residuals are not strongly correlated.

Residual morphology	Possible implication
Broad smooth curvature	Continuum or flux-calibration mismatch.
Symmetric width mismatch around many lines	Resolution/LSF assumption or missing broadening physics.
Repeated centroid offsets	RV/frame/wavelength-calibration issue.
Specific metal lines only	Abundance pattern, blends, or local model inadequacy.
Balmer cores only while wings agree	Core-specific physics, activity, normalization, or deliberate core-mask boundary.
Narrow isolated spikes	Bad pixels, cosmic rays, telluric/interstellar contamination.

21.8 Plotting XSL and other multi-segment library products

Released library products need plotting choices that preserve their product semantics. XSL validation plots currently default to a global scale for full-spectrum display rather than independently normalizing every segment. Independent per-segment scaling can be useful diagnostically, but if used as the default it can hide arm-to-arm or model-versus-product mismatches. Spyctres therefore preserves XSL DR3 scaling/rest-frame semantics and does not silently re-align or re-scale the released product merely to improve the appearance of the comparison.

21.9 Plot files are part of provenance

The reporting layer can record generated figure paths so that a result report points to the diagnostic artifacts produced with it. Relative paths are preferable for a portable report. The default reporting policy also avoids leaking local absolute paths into reviewer- or web-facing JSON. If a plot is regenerated after changing masks, regions, or resolution assumptions, treat it as a new artifact associated with that new result rather than silently replacing the old evidence.

21.10 Plotting mistakes to avoid

- Do not normalize each arm independently in a global comparison unless the figure explicitly says that this is a diagnostic display choice.
- Do not omit masked pixels from view when seeing the model behaviour there is scientifically useful; distinguish them visually instead.
- Do not show only the best-looking line or wavelength interval when other fitted regions exhibit structured failure.
- Do not label a candidate DIB/telluric overlap as a detection solely because a residual falls near a catalogued wavelength.
- Do not use a scalar chi-square annotation as a substitute for residual panels or feature-level plots.
- Do not let a plotting transformation imply a physical calibration that the data product does not possess.

22. Quality assessment

Quality assessment is the process of deciding what a particular result can support. It combines pre-fit readiness with post-fit diagnostics. Spyctres intentionally does not collapse this into one universal quality score because the same spectrum can be useful for visual inspection, marginal for quick classification, and unsafe for a precise radial-velocity or atmospheric-parameter interpretation.

22.1 Readiness and result quality are different

Stage	Question	Examples
Before fitting	Is the dataset suitable for the intended inference?	Known frame? Usable uncertainties? Resolution provenance? Artifact review? Adequate windows?
During fitting	Did the numerical procedure behave sensibly?	Multiple starts, convergence, objective behaviour, parameter bounds, continuum solution.
After fitting	Does the result look scientifically credible?	Structured residuals, model-grid edge, implausible nuisance continuum, cross-line/arm disagreement, uncertainty caveats.
Across variants/standards	Is the inference robust and externally supported?	Sensitivity tests, benchmark recovery, cross-instrument consistency.

A clean numerical optimization cannot repair a pre-fit metadata blocker. Conversely, a spectrum that passes readiness can still produce a poor physical fit. Both stages need to survive review.

22.2 Start with the structured quality report

```
report = result.quality_report()
print(result.quality_report_text())
```

The structured form is useful for programmatic decisions and batch triage; the text form is useful for a human review. The report should preserve interpretation-relevant flags rather than only the optimizer status. The exact fields can evolve during alpha development, so scripts should use the documented result/report interface rather than depending on private internal dictionaries.

22.3 Quality flags are diagnoses, not grades

A quality flag should trigger a question. A grid-edge flag means “investigate what boundary is limiting the solution”; a structured-residual flag means “inspect where the model is failing”; an uncertainty warning means “do not give the objective its textbook chi-square interpretation without qualification”. A result with more flags is not automatically worse for every use, and a result with no flags is not automatically scientifically correct.

Flag or condition	What it should trigger
Optimizer failed or attempts disagree	Revisit initialization/search landscape before scientific interpretation.
Parameter at optimizer/model-grid edge	Determine whether the physical solution lies outside supported model space or whether the setup bound is too narrow.

Flag or condition	What it should trigger
High or structured residuals	Inspect line-specific morphology, continuum, LSF, contamination, and model regime.
Suspicious/missing uncertainties	Qualify chi-square and any formal covariance; consider a justified uncertainty model or error floor where supported.
Artifact review required	Inspect the affected pixels/regions; do not refine automatically in batch mode.
Ambiguous RV/frame semantics	Do not attach a physical stellar velocity interpretation until the active frame is established.
Large continuum correction	Check whether nuisance flexibility is absorbing astrophysical information.
Cross-line or cross-arm tension	Report the inconsistency and test which segment/assumption drives it.

22.4 Numerical convergence

Optimizer convergence is necessary but weak evidence. A converged solution can sit on a parameter boundary, use a continuum that compensates for model mismatch, or fit a subset of lines while failing another. Numerical robustness is stronger when multiple starting points converge to the same interior solution and the reconstructed spectra agree feature by feature. It is still not proof of physical adequacy.

22.5 Boundary-aware interpretation

Several boundaries can be active: user-supplied optimizer bounds, local search ranges, an interpolation cube, the installed PHOENIX subset, the full atmosphere grid, or a regime in which the chosen model family is physically inadequate. A result should record enough context to identify which boundary was reached. Quoting an uncertainty around a boundary-pinned optimum without stating the limitation is misleading.

22.6 Uncertainties and chi-square

Where the input errors are meaningful independent 1-sigma standard deviations, the weighted residual objective has the familiar chi-square interpretation. Real spectra often violate those ideal conditions through correlated pixels, resampling, reduction systematics, uncertain error arrays, model mismatch, continuum nuisance terms, or imposed error floors. Reduced chi-square is therefore a diagnostic, not a universal acceptance threshold.

Do not tune the analysis to chi-square = 1

An over-flexible continuum or inflated uncertainty model can make reduced chi-square look reassuring while weakening the physical inference. Conversely, a clean benchmark spectrum with extremely small formal errors can produce a large reduced chi-square for scientifically modest model imperfections.

22.7 Continuum quality is a scientific diagnostic

The per-segment multiplicative Legendre continua are nuisance parameters, but their behaviour should still be inspected. A smooth low-amplitude correction can be consistent with residual calibration shape. A high-degree correction that bends strongly around broad lines is evidence that the nuisance model may be carrying astrophysical information. Continuum sensitivity should be assessed through the bounded variants described in Chapter 18.

22.8 Quality in batch analysis

Batch processing makes conservative quality gates more important. The current workflow allows quicklook triage on suspicious spectra but can skip focused refinement when artifact flags, structured residuals, or metadata ambiguity make a stronger interpretation unsafe. This prevents an automated pipeline from converting poor inputs into apparently precise catalog values simply because an optimizer returned numbers.

22.9 No universal acceptance thresholds yet

Spyctres 0.5.0a1 does not yet have scientifically calibrated universal recovery thresholds for Teff, log g, [Fe/H], RV, reduced chi-square, or residual morphology across all supported data. Thresholds must eventually be tied to stellar regime, spectral coverage, S/N, resolution, and model adequacy. Until that calibration is complete, quality flags should remain transparent and conservative rather than being presented as a validated probability of correctness.

22.10 Minimum quality-assessment statement

For a result intended for scientific communication, summarize at least: the intended inference; readiness status; key quality flags; whether parameters are interior to model/search boundaries; the dominant residual structures; uncertainty caveats; the adopted resolution/LSF approximation; continuum treatment; and whether bounded stability tests or external validation support the result.

23. External validation and standards

A fitting package cannot validate itself merely by fitting the data on which its workflow was developed. External validation asks whether the complete chain - ingestion, wavelength semantics, forward model, nuisance treatment, optimization, and reporting - behaves sensibly on spectra with independent reference information or on deliberately controlled synthetic cases. Spyctres already contains several layers of this validation, but the external network is not yet complete.

23.1 A validation ladder

Level	What it tests	Current Spyctres role
Unit and convention tests	Individual transforms and data semantics.	Mask polarity, uncertainty conversion, aliases/readers, wavelength/frame utilities, RV sign, LSF undersampling behaviour.
Synthetic / same-model recovery	Whether the implemented forward model and optimizer can recover parameters under controlled assumptions.	Used in validation scaffolds and optional injection/recovery checks; useful for catching implementation failures, but not model inadequacy.
Benchmark-star recovery	Whether fitted parameters agree with independent stellar reference values.	Gaia FGK Benchmark Stars subset and validation runner.
Released-library validation	Whether realistic calibrated/reduced library products are handled correctly.	XSL validation with product semantics preserved; advanced PEPSI regression material.
Instrument stress cases	Whether real artifacts, uncertain continua, masks, and multi-arm complications are exposed rather than hidden.	Gaia21ccu X-SHOOTER examples and reviewed-analysis scaffold.
Cross-instrument network	Whether the same standards are recovered consistently across instruments.	Broader PEPSI/HARPS/NARVAL/UVES-POP/ELODIE/CARMENES overlap validation remains incomplete/planned.

23.2 Synthetic recovery: necessary but not sufficient

A same-model injection/recovery test generates data from the same model family and transformation machinery used by the fitter, optionally adding noise or masking, and asks whether the input parameters are recovered. This is excellent for detecting RV sign mistakes, interpolation bugs, masking errors, LSF mistakes, optimizer failures, and parameter degeneracies under controlled conditions.

It is intentionally an easy scientific universe. Passing same-model recovery does not establish that PHOENIX is adequate for a real star, that the instrument has a Gaussian constant-R LSF, or that the data reduction uncertainties are correct. Synthetic recovery is therefore a verification layer, not a replacement for empirical standards.

23.3 Gaia FGK Benchmark Stars

The bundled Gaia FGK Benchmark Stars material is the clearest current external parameter-recovery scaffold. These are benchmark-library spectra of well-studied stars, not Gaia RVS spectra. The distinction matters: success tests the stellar-analysis workflow on a reference spectrum; it does not validate the Gaia RVS instrument or its reduction pipeline.

The benchmark reference Teff, log g, metallicity, and related information must remain independent of the fit. Spyctres deliberately avoids feeding those reference values back as hidden priors. The small bundled subset carries manifest/checksum information so the validation inputs are identifiable and reproducible.

Avoid circular validation

A benchmark fit is not a recovery test if the benchmark answer is used to constrain the same parameters strongly enough that the output is predetermined. Reference parameters should be compared after fitting, or used only in a clearly labelled experiment whose purpose is different from independent recovery.

23.4 Stress stars versus ordinary recovery statistics

Peculiar stars, strong activity, rapid rotation, abundance anomalies, hot-star regimes with missing model physics, or spectra with severe calibration problems should not be allowed to dominate an ordinary “accuracy” statistic without explanation. They are scientifically valuable stress tests precisely because failure can reveal the validity domain. A mature validation report should therefore separate ordinary reference-star recovery from known model-mismatch or stress cases rather than averaging them into one opaque performance number.

23.5 XSL validation

Maintained Example 7 is the XSL validation workflow. The X-shooter Spectral Library provides a complementary validation mode: realistic released spectra with broad wavelength coverage and well-defined product semantics. The DR3 products used in the current workflow are air-wavelength, stellar-rest, and merged/scaled according to the XSL release. PHOENIX HiRes is natively vacuum-wavelength, so Spyctres converts the PHOENIX wavelength grid to the target XSL segment medium before RV shifting, LSF convolution, and resampling. Validation therefore tests both that this product semantics is respected and that PHOENIX plus the adopted nuisance/LSF treatment gives a plausible spectral comparison. It should not re-align, re-convert, or independently rescale the released XSL data merely to improve agreement.

For full-spectrum XSL plots, the current default uses global scaling. Per-segment scaling remains useful as a diagnostic mode, but a globally scaled plot is the more honest first view of whether the released spectrum and model are mutually consistent across the full range.

XSL is external real-spectrum validation, but its broad spectral shape is not completely independent of PHOENIX: PHOENIX models contributed to the DR3 extinction/scaling procedure. Global spectral-shape agreement should therefore not be treated as a wholly independent confirmation of the PHOENIX atmosphere family. Localized line morphology, residual structure, correct product-semantics handling, and consistency across independently informative features provide more independent validation evidence than global SED agreement alone.

Figure 23.1 is an appropriate example. It shows a globally scaled XSL comparison for the B9 V standard X0245. The overall agreement is good across most of the wavelength range, while the blue end and specific line cores retain visible residual structure. This is exactly the kind of information that would be hidden if each arm were silently rescaled for cosmetic agreement.

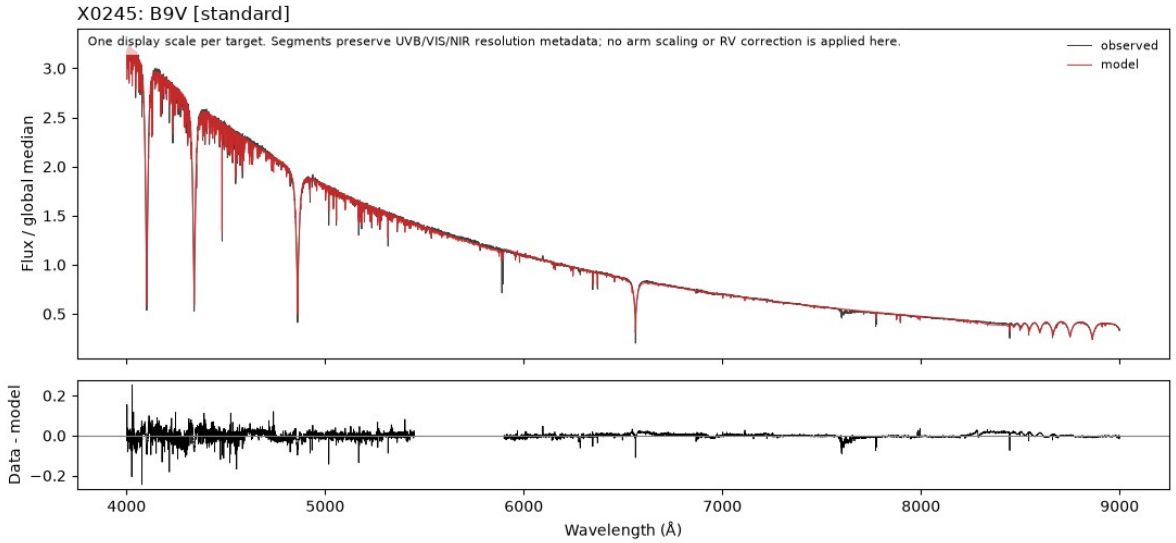

Figure 23.1. XSL validation overview for X0245 (B9 V standard). The released spectrum and model are shown on one global flux scale, with a residual panel below. This is the scientifically appropriate first-look display for a merged library product.

23.6 X-SHOOTER as a realistic stress case

The Gaia21ccu X-SHOOTER examples serve a different role from the clean benchmark stars. They expose artifact handling, uncertain continuum shape, line-core sensitivity, resolution assumptions, residual catalog overlaps, and multi-arm calibration differences. The point is not that this one star calibrates the package. The point is that a guarded workflow should make these real-world problems visible and prevent them from being silently converted into confident parameters.

23.7 PEPSI and legacy regression

Maintained Example 8 is the PEPSI legacy regression workflow. It is a compatibility check, not the recommended starting point for new analyses. The current source state retains this PEPSI material to check that modern refactoring does not silently break established line-fitting or product-handling behaviour. Because PEPSI product frame/resolution semantics can vary, such tests should retain the original product provenance and should not infer corrections merely from filename conventions.

Figure 23.2 is appropriate as a regression-style diagnostic. It does not by itself establish broad external accuracy, but it does show that the present code path still reproduces sensible local line-window behaviour across several features. The agreement is strongest in the better-behaved windows, while the scatter and uncertainty structure in some of the redder windows remind the reader that regression material is not the same thing as a universal validation benchmark.

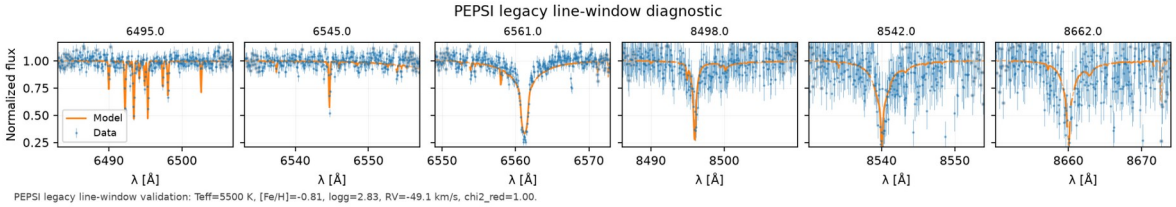

Figure 23.2. PEPSI legacy line-window diagnostic. Selected local windows are compared in normalized flux. This is best interpreted as regression evidence that the refactored workflow still reproduces established line-window behaviour, not as a complete cross-instrument validation summary.

23.8 Negative and adversarial validation cases

A trustworthy scientific package should test expected failures as well as successes. Useful negative cases include spectra with missing uncertainties, deliberately unknown wavelength frames, insufficient resolution metadata, aggressive masks, parameter values at grid edges, LSF undersampling, incorrect RV sign, and fit regions containing little diagnostic leverage. The expected result is often a warning, blocked interpretation, or degraded recovery rather than a successful fit.

Negative test	Scientifically desirable behaviour
Unknown wavelength frame	Allow limited inspection where safe; block or qualify physical RV interpretation.

Negative test	Scientifically desirable behaviour
No usable uncertainties	Do not pretend a textbook chi-square likelihood is available; expose the limitation.
Model truth outside installed grid	Do not silently extrapolate into an apparently precise solution.
LSF narrower than sampling can resolve	Warn about undersampling rather than applying a numerically misleading convolution.
Heavily contaminated diagnostic window	Flag/annotate or exclude with explicit provenance; do not let nuisance continuum hide the issue.
Overly large Balmer core mask	Expose information loss and test sensitivity rather than accepting a cosmetically cleaner fit.

23.9 Cross-instrument validation is still incomplete

The present validation network is a strong alpha scaffold, not a finished calibration campaign. Broader overlap testing with PEPSI, HARPS, NARVAL, UVES-POP, ELODIE, CARMENES, and other standards remains important. Ideally the same reference stars should be analysed through multiple independent products so that model error can be separated from reader/instrument/LSF effects.

23.10 What a validation report should show

1. Define the sample and state which cases are ordinary benchmarks versus stress/model-mismatch cases.
2. Record the exact spectra, checksums or release identifiers, reader/product semantics, and reference-parameter sources.
3. Fit without using the reference answer as a hidden prior unless the experiment explicitly studies prior-assisted inference.
4. Report recovered minus reference values for Teff, log g, [Fe/H], and RV together with grid-edge and quality flags.
5. Show feature-level residual plots for representative successes and failures, not only aggregate parameter statistics.
6. Stratify performance by stellar regime, S/N, wavelength coverage, resolution, and instrument when the sample becomes large enough.
7. Keep acceptance thresholds provisional until they are empirically calibrated on an adequate sample.

24. Reproducibility and provenance

Reproducibility in Spyctres has two levels. Computational reproducibility asks whether another user can reconstruct the same calculation. Scientific reproducibility asks whether the conclusion remains defensible when reasonable analysis choices or independent data are considered. The structured setup, report, and validation layers are designed to support both, but neither is automatic.

24.1 Preserve the data product, not only the filename

A filename is often insufficient provenance. Record the product family/reader, observation or library release information, wavelength/flux units, active frame and wavelength medium, uncertainty representation, mask provenance, resolution/LSF information, and flux-calibration/normalization state. For shared or mutable input files, an optional SHA256 checksum can identify the exact bytes used in the analysis.

24.2 Preserve the exact FitSetup

Automatic suggestions become reproducible only when their output is stored. FitSetup therefore has inspectable summaries and a stable setup hash, and the exact reviewed setup can be embedded in the fit result/report. This records fit windows, bounds/seeds, search policy, continuum choices, adopted resolution assumptions, and readiness interpretation rather than requiring a collaborator to reverse-engineer defaults months later.

```
setup = sp.suggest_fit_setup(spec)
print(setup.summary_text())

result = sp.fit_stellar_spectrum(
    spec,
    model="phoenix",
    setup=setup,
)
```

24.3 Preserve model-library provenance

A PHOENIX result depends on the installed model library and the subset/cache used to build the interpolator. Reports can therefore include compact PHOENIX cache/model manifest information and template-axis summaries without exposing local template paths. This distinction matters if two systems have different installed subgrids or if the cache schema changes between package versions.

24.4 Preserve software provenance

A versioned report can record the Spyctres version and, when available, the Git commit. For development/alpha analyses this is especially important because behaviour may evolve between commits without a formal release. Reproducible scientific hand-off should also record the Python environment or at least the important package versions when the result is meant to be regenerated exactly.

24.5 Use the versioned report envelope

```
result.save_report_json("fit_report.json")

# A compact backwards-compatible result JSON may also be available:
result.save_json("fit.json")
```

The versioned report is the preferred scientific hand-off format because it can include schema/type information, Spyctres version, Git commit, setup hash, result payload, provenance summary, PHOENIX manifest, uncertainty caveats, mask/readiness/resolution policy, generated plot paths, and optional input checksum. The compact JSON remains useful for older scripts but contains less context.

24.6 Path sanitization is deliberate

Reports intended for collaborators, reviewers, web interfaces, or Django should not expose private workstation paths to PHOENIX caches, input directories, or generated files. Spyctres therefore sanitizes local paths by default and can record generated figure paths relative to the report location. Portability and privacy are part of reproducibility: another user needs to know what resource was used, not how one developer happened to mount a disk.

24.7 Generated figures belong to the result record

If a plot is part of the evidence used to accept or reject a fit, preserve it with the corresponding result and setup rather than regenerating an unlabeled figure later. A plot made after changing a mask, continuum degree, or resolution assumption belongs to that variant. Keeping generated artifact paths in the report reduces the risk of attaching a visually similar but scientifically different figure to the wrong result.

24.8 Atomic machine-readable artifacts

The current workflow consolidates shared JSON/CSV writing and validation-artifact handling so that examples and scripts do not each invent slightly different serialization behaviour. For long batch/validation jobs, atomic writes and checkpoint semantics reduce the risk of leaving a partially written file that looks like a valid completed result.

24.9 A reproducible paper/report methods statement

Record	Minimum information to report
Input data	Instrument/library product, reader, release/reduction identity, wavelength coverage, flux state.
Wavelength semantics	Air/vacuum medium, active frame, whether stellar-rest/barycentric corrections were already applied.

Record	Minimum information to report
Uncertainties and masks	Source of 1-sigma errors, error floors/scales if any, archive/user/telluric/DIB/core masks, warning-only regions.
Resolution/LSF	Recorded resolution provenance, adopted R/LSF, and the fact that current production broadening is constant Gaussian where used.
Model grid	PHOENIX library/version/subgrid or manifest sufficient to identify the interpolation domain.
Fit configuration	Fit windows, Teff/log g/[Fe/H]/RV bounds and initialization policy, continuum degree, search mode, FitSetup hash.
Result qualification	Quality/readiness flags, grid-edge status, dominant residuals, uncertainty caveats, exploratory overrides.
Robustness/validation	Bounded systematic variants and external benchmark or cross-instrument checks relevant to the claimed conclusion.
Software	Spyctres version and Git commit where applicable; important environment information for exact reruns.

24.10 Reproducibility does not mean freezing every path

A robust record identifies the scientific inputs and transformations, not every ephemeral implementation detail. Absolute filesystem locations, temporary cache paths, or notebook cell numbers are weak provenance because they cannot be replayed elsewhere. Stable identifiers, checksums, setup hashes, release names, model manifests, and explicit parameter choices are stronger.

24.11 Scientific reproducibility requires sensitivity information

Repeating the exact same deterministic fit and obtaining the same optimum demonstrates computational reproducibility. It does not show that the answer is robust to a plausible alternative continuum degree, core mask, LSF, or diagnostic-window choice. For serious inference, store the baseline together with the bounded variants that support the claim and make the remaining model limitations explicit.

24.12 A final reproducibility checklist

1. Can another user identify the exact input spectrum or release product?
2. Are wavelength medium, frame, correction status, uncertainty convention, mask provenance, and flux state recorded?
3. Is the adopted resolution/LSF assumption distinct from merely available resolution metadata?
4. Is the exact FitSetup, including fit regions and continuum/search choices, preserved with a stable hash?
5. Can the PHOENIX model grid/cache provenance be identified without relying on a private local path?
6. Are Spyctres version and development commit recorded where relevant?
7. Are quality flags, readiness caveats, grid-edge status, and uncertainty limitations preserved?
8. Are the diagnostic figures linked to the correct result/variant?
9. Have bounded systematic tests and external standards been recorded for conclusions that require them?
10. Could a collaborator understand not only how the number was produced, but why it was considered scientifically usable?

End point of Part V

At this stage the user should be able to distinguish a visually convincing fit from a qualified fit, distinguish a qualified fit from a validated method, and package the calculation so that another researcher can reconstruct both the numerical setup and the scientific caveats.

Part VI - Troubleshooting and reference

Troubleshooting, the curated public API, scientific conventions, PHOENIX setup, reproducibility, and compatibility.

26. Troubleshooting

Most difficult Spyctres failures are not single software bugs. They are mismatches between layers: the wrong Python environment, an unconfigured model library, a reader applied to the wrong product, incomplete wavelength metadata, an inverted mask, an inappropriate LSF assumption, a fit outside the supported atmosphere regime, or a numerical result that is being asked to support more than the data justify. Troubleshooting should therefore proceed from the outside inward.

26.1 First identify the failure layer

Layer	Typical symptom	First action
Environment / import	Import fails, wrong checkout is imported, dependency error appears before any spectrum is read.	Confirm Python executable, environment, and Spyctres import path.
PHOENIX configuration	Inspection works but PHOENIX fitting cannot find wavelength grid/templates, or cache/grid mismatch appears.	Run the setup checker; verify the discovered root, wavelength file, and template axes.
Reader / ingestion	Wrong coverage, implausible flux, almost all pixels invalid, missing expected errors, reader alias rejected.	Confirm the actual data product and reader; inspect native arrays and reader metadata.
Scientific metadata	Spectrum looks plausible but RV, line positions, or resolution interpretation is ambiguous.	Check air/vacuum medium, active frame, correction history, flux state, and LSF provenance.
Readiness / masking	Fit is blocked or marked exploratory; artifact review is required.	Inspect which regions are warning-only, excluded, or invalid and why.
Numerical fitting	Optimizer fails, attempts disagree, result depends strongly on starting point, or parameters pin to bounds.	Inspect setup/bounds, RV initialization, grid support, and multistart diagnostics.
Model adequacy	Optimization succeeds but residuals remain structured or different lines disagree.	Test continuum/LSF/mask/line-selection sensitivity and consider model-family limitations.
Output / reporting	Plot missing, report lacks provenance, wrong figure appears with a result, path issues.	Inspect result object, generated artifact paths, report JSON, and output directory policy.

26.2 Establish which Spyctres you are actually running

A surprisingly common development failure is using a different Python interpreter or Spyctres checkout from the one you think is active. Before investigating scientific behaviour, establish the executable and import path.

```
which python
python --version
python -c "import Spyctres as sp; print(sp.__file__)"
```

If the printed path is not the intended development checkout or installed environment, fix that first. Re-running a fit in the wrong environment only produces additional misleading evidence.

Alpha-version caution

A notebook kernel can remain attached to an older environment after the shell has been switched. Check the Python executable from inside Jupyter as well as from the terminal when behaviour differs between them.

26.3 Use doctor before debugging the fitter

Start with the current compact CLI diagnostic. If PHOENIX is intentionally not configured, check the core installation with:

```
spyctres doctor --skip-phoenix
```

For deeper development diagnostics, the lower-level setup-checker script can inspect the environment, PHOENIX discovery, and representative spectrum ingestion:

```
python scripts/check_spyctres_setup.py --help

python scripts/check_spyctres_setup.py \
  --spectrum examples/data/TOO_Gaia21ccu_SCI_SLIT_FLUX_MERGE1D_UVB.fits \
  --instrument xshooter
```

The lower-level setup checker also supports PHOENIX-skipping modes; consult its `--help` output for the exact option in the current checkout. Do not diagnose an intentionally absent external model library as a reader or optimizer failure.

Use `spyctres doctor` as the preferred compact diagnostic and the setup-checker script when detailed development information is needed. These diagnose configuration; they do not replace spectrum metadata, readiness, or residual review.

26.4 Import errors and dependency failures

Symptom	Likely cause	Recommended response
import Spyctres fails	Wrong environment, incomplete installation, incompatible dependency, or import-time legacy dependency issue.	Confirm interpreter and import path; run pip check; reinstall the intended environment rather than modifying scientific code.
pysynphot / stsynphot / pkg_resources warning or failure	Legacy synthetic-photometry compatibility layer and current alpha packaging.	If using only the modern PHOENIX workflow, separate this from a fitting failure. Legacy dependencies are not evidence that the PHOENIX algorithm is broken.
Matplotlib/headless failure	Display/backend/configuration problem.	Use non-interactive plotting for batch/headless work and confirm that basic numerical workflow runs before changing analysis settings.
Optional feature fails only when invoked	Missing optional dependency.	Install the relevant optional stack only if that workflow is actually required; do not broaden the core environment without need.

26.5 PHOENIX cannot be found or loaded

PHOENIX fitting requires a local HiRes library. The model data are not a small bundled Python resource. A valid root must contain the native wavelength grid and discoverable stellar templates in the expected PHOENIX layout. If spectrum inspection works but fitting fails at model loading, debug the PHOENIX layer independently.

1. Run the setup checker and confirm which PHOENIX root Spyctres resolved.
2. Confirm that the native wavelength FITS file exists and is readable.
3. Confirm that metallicity directories and template filenames match the supported PHOENIX-ACES HiRes layout.
4. Confirm that the installed grid actually contains the Teff/log g/[Fe/H] region required by the fit.
5. If a cache error occurs after the model library changed, rebuild or invalidate the cache using the supported mechanism in the checkout rather than editing cache files manually.
6. Record the final model/cache manifest in the result report for reproducibility.

Installed grid != nominal grid

The parameter space available on one workstation is defined by the templates actually installed and accepted by the interpolator. A nominal PHOENIX model family can be broader than the local usable subgrid.

26.6 The reader name is rejected

Use discovery rather than guessing aliases. Reader selection is product-specific, not merely instrument-specific.

```
import Spyctres as sp
print(sp.list_instruments())
print(sp.get_instrument_info("xshooter"))
help(sp.read_spectrum)
```

The read-only command-line interface can also inspect the supported readers and a specific file:

```
spyctres readers
spyctres reader-info xshooter_merge1d
spyctres inspect-spectrum my_spectrum.fits --reader xshooter_merge1d
```

If a file comes from a supported instrument but a scientifically different reduction/product type, do not force it through a convenient reader. Use the reader that matches the actual product, or treat it as an external/generic spectrum with explicit metadata. The compatibility `--instrument` form remains available, but `--reader` is the preferred language for new workflows.

26.7 The spectrum loads but looks wrong

Observed problem	Check before changing anything
Wrong wavelength range or order	Native file columns/extensions, wavelength units, sorting, reader/product match.
Flux is negative or wildly unstable at an edge	Whether this is a real low-S/N/calibration edge rather than a parser failure; inspect the original product.
Almost all pixels are invalid	Mask polarity, archive quality bits, non-finite errors, positive-error requirement, wrong extension.
No uncertainty array	Whether the product actually provides variance/ivar/errors and whether the reader supports that representation.
Arm flux levels disagree	Slit losses, arm scaling, flux calibration and product semantics; do not silently rescale at ingestion.
Spectrum appears shifted	Active frame, air/vacuum medium, whether barycentric/stellar-rest correction was already applied.

```
import numpy as np
print(type(spec))
print("coverage [A]:", np.nanmin(spec.wave), np.nanmax(spec.wave))
print("pixels:", spec.wave.size)
print("valid:", np.count_nonzero(spec.mask))
print("uncertainties:", spec.err is not None)
print("medium:", spec.wave_medium)
print("frame:", spec.wave_frame)
print("metadata keys:", sorted(spec.meta))
```

26.8 The wavelength frame or RV interpretation is unclear

Do not solve a metadata ambiguity by letting RV float and then naming the fitted shift a systemic velocity. The numerical alignment parameter and its physical interpretation are separate. Positive Spyctres RV corresponds to redshift/recession, but the meaning of the number depends on the active wavelength frame.

- Topocentric: observer motion remains part of the active wavelength scale unless corrected elsewhere.
- Heliocentric/barycentric: interpret the fitted shift relative to that corrected scale, subject to calibration/model systematics.
- Stellar-rest: the fitted RV is normally residual alignment, not the original stellar systemic velocity.
- Unknown: a numerical shift may be found, but a physical RV interpretation should remain blocked or explicitly ambiguous.

A correction value stored in a FITS header may be provenance for a correction that was already applied. Never apply it a second time solely because the keyword exists.

26.9 The resolution is missing, uncertain, or apparently wrong

Separate three questions: what resolution information is available in the product, what resolution the analysis adopted, and what broadening the forward model actually applied. The current production likelihood uses a constant Gaussian LSF when broadening is requested and scientifically specified.

Wavelength-dependent/non-Gaussian LSF metadata may be preserved without being active in the likelihood.

Symptom	Likely interpretation	Response
Reader has no R/LSF	Product does not provide enough information.	Leave unknown or adopt an explicitly justified assumption; test sensitivity.
Nominal slit-based R differs from line widths	Nominal instrument resolution is not the effective exposure LSF.	Treat nominal R as provenance, not a measurement; compare plausible R variants.
Model lines systematically too narrow/broad	LSF, rotation/macroturbulence, sampling, or model mismatch.	Check all broadening assumptions; do not attribute every width mismatch to instrumental R.
LSF undersampling warning	Requested convolution cannot be represented reliably on the working grid.	Do not suppress the warning without checking sampling and requested resolution.
SDSS wdisp exists but fit uses constant R	Recorded wavelength-dependent LSF is not automatically applied.	Document the approximation; do not claim wdisp was used in the likelihood.

26.10 Readiness blocks the fit or interpretation

A readiness blocker is usually a scientific safeguard, not an inconvenience to remove. Inspect the intent and the reported reason. The same spectrum may be adequate for visual inspection or quicklook classification while unsafe for a stronger atmospheric-parameter or RV claim.

- Artifact review required: inspect the affected region and mask provenance; decide whether exclusion or warning-only treatment is justified.
- Unknown/ambiguous frame: establish the product semantics before attaching a physical RV interpretation.
- Missing or suspicious uncertainties: qualify the likelihood; do not invent precision.
- Resolution uncertainty: state the assumption and test a plausible range if the conclusion depends on it.
- Insufficient useful pixels/windows: broaden the observation or change the scientific question rather than forcing a parameter estimate.

An explicit exploratory override can permit computation for diagnosis. It does not remove the original blocker or upgrade the result to reviewed scientific status.

26.11 The optimizer fails or different starts disagree

1. Inspect the FitSetup summary and confirm that bounds and initial values are physically sensible and inside the installed PHOENIX support.
2. Check RV alignment first. Severe wavelength misalignment can make atmospheric initialization meaningless.
3. Inspect whether only one start reaches a much lower objective. This can indicate multiple basins or a difficult search landscape.
4. Check whether a parameter is pinned to an optimizer bound or model-grid edge.
5. Change one assumption at a time: search budget, fit regions, continuum degree, core mask, or resolution. Do not simultaneously change several ingredients and attribute the improvement to one of them.
6. If a stronger search finds a different solution, compare the spectral reconstruction and residuals. A lower objective is not automatically more physical.

26.12 The fit converges but chi-square is very high

High reduced chi-square is not synonymous with optimizer failure. It can arise from underestimated formal uncertainties, correlated noise, flux-calibration residuals, incorrect LSF assumptions, abundance differences, missing physics, or a few coherent artifacts. Conversely, forcing chi-square_red to one by rescaling uncertainties is not automatically justified.

Residual pattern	Investigate first
Broad smooth residual curvature	Continuum/calibration state and continuum-degree sensitivity.
Symmetric line-width mismatch	Resolution/LSF and missing broadening physics.
Line centers shifted together	RV/frame/wavelength calibration.
Only selected metal lines fail	Abundance pattern, blends, model limitations.

Residual pattern	Investigate first
Balmer cores fail while wings agree	Core physics/activity/normalization; explicit core masking for wing analysis.
Narrow isolated spikes	Archive mask, bad pixels, cosmic rays, telluric/interstellar features.

Do not optimize the statistic cosmetically

A fit that looks statistically cleaner only because the error bars were inflated or the continuum was made excessively flexible can contain less physical information than the original fit.

26.13 The fit lands on a grid edge

A boundary solution is a diagnostic, not a measurement of the boundary value. Determine which boundary was reached: user setup, local search, interpolation cube, installed subgrid, full atmosphere grid, or physical validity of the model family. If the preferred direction lies outside model support, report that limitation rather than silently extrapolating.

26.14 One line or arm disagrees with the others

Cross-line or cross-arm tension should usually be preserved long enough to diagnose it. Possible causes include arm-specific calibration, resolution differences, telluric contamination, stellar activity, abundance effects, line-core physics, or genuine model inadequacy. The multi-arm workflow deliberately allows broad inspection while restricting the actual fit to a conservative subset.

1. Plot the segments/lines separately using the same fitted atmospheric solution.
2. Check frame and wavelength-medium consistency across segments.
3. Check per-arm resolution and continuum treatment.
4. Run leave-one-line-out or retained-window sensitivity tests.
5. Report persistent tension rather than hiding it through per-arm rescaling or a higher-degree continuum.

26.15 Diagnostic windows look wrong or unhelpful

Diagnostic-window selection is coverage-driven and advisory. A selected window means the spectrum covers a potentially useful diagnostic region; it is not a promise that the feature is strong, uncontaminated, or appropriate for the intended stellar regime. If the suggestions are not useful, inspect the full spectrum and construct scientifically justified regions explicitly. The fit window is a user-reviewed decision, not an automatic consequence of the diagnostic selector.

26.16 Local line fitting gives a nonsensical result

A local Gaussian or other simple line profile is a diagnostic model. Broad Balmer lines, blends, rotation-dominated features, molecular bands, and strongly asymmetric profiles can be poor matches. If the fit converges but the residual morphology is obviously wrong, the correct conclusion is that the local profile is inadequate for that feature. Do not interpret the fitted centroid/width/equivalent width as a precise physical measurement without checking model adequacy.

26.17 Batch refinement is skipped

In the current batch philosophy, skipping focused refinement can be the correct outcome. Quickscan is allowed to triage spectra that later fail readiness or quality gates. Artifact flags, structured residuals, ambiguous RV semantics, or other blockers can prevent the pipeline from converting a suspicious spectrum into an apparently precise catalog entry. Inspect the stored quickscan report and reason for the gate before using force/override options.

26.18 A plot is misleading or appears to hide a problem

- Confirm whether masked pixels are merely visually suppressed or actually excluded from the likelihood.
- For XSL or broad multi-arm views, prefer global scaling when evaluating cross-arm consistency; independent per-segment normalization can hide real disagreement.
- Check whether a dashed model continuation marks pixels outside the fit. Visibility of a model through a mask does not imply those data were fitted.

- If a full-spectrum plot is too compressed, use selected line windows plus residual panels rather than declaring the global fit satisfactory.

26.19 The report JSON or saved artifacts are incomplete

Prefer the versioned report envelope for scientific hand-off. It can preserve the result payload, setup hash, model/cache provenance, uncertainty caveats, readiness/resolution/mask policies, software version, generated plot paths, and optional input checksum. The compact backwards-compatible JSON contains less context.

```
result.save_report_json("fit_report.json")
# Compatibility / compact output where needed:
result.save_json("fit.json")
```

Absolute local paths are sanitized by default for portable reviewer/web hand-off. If a collaborator needs to reproduce the run, share stable product identifiers, checksums, model manifests, setup hashes, and documented configuration rather than private mount paths.

26.20 A minimal diagnostic bundle for asking for help

When reporting a Spyctres problem, provide enough context to separate software failure from data/model ambiguity. A useful bundle contains:

- Spyctres version and, for a development checkout, Git commit if available;
- Python version and import path;
- the exact reader/product family and a short description of the input data;
- setup-checker output relevant to the failure;
- wavelength medium/frame, uncertainty status, valid-pixel fraction, and resolution provenance;
- FitSetup summary/hash for fit problems;
- quality report and one diagnostic data/model/residual plot;
- the complete exception traceback for software errors, not only the last line;
- a minimal reproducible case or bundled-example comparison where possible.

Best troubleshooting question

At what earliest layer does the result stop matching what should be true? Answering that question is usually faster than changing fit parameters until the symptom disappears.

28. Public API reference

The package root exposes a sizeable current alpha surface. This reference is intentionally curated: it documents the functions and classes that belong to the ordinary user path plus selected advanced tools that are directly used by the maintained examples. It is not a promise that every symbol currently re-exported by Spyctres.__all__ will remain at the package root after the structural refactor. Use the help registry and Python help() for the exact signature in the checkout used for an analysis.

28.1 Discover the API from the package itself

```
import Spyctres as sp

print(sp.format_public_api_guide())
print(sp.list_public_function_groups())
print(sp.describe_public_function_group("fitting"))
print(sp.list_public_functions("fitting"))
print(sp.format_public_function_help("fit_stellar_spectrum"))
help(sp.fit_stellar_spectrum)
```

The curated help registry exists so that users do not need to learn the internal module tree. Advanced and compatibility functions should remain discoverable without competing with the main workflow.

28.2 Core containers and input

Object / function	Use	Scientific note
SpectrumSegment	Canonical one-dimensional spectrum container.	Carries wavelength/flux, optional 1-sigma errors, valid mask, medium/frame, name, metadata, and resolution provenance.
SpectrumCollection	Ordered set of segments for joint/multi-arm analysis.	Keeps segments distinct so each can retain its own sampling, resolution, calibration, and nuisance continuum.
ResolutionDescriptor	Represents resolution/LSF information in the current I/O layer.	Available metadata is not the same as broadening applied by the fitter.
InstrumentInfo	Structured discovery information for reader/instrument families.	Use for supported aliases and product-oriented discovery.
read_spectrum(path, reader=...)	Primary product-aware ingestion entry point.	Prefer reader= because the same instrument can produce scientifically different products.
list_instruments()	List discoverable instrument/reader families.	Prefer discovery over hard-coding aliases.
get_instrument_info(name)	Describe one supported family and aliases.	Useful before deciding which product reader to use.

28.3 Inspection and diagnostic windows

Function	Primary purpose	Important limitation
plot_spectrum(...)	Full-spectrum orientation/inspection.	A plot does not establish frame, uncertainty, or LSF semantics.
select_diagnostic_windows(...)	Suggest covered stellar-diagnostic regions.	Coverage-driven and advisory; not an automatic fit-region selector.
plot_diagnostic_windows(...)	Visualize suggested windows in spectral context.	Shading does not mean every pixel is fitted or valid.
list_known_lines(...)	Discover line names/aliases known to local-line tools.	Catalogue presence does not imply suitability for atmospheric inference.

28.4 Masks, preprocessing, and readiness

Function	Primary purpose	Convention / caution
build_mask(...)	Construct a reviewed mask from supported sources/policies.	Keep provenance for archive, telluric, DIB, user, and other exclusions.
exclusion_mask(...)	Build/represent explicit exclusion semantics.	For exclusion masks, True means reject; do not confuse with SpectrumSegment.mask.

Function	Primary purpose	Convention / caution
audit_spectrum_for_fit(...)	Intent-aware pre-fit audit.	A spectrum can be usable for one intent and blocked for another.
analysis_readiness_audit(...)	Reviewed-analysis readiness layer used in maintained examples.	Does not certify publication quality; it exposes unresolved blockers and warnings.

Older names related to publication-readiness may remain for compatibility in some revisions. New manual examples prefer analysis-readiness terminology so that the package does not imply that software alone can certify a publication result.

28.5 Fit setup and standard fitting

Object / function	Primary purpose	Important behaviour
FitSetup	First-class reviewed fit configuration.	Stores regions, bounds/initial values, search policy, continuum choices, readiness interpretation, and stable setup hash.
suggest_fit_setup(spec, ...)	Propose a first-pass FitSetup without running PHOENIX.	Review the summary before fitting; suggestions are not hidden defaults.
fit_stellar_spectrum(spec, model="phoenix", setup=...)	Main modern stellar-spectrum fitting entry point.	Transforms PHOENIX model into observational space and returns a structured result.
fit_phoenix_spectrum(...)	More PHOENIX-specific fitting entry point.	Useful for expert/compatibility code; the higher-level fit_stellar_spectrum workflow is usually clearer.
classify_spectrum(...)	Friendly exploratory classification-oriented alias/path.	Not a formal MK classification certificate.
compare_fits(...)	Compare alternative fitted results.	Use to understand sensitivity, not merely to choose the lowest objective.
build_fit_collection_from_windows(...)	Create retained multi-segment/window collection for fitting.	Supports “inspect broadly, fit narrowly” without manual arm/window misalignment.

28.6 Local line diagnostics

Object / function	Purpose	Interpretation
LineSpec	Describes a local spectral feature/window.	Diagnostic definition, not an atmosphere model.
LineFitConfig	Configuration for local profile fitting.	Continuum/profile assumptions must suit the feature.
LineFitResult	Structured result of one local line fit.	Convergence does not prove physical adequacy.
fit_line(...)	Fit one known/custom local feature.	Useful for centroid, local continuum and morphology diagnostics.
fit_lines(...)	Apply local fitting to several lines.	Compare consistency rather than averaging incompatible failures.
compare_line_fits(...)	Summarize several line-fit results.	Useful for diagnostic comparisons.
plot_line_fit(...)	Plot local data, model and residual behaviour.	Residual shape is often more informative than local chi-square.

28.7 Plotting fitted spectra

Function	Use	What it should reveal
plot_fit_referee(result, ...)	Reviewer-style model/data/residual diagnostic.	Feature-level fit quality, excluded pixels, structured residuals.
plot_model_line_windows(...)	Show fitted model around selected known lines.	Per-line morphology and behaviour through excluded regions.
plot_fit_comparison_line_windows(...)	Overlay baseline and stability variants in selected line windows.	Sensitivity to continuum, masks, line selection, resolution, etc.
plot_xsl_validation_payload(...)	Display XSL validation comparison.	Use global scaling for honest full-spectrum comparison unless explicitly diagnosing segment scaling.

28.8 Wavelength utilities

Function	Purpose	Caution
convert_wavelength_medium(...)	Explicit air/vacuum conversion for wavelength arrays.	Unit conversion and medium conversion are different operations; apply exactly once.
convert_segment_wavelength_medium(...)	Convert a SpectrumSegment while preserving aligned arrays/metadata.	Update provenance/medium state; do not use to “fix” an unknown medium by assumption.

28.9 Results and reporting

Object / method	Purpose	Use
PhoenixFitResult	Structured PHOENIX fit result.	Best-fit parameters, reconstruction, setup/provenance, diagnostics and quality information.
PhoenixFitDiagnostics	Structured numerical/quality diagnostics.	Optimizer attempts, boundaries and other interpretation-relevant information.
result.quality_report()	Machine-readable quality summary.	Programmatic gates and batch triage.
result.quality_report_text()	Human-readable quality summary.	Read alongside residual plots.
result.to_report_dict()	Build report-style mapping.	Useful for application/report integration.
result.save_report_json(path)	Write versioned scientific report envelope.	Preferred reproducible hand-off format.
result.save_json(path)	Write compact backwards-compatible JSON.	Less provenance/context than report envelope.
input_checksum_provenance(...)	Record checksum provenance for pre-loaded input files.	Useful when a GUI/notebook reads the file before fitting.

28.10 Advanced recipes and namespaces

The maintained examples use recipe helpers for specialized workflows, notably the X-SHOOTER Balmer analysis. These should be treated as advanced workflow surfaces rather than as replacements for the core spectrum/fit API. A representative example is:

```
balmer_case = sp.recipes.prepare_xshooter_balmer_case(
    spec,
    window_mode="notebook",
    norm_mode="sideband",
    sideband_width=10.0,
    sideband_order=1,
    core_mask=3.0,
)
```

Use the example/notebook and help() for the exact parameters in the installed revision. Recipe functions encode scientific workflow choices and may evolve more quickly than the core container/read/fit/result interfaces.

28.11 Individual readers

Specialized reader functions exist for several product families, including X-SHOOTER/XSL, PEPSI, FLOYDS, Gemini/GMOS, SDSS/SEGUE, UVES-POP, and Gaia FGK Benchmark Stars. New user code should normally call read_spectrum(..., reader=...) rather than importing a product-specific function directly. Direct reader functions are mainly useful for testing, expert debugging, and cases where their product-specific options are explicitly needed.

28.12 Command-line surface

The current public CLI is setup- and discovery-oriented. The canonical reader-based commands are:

```
spyctres doctor --skip-phoenix
spyctres readers
spyctres reader-info xshooter_merge1d
spyctres inspect-spectrum my_spectrum.fits --reader xshooter_merge1d
```

The compatibility forms `spyctres instruments`, `spyctres instrument-info`, and `inspect-spectrum --instrument ...` remain available, but new documentation should use the reader-oriented names above. There is intentionally no general `spyctres fit` command in the reviewed alpha; scientific fitting remains a Python API workflow so that setup review, masks, readiness, and result semantics remain explicit.

28.13 Stability levels

Level	Examples	How to treat it
Core/recommended	SpectrumSegment, SpectrumCollection, read_spectrum, suggest_fit_setup, fit_stellar_spectrum, plot_fit_referee.	Build new analyses around these concepts.
Advanced/reviewed workflow	recipes, stability comparison helpers, retained-window collection builders, validation plotting.	Useful and supported in examples, but likely to be reorganized internally.
Compatibility	fit_phoenix_spectrum aliases, older readiness names, legacy module exports.	Use to reproduce older work; prefer modern equivalents for new code.
Legacy scientific path	Historical synthetic-photometry/SED utilities in Spyctres.py.	Do not treat as the modern SED implementation; isolate dependencies and assumptions.

Appendix A. Scientific conventions

This appendix is the compact convention sheet for the current Spyctres spectral workflow. When behaviour in a notebook seems surprising, check these conventions before changing code or fit settings.

A.1 Wavelength arrays and units

- Current readers normally expose active wavelengths in Angstrom.
- The numerical unit is separate from wavelength medium. Converting nm to Angstrom does not convert air to vacuum.
- Air/vacuum conversion must be explicit and should occur exactly once.
- Unknown medium should remain unknown rather than be inferred from numerical range or filename.

A.2 Active wavelength frame

Frame	Meaning	RV caution
topocentric	Active wavelengths retain observatory-frame motion.	Fitted shift is not automatically the barycentric/systemic stellar RV.
heliocentric / barycentric	Observer-motion correction is represented in the active wavelength scale according to product semantics.	Do not apply a second correction because a header records its value.
stellar_rest	A stellar-rest transformation has already been applied.	Fitted RV is normally residual alignment, not the original absolute stellar RV.
unknown	Product does not establish the frame safely.	Numerical alignment may be possible, but physical RV interpretation is blocked/ambiguous.

A.3 Radial velocity sign

Spyctres uses the standard astronomical sign convention in the forward model: positive stellar RV corresponds to recession/redshift. The sign convention does not by itself define the physical meaning of the fitted velocity; that still depends on the active data frame and prior corrections.

A.4 Valid mask versus exclusion mask

Mask polarity

SpectrumSegment.mask uses True = usable/valid pixel. An explicitly named exclusion mask uses True = reject. Do not use the two conventions interchangeably.

Finite-data validity, archive/product quality flags, telluric exclusions, DIB exclusions, Balmer-core masks, user-defined regions, and warning-only regions are conceptually distinct. A warning is not automatically an exclusion.

A.5 Uncertainties

- Where present, err is interpreted as 1-sigma standard deviation, not variance and not inverse variance.
- Product readers convert documented inverse-variance/variance representations into this convention where safe.
- Missing uncertainties do not justify invented precision.
- Pipeline uncertainties may omit correlated noise, model inadequacy, calibration uncertainty, and LSF uncertainty; reduced chi-square must be interpreted accordingly.
- Rescaling uncertainties to force reduced chi-square to one is not automatically scientifically justified.

A.6 Resolution and LSF

- Resolution information available in the product, the resolution adopted by the analysis, and the LSF actually applied are separate states.
- The production PHOENIX likelihood currently uses a constant Gaussian LSF when broadening is requested and adequately specified.
- Wavelength-dependent and non-Gaussian LSF information can be preserved as provenance without being applied in the likelihood.

- A nominal slit-based resolving power is not equivalent to a measured exposure-specific LSF.
- LSF undersampling warnings should be investigated rather than suppressed by default.

A.7 Flux state and continuum

Flux state	What can safely be inferred
absolute	Potentially preserves physical flux scale, subject to calibration/slit/aperture uncertainties; the current standard spectral workflow still does not provide the modern angular-radius/SED layer.
relative shape	Broad spectral shape may be meaningful while overall scale is uncertain; calibration nuisance treatment may be required.
continuum normalized	Useful for line morphology; cannot by itself constrain an absolute angular scale.
unknown	Do not infer physical calibration from the numerical flux range.

The standard PHOENIX fitter can solve a per-segment multiplicative Legendre continuum. These coefficients are linear nuisance terms solved inside each nonlinear model evaluation. They should absorb smooth residual calibration shape, not arbitrary line-by-line model failure.

A.8 Multi-segment objective

SpectrumCollection keeps segments distinct. Shared nonlinear stellar parameters can be fitted jointly while each segment retains its own sampling, mask, resolution state, and continuum nuisance. Segment weights are scientifically relevant; a segment with many high-weight pixels can dominate the objective unless the weighting policy is justified.

A.9 Diagnostic windows, masks, and fit regions are different

- Diagnostic window: a potentially informative region covered by the data.
- Valid mask: pixels currently usable according to numerical/product validity.
- Exclusion mask: pixels deliberately rejected for a stated reason.
- Fit region: wavelength interval permitted to contribute to the objective after validity/exclusions are applied.
- Warning region: area needing attention but not automatically rejected.

A.10 Fitting order

1. Interpolate/evaluate the PHOENIX atmosphere spectrum at candidate Teff, log g, and [Fe/H].
2. Apply the candidate stellar RV shift. Positive rv_kms redshifts the model; for $|rv\_kms| << c$, $\lambda\_shifted = \lambda \times (1 + rv\_kms/c)$. The physical meaning of the fitted shift still depends on the active spectrum frame.
3. Apply the specified constant-Gaussian instrumental broadening when appropriate.
4. Resample to each observed pixel grid.
5. Solve the multiplicative Legendre continuum coefficients linearly for each segment.
6. Compute weighted residuals over accepted fit pixels.
7. Use RV/coarse initialization and bounded multistart nonlinear optimization to locate the best numerical solution.
8. Reconstruct the model and retain diagnostics/provenance in PhoenixFitResult.

A.11 Interpretation hierarchy

A numerical fit can be computed even when a particular scientific interpretation is unsafe. Readiness, quality flags, residual inspection, bounded sensitivity tests, and external validation determine what the result can support. Convergence is evidence about the optimizer, not a certificate of model adequacy.

Appendix B. PHOENIX model-library setup

The modern Spyctres spectral fitter uses a local PHOENIX HiRes library. The library is external model data, not a lightweight Python dependency. Correct setup therefore requires both the Python package and a discoverable model tree.

B.1 Expected model-family layout

```
<PHOENIX_ROOT>/
  WAVE_PHOENIX-ACES-AGSS-COND-2011.fits
  Z-0.0/
    lte05000-4.00-0.0.PHOENIX-ACES-AGSS-COND-2011-HiRes.fits
    ...
  Z-1.0/
    ...
```

Spyctres reads the native wavelength grid, scans installed templates, discovers the actual Teff/log g/[Fe/H] axes, and builds interpolation data from that installed subset. The usable parameter domain is therefore the intersection of the supported code path and the templates actually present.

B.2 Verify discovery

```
python scripts/check_spyctres_setup.py --help
python scripts/check_spyctres_setup.py \
  --spectrum examples/data/TOO_Gaia21ccu_SCI_SLIT_FLUX_MERGE1D_UVB.fits \
  --instrument xshooter
```

`spyctres doctor` should establish the status of the core installation and, unless PHOENIX is skipped, the model-library configuration. The lower-level setup checker remains useful for detailed checks that include the resolved PHOENIX root, native wavelength grid, discoverable templates, and representative spectrum ingestion. Use `--help` for the exact options in the checkout being documented.

B.3 Do not assume a path is active merely because it exists

The alpha package resolves PHOENIX through its configuration/path machinery. Multiple checkouts, environments, or older configuration files can point at different roots. Always verify the path reported by the active Spyctres process before running an expensive analysis.

B.4 Cache behaviour

PHOENIX interpolation is expensive, so the backend uses cached interpolation data. Cache provenance is part of the scientific result. If templates, grid axes, code schema, or cache format change, use the supported rebuild/invalidation mechanism rather than manually editing cache files. A result report should preserve a compact model/cache manifest or hash without publishing private absolute paths.

B.5 Grid boundaries

Interpolation is permitted only within supported installed model space. A parameter driven to a PHOENIX boundary can mean that the star lies outside the local grid, that the chosen regions favour an unsupported solution, or that a different model limitation is pushing the optimizer to the edge. Do not silently extrapolate and report the extrapolated value as ordinary output.

B.6 Practical model-domain caution

The current PHOENIX-backed workflow is intended primarily for ordinary stellar spectra within the model family and installed grid. Hotter stars, carbon-rich/peculiar spectra, strong activity, rapid rotation, non-solar abundance patterns, and other regimes can require additional physics or different atmosphere libraries. Structured residuals that persist under sensible numerical settings should be treated as model evidence, not automatically as a search problem.

B.7 PHOENIX setup troubleshooting table

Problem	Likely cause	Action
Wavelength file not found	Wrong root or incomplete PHOENIX download.	Verify root and expected WAVE_PHOENIX file.
No templates discovered	Unexpected directory/filename layout or empty metallicity directories.	Compare with supported HiRes layout and setup-checker output.
Requested star outside grid	Installed subset lacks required Teff/log g/[Fe/H] support.	Install appropriate supported templates or report unsupported regime; do not extrapolate.
Cache mismatch/corruption	Library or cache schema changed.	Rebuild through supported cache mechanism and preserve new manifest.
Fit suddenly changes after grid update	Different installed model support/cache state.	Compare model manifests/axes and rerun validation fixtures before interpreting changes.

Appendix F. Reproducibility checklist

Use this checklist when a fit is intended to leave the exploratory notebook and become a scientific result, collaborator hand-off, validation artifact, or paper method.

1. Identify the exact input spectrum: product family/reader, observation or library release identifier, and file checksum when appropriate.
2. Record wavelength unit, air/vacuum medium, active frame, and whether observer-motion or stellar-rest corrections were already applied.
3. Record uncertainty provenance and any error floors, scales, replacements, or caveats.
4. Record valid-mask provenance and every scientifically motivated exclusion: archive flags, user regions, tellurics, DIBs, Balmer cores, artifacts, etc.
5. Record the distinction between warning-only regions and excluded pixels.
6. Record available resolution metadata, the resolution/LSF adopted for the fit, and the broadening actually applied.
7. Record flux-calibration/normalization state and any continuum normalization performed before fitting.
8. Preserve the exact FitSetup: regions, Teff/log g/[Fe/H]/RV bounds and initial values, search mode/budget, continuum degree, segment weights, and stable setup hash.
9. Record the PHOENIX model family, installed subgrid/cache manifest or equivalent compact provenance.
10. Record Spyctres version and Git commit for development analyses; record important environment/package versions when exact reruns matter.
11. Preserve the best-fit result together with optimizer attempts, boundary diagnostics, quality flags, readiness status, and uncertainty caveats.
12. Save diagnostic figures showing data/model/residual behaviour in the fitted features, not only a scalar objective.
13. Preserve bounded sensitivity variants that materially support the conclusion: continuum, mask/core, line selection, resolution, or other justified assumptions.
14. Preserve external benchmark/cross-instrument evidence relevant to the claimed precision or validity domain.
15. Use the versioned report JSON for scientific hand-off; use compact legacy JSON only when compatibility requires it.
16. Ensure generated artifact paths are portable/sanitized and point to the correct result variant.
17. Write a methods summary that states both what was done and what limitations remain.

Two kinds of reproducibility

Re-running the same deterministic setup and obtaining the same optimum is computational reproducibility. Showing that the conclusion survives reasonable analysis variants and external data is scientific robustness. Serious results need both.

F.1 Minimal methods statement template

A concise paper/report description should normally identify: input product and reader; wavelength medium/frame; uncertainty and mask policy; fit windows; adopted resolution/LSF; PHOENIX grid/subgrid; continuum treatment; fit bounds/search strategy; readiness/quality caveats; sensitivity tests; and relevant external validation. The exact setup hash and versioned report can then carry the machine-readable detail.

Appendix H. Legacy and compatibility functionality

Spyctres retains historical functionality so that older analyses can still be reproduced while the modern spectral workflow evolves. This compatibility layer should be treated as a separate scientific path, not as a second equally recommended interface for new work.

H.1 Where the legacy code lives

The reviewed source retains `Spyctres/Spyctres.py` as a legacy public/compatibility module and also retains an older copy in `Spyctres/Spyctres_old.py`. The legacy module includes older fitting and synthetic-photometry/SED functionality and has substantially weaker user-facing documentation than the modern API. The structural plan explicitly aims to isolate it while preserving compatibility where practical.

H.2 Why the dependencies differ

Path	Typical dependencies	Status
Modern PHOENIX spectral workflow	numpy, scipy, matplotlib, astropy, speclite plus local PHOENIX model data.	Recommended current path.
Legacy synthetic-photometry / SED workflow	pysynphot, synphot, stsynphot and current alpha packaging constraints around setuptools/pkg_resources.	Compatibility only; planned to become a separate optional extra.

A failure in a legacy photometry dependency is therefore not evidence that `fit_stellar_spectrum()` or the PHOENIX forward model is scientifically broken. Diagnose the workflow layer separately.

H.3 Confirmed compatibility example

Historical functions such as `fit_spectra_chichi` remain available through the legacy module in the reviewed development environment. Older notebooks/scripts can therefore still be inspected or reproduced. New analyses should prefer the modern path because it has explicit spectrum containers, frame/mask/resolution semantics, first-class FitSetup objects, structured results, quality reporting, and versioned provenance.

```
# Compatibility check only - not the recommended new workflow
from Spyctres import Spyctres as legacy
print(hasattr(legacy, "fit_spectra_chichi"))
```

H.4 What not to assume about the old SED path

- Do not describe the historical SED/synthetic-photometry code as the planned modern microlensing-source SED layer.
- Do not assume old photometric scaling parameters have the same semantics as the future explicit angular-radius/extinction/source-blend model.
- Do not mix modern spectral FitSetup/readiness/report semantics into an old script without checking how the legacy function actually constructs its objective.
- Do not import legacy functionality merely because an old example did so if the current task is ordinary spectral fitting.

H.5 Migration strategy for an old analysis

1. Freeze the old environment, input data, configuration and expected output before modifying the script.
2. Identify which steps are data ingestion, local diagnostics, atmospheric fitting, synthetic photometry, or microlensing/SED-specific logic.
3. Replace spectral ingestion first with `read_spectrum()` and explicit metadata; verify wavelengths, masks and uncertainties are unchanged scientifically.
4. Replace old PHOENIX fitting calls with `suggest_fit_setup()` plus `fit_stellar_spectrum()` only after matching the old fitted regions, continuum assumptions, resolution and RV convention.
5. Keep old SED/synthetic-photometry calculations isolated until a modern supported replacement exists; do not silently reinterpret legacy scale parameters.
6. Compare old and new model spectra/residuals, not only final parameter values.
7. Record any non-zero numerical change and determine whether it comes from a corrected bug, changed convention, different model grid/cache, or a deliberate scientific change.

H.6 When compatibility should win over modernization

For reproducing a published or archival result, preserving the original computation can be more important than immediately translating it into the modern API. In that case, run the legacy code in an isolated environment, record the dependency/model state, and treat the result as a historical reproduction. Build the modern reanalysis separately so that differences remain auditable.

H.7 When modernization should win

For new spectral analyses, use the modern containers/readers, explicit metadata, reviewed FitSetup, readiness gates, PHOENIX forward model, structured results, stability tests, and versioned reports. The purpose of the compatibility layer is to keep old work reproducible, not to define the design of new work.

Key scientific reference

PHOENIX HiRes library: Husser, T.-O. et al. 2013, "A new extensive library of PHOENIX stellar atmospheres and synthetic spectra", Astronomy & Astrophysics, 553, A6. DOI: 10.1051/0004-6361/201219058.